twn_fuse Linux driver 開發講義

從 UIO 到 kernel driver：probe、DMA、ioctl、中斷、記憶體隔離與發佈（v1.1）

Wei-Jhe Huang

2026 年 10 月 27 日

目錄

導讀 **2**

1 Device tree、probe 與 binding：kernel 怎麼找到 driver **7**

2 MMIO：ioremap、readl / writel 與 barrier **15**

3 DMA buffer：配置、cache 與記憶體屬性 **20**

4 mmap：只把 buffer 給 user space **26**

5 file operations 與 ioctl：控制介面、copy_from_user 與 offset **30**

6 等待完成：從輪詢到中斷 **37**

7 並行控制：兩把 mutex 與獨佔開啟 **48**

8 看得見硬體：sysfs 與 debugfs **52**

9 App 端：同一份 HAL 介面，以及影像怎麼進 buffer **58**

10 隔離之一：限制 /dev/mem、拿掉保留區 **62**

11 隔離之二：SMMU 限制 PL 能碰的記憶體 **67**

12 改了 RTL 就要重建：timing closure **74**

13 板上驗證與靜態檢查：證明了什麼、沒證明什麼 **78**

14 進映像與發佈：recipe、預設密碼、合成 fixture **83**

A 名詞表 **92**

B 面試題：30 秒版本 **95**

C 如何重現 **97**

導讀

這份講義在講什麼

這是 `twn_fuse` driver 的開發筆記。這個 Linux kernel driver 管理 FPGA 上的三值（ternary，權重只有 -1、0、$+1$）MobileNetV2 加速器，跑在 ALINX AXU2CGB 開發板（Zynq UltraScale+ XCZU2CG）上。專案經過三代：

版本	程式碼	做法
v1.0.0	commit `3bdd485`	**UIO 加上 `/dev/mem`**：控制暫存器和 DDR 都直接開放給 app，app 自己寫暫存器、自己等硬體做完；device tree 為它保留 256 MB
driver v1.0	commit `7615324`	最早的 kernel driver：kernel 管暫存器、DMA buffer 和操作順序，app 只能用 `ioctl` 與 `mmap`；但只能輪詢、要手動 `insmod`，和 UIO、`/dev/mem` 並存，PL 的 DMA 沒有隔離
v1.1（發佈）	tag `v1.1.0`（之後只改了文件、註解與測試腳本）	補上中斷、開機自動載入、限制 `/dev/mem`、SMMU、debugfs、binding，並發佈可以直接開機的映像

講義只講現在的 v1.1：每一章的程式碼都是現行版本；舊版本只出現在「原本怎麼做」的框裡，用來說明改了什麼。driver v1.0 這個中間版本，只在它和 v1.1 不同、而且差別有教學價值的地方出現（例如輪詢改中斷、除錯函式失效）。

怎麼讀

每一章都有這八個部分，順序不變；實際例子較長的章拆成幾個小節：

一句話	讀完這一句，就知道這章在講什麼
原本怎麼做	舊的寫法，以及問題出在哪裡（橘色框）；框的標題寫明是 v1.0.0 還是 driver v1.0
用了什麼技術	這一章用到的 kernel API 與機制（藍色框）
改善了什麼	具體、可以在板上驗證的改變；沒在板上驗證的會註明（綠色框）
實際例子	實際的程式碼，加上板上的實際輸出（黑底）
原理	為什麼這樣做是對的；引用 Linux 6.1 的原始碼
面試會問	約 1 分鐘的完整回答（紫色框）；30 秒版本在附錄 B
關鍵函式速查	名稱、在哪個檔案第幾行、做什麼

有些章另外有紅色框的「踩過的坑」，記錄開發時真的出過的錯。

程式碼都是真的。每段程式碼都由 `extract.py` 從 repo 的實際檔案抽出，框上方是檔名和真實行號，不是手抄的。舊版本的程式碼直接從 git 取出，框上方的檔名後面標著 `@ v1.0.0 (3bdd485)` 或 `@ driver v1.0 (7615324)`；沒有這個標記的就是現行版本。

紀錄也是真的。黑框直接從 `results/raw/` 底下的紀錄檔讀出原文，框上方是檔名、行號與開機組，框左邊的灰色數字是紀錄檔的行號。行首是 `#` 加一個空格的那一行，在板上紀錄裡是執行的指令；例外是 S 檔與 H1、H2、H9 裡的幾行說明文字（例如實際按了哪些鍵），以及 D5 第 37 行 `# CONFIG_STRICT_DEVMEM is not set`（kernel 設定檔本身的內容）。板上紀錄的時間是 UTC（台灣時間減 8 小時）；大多數紀錄的第 1 行是日期與開機檔的雜湊。雜湊都是 SHA-256 的前 16 個十六進位字元。紀錄依「開機組」(一組開機檔，可以開機很多次）分類，檔名的第一個字母就是開機組：

代號	開機檔與用途
A、B、C	v1.0.0 的開機檔（`BOOT.BIN` `690d87cc5f4b6897`），手動載入 driver v1.0（`8b3649080da3254d`）。A 是原本的 `system.dtb`，B 把節點的 `compatible` 改成 driver 的字串，C 是回退。紀錄在 `results/raw/linux_driver_20261026/`；這份講義只用它們呈現 v1.0.0 的狀態，以及和 driver v1.0 比較
D	v1.0.0 的開機檔（有 UIO 與 `/dev/mem`、沒有中斷線），手動載入 v1.1 的 driver 程式，只能輪詢。A/B 實驗、CPU 存取 buffer 的速度、pipeline 各階段的時間、v1.0.0 開機時的狀態都在這裡量
E	發佈的開機檔（`BOOT.BIN` `5d226057efc8760b`、`system.dtb` `bc4255b1c54ae27e`：有中斷、SMMU 打開），搭配發佈前的候選版 rootfs（`8c54689ab21fe45c`）；它的 driver 和發佈版只差兩個 probe 失敗路徑，改密碼的方式也不同（第 14 章）
F	找 SMMU stream ID 用的診斷組：最終的 bitstream 另一次打包的 `BOOT.BIN`（`18a066f87cfbad8d`），搭配第一次建出的 v1.1 kernel 與 rootfs，開機參數多一個 `arm_smmu.disable_bypass=1`；`system.dtb` 在 F0 只登記推算的三個 stream ID（`4889daa18dd6a320`），在 F1 登記量到的四個（`1c39e2e7a20bd5a1`）(第 11 章）
G	開機組 E，只把 `system.dtb` 換成關掉 SMMU 的版本
H	**發佈的 v1.1 映像**：開機檔與 E 相同，rootfs `96b9e56733ad36a9`，板上工具包 `3a4eb0997028ba0e`
I	開機組 H，只把 `system.dtb` 換成關掉 SMMU 的版本（`6ceadb90b64b6a98`）
S	PC 上的模擬、檢查與 bitstream 建置

D 到 I 與 S 的紀錄在 `results/raw/linux_driver_v11_20261027/`；那裡的 `first_bitstream/` 子目錄是第一版 v1.1 bitstream 的紀錄，因為 SMMU 的問題（第 11 章）而被取代。

先認識幾個名字

PS / PL	Zynq 晶片的兩半。PS（Processing System）是 Cortex-A53 CPU，跑 Linux；PL（Programmable Logic）是 FPGA 邏輯，放加速器
AXI、beat	AXI 是 ARM 定義的晶片內部匯流排。PL 存取 DDR 時一次搬一個 beat；這個設計的資料寬度是 128 bits，所以一個 beat 是 16 bytes
HP 埠	PL 存取 DDR 的高效能 AXI 埠，加速器用四個（HP0 到 HP3）
CSR	Control/Status Registers，加速器的控制暫存器，共 64 KB，實體位址 `0xA000_0000`
arena、buffer	PL 讀寫的那塊 DDR：權重、輸入影像、中間結果、輸出都放在這裡。v1.0.0 叫 arena（256 MB 保留區），driver 版本是 driver 配的 8 MB DMA buffer
backbone / head	PL 裡兩個獨立的運算引擎。backbone 跑網路主體的 17 個 block；head 跑最後四個 operator（運算層），輸出 heatmap（每個關節一張的位置機率圖）。兩者各有自己的 start、busy、done 位元
描述子表	descriptor table：17 個 block 各 16 個 32-bit word 的設定（形狀、權重位置、輸入輸出位置），由 app 提供、經 driver 寫進 CSR，PL 依表執行
NBLK	「這一幀要跑幾個 block」的暫存器，平常是 17
BB2HD	一個模式：backbone 的最後一個 block 直接把結果寫進 head 的輸入區，省掉 CPU 搬資料
fixture	四張測試輸入，各附一份 golden（標準答案）輸出
pipeline	量吞吐量用的 app `mbv2_fuse_pipe`：一個 producer thread 驅動 PL，讓 backbone 跑第 k 幀時 head 同時跑第 $k-1$ 幀（第 7 章）；`mbv2_fuse_test` 則依序跑 4 張 fixture，每張跑完就逐位元比對
`FUSE_HEAD_PL`、`FUSE_BB2HD`	測試程式的兩個環境變數：`FUSE_HEAD_PL=1` 時 head 在 PL 上跑，`=0` 時在 CPU 上跑；`FUSE_BB2HD=1` 時開啟 BB2HD 模式
bit-exact	逐位元相同：輸出和 golden 的每一個 byte 都一樣，最大差值是 0
`twn_fuse`	加速器的 IP 名稱：TWN 是 Ternary Weight Network（三值權重網路），fuse 是「把一個 block 的三層融合成一次串流」

整體架構：改了什麼

圖 1 左邊是 v1.0.0，右邊是 v1.1。差別在誰碰硬體、PL 能碰到哪裡：左邊是 app 透過兩個裝置檔自己去碰暫存器和實體記憶體，PL 拿實體位址存取 DDR；右邊只有 driver 碰暫存器，app 只映射得到 driver 配的 buffer，PL 的存取經過 SMMU。

右圖的「只有 driver」有邊界：`/dev/mem` 仍能映射「不是 System RAM、也沒有 driver 佔用」的位址，例如 root 卸載 driver 之後的加速器暫存器（第 10 章）；SMMU 擋不住 Secure 交易與沒登記的 stream ID（第 11 章）。

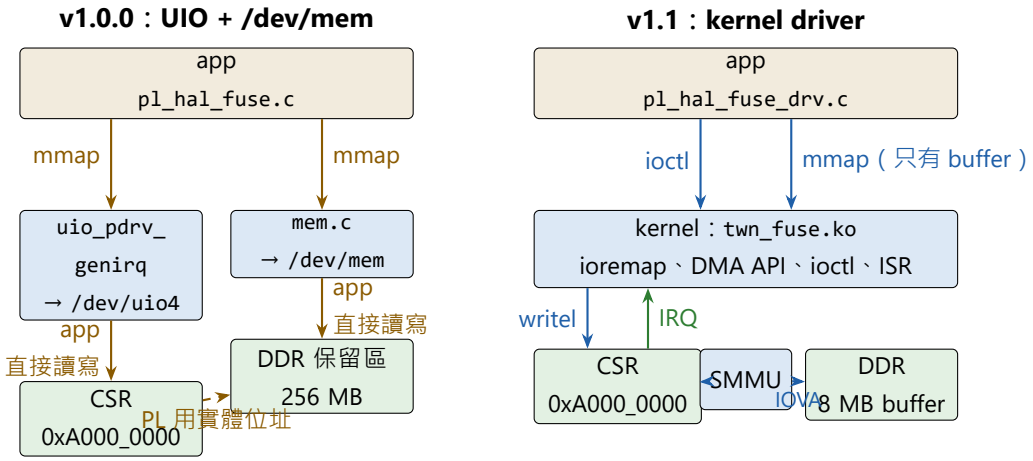


圖 1: 誰碰硬體、PL 能碰到哪裡。左：app 經兩個裝置檔，自己映射暫存器與整塊保留區，PL 拿實體位址存取 DDR。右：只有 driver 碰暫存器，硬體做完用中斷通知；app 只映射得到 driver 配給它的 buffer；PL 的存取經 SMMU 轉換，只到得了這塊 buffer。

一幀在 v1.1 裡怎麼走

1. app 把輸入影像寫進 `mmap` 得到的 buffer（第 4、9 章）。
2. app 呼叫 `ioctl(fd, TWN_IOC_BB_RUN, &r)`，`r` 裡只有 offset：輸入在 buffer 的哪裡、輸出要寫到哪裡（第 5 章）。
3. driver 先拿 backbone 的 mutex（第 7 章），再用 `copy_from_user` 把 `r` 複製進來，檢查 offset 在 buffer 範圍內（第 5 章）。
4. driver 等 backbone 閒下來、清掉上一次的 done、重設 completion，把「buffer 的 DMA 位址 + offset」寫進位址暫存器，再寫 START；`writel` 前面的 barrier 保證剛寫的影像比 START 先被 PL 看到（第 2、6 章）。
5. driver 在 completion 上睡，最多等 3 秒（第 6 章）。
6. PL 經 SMMU 讀寫 buffer（第 11 章）；做完拉起中斷，ISR 清掉 done、喚醒 driver（第 6 章）。
7. driver 讀六個 cycle counter，用 `copy_to_user` 交回 app（第 5、8 章）。
8. app 直接從 buffer 讀輸出。

結果總覽

發佈映像（開機組 H）上量到的，比較的對象寫在括號裡：

項目	結果	章
4 張 fixture，head 在 PL / 在 CPU	4/4、4/4 bit-exact	13
錯誤路徑（14 項：12 項該拒絕、2 項該成功）	14/14 符合預期	5、13
pipeline 吞吐量（120 幀，3 次平均）	232.27 fps	13
每幀 CPU 時間（中斷，對照輪詢，開機組 E）	10.7 µs 對 465.8 µs	6
吞吐量（中斷，對照同一次開機的輪詢）	233.13 對 231.26 fps	6
CPU 寫 buffer（Normal non-cacheable，對照 Device 映射，開機組 D）	寫輸入 0.09–0.10 對 0.14–0.15 ms（每次執行的最後一幀）	3
kernel 可用記憶體（對照 v1.0.0）	多約 255.5 MB	10
`/dev/mem` 讀 System RAM、driver 的暫存器	拒絕	10
PL 越界讀取（SMMU 開，對照關）	context fault（關掉時沒有察覺）	11
bitstream timing（240 MHz）	WNS 0.000 ns、WHS +0.008 ns	12

同一個設定的吞吐量，不同次執行之間會差到約 1 fps（例如上表兩列都是同一次開機、中斷模式，平均差 0.86 fps；原因沒有查）。所以中斷與輪詢只拿同一次開機、交替執行的三輪來比（第 6 章）。

章節

章	內容
1–4	基礎：kernel 怎麼找到 driver（probe、binding）、怎麼碰暫存器（MMIO、barrier）、DMA buffer 與記憶體屬性、把 buffer 交給 app（mmap）
5–7	介面與同步：ioctl、中斷、鎖
8–9	周邊：sysfs 與 debugfs、app 端的 HAL 與攝影機輸入
10–11	隔離：限制 `/dev/mem`、SMMU
12–14	工程：timing closure、驗證與靜態檢查、進映像與發佈

全部讀完大約 8 到 10 小時。第 1 到 6 章是核心，面試最常問；第 10、11 章是安全性；第 12 章偏硬體，只關心 driver 可以跳過。

1 Device tree、probe 與 binding：kernel 怎麼找到 driver

kernel 拿 device tree 節點的 `compatible` 字串去比對每個 driver 的比對表，對上了就呼叫那個 driver 的 `probe()` 接管硬體；binding 則把「這個節點該長什麼樣」寫成可以檢查的規格。

原本怎麼做（v1.0.0 的 UIO 版本）

PetaLinux 從 Vivado 匯出的硬體描述自動產生節點。v1.0.0 在 `system-user.dtsi` 把它的 `compatible` 改寫成 `"generic-uio"`：

linux/project-spec/meta-user/recipes-bsp/device-tree/files/system-user.dtsi @ v1.0.0 (3bdd485) : 18--21

```
&fuse_0 {
    compatible = "generic-uio";
    status = "okay";
};
```

開機參數 `uio_pdrv_genirq.of_id=generic-uio` 再讓 UIO driver 認這個字串。UIO driver 不知道這是什麼硬體，只把節點的位址範圍開成 `/dev/uio4`；app 自己去 `/sys/class/uio/` 找名字對的編號，再 `mmap` 進來：

sw/app/pl_hal_fuse.c @ v1.0.0 (3bdd485) : 115--136

```
void fuse_hal_init(void) {
    char path[64];
    const char *uio = getenv("PL_UIO");
    if (!uio) {
        if      (find_uio_by_name("twn_fuse",     path, sizeof path) == 0) uio = path;  /* DT node of the fused design */
        else if (find_uio_by_name("fuse_0",       path, sizeof path) == 0) uio = path;  /* its BD cell name */
        else if (find_uio_by_name("twn_fuse_top", path, sizeof path) == 0) uio = path;
        else if (find_uio_by_name("mbv2_fuse",    path, sizeof path) == 0) uio = path;
        else if (find_uio_by_name("twn_top",      path, sizeof path) == 0) uio = path;
        else {
            fprintf(stderr, "fuse_hal: no UIO named 'twn_fuse'/'fuse_0'/'twn_fuse_top'/'mbv2_fuse'/'twn_top'\n");
            exit(1);       /* no fallback to uio0 */
        }
    }
    fprintf(stderr, "fuse_hal: control UIO = %s\n", uio);
    int fd = open(uio, O_RDWR | O_SYNC);
    if (fd < 0) { perror("open uio"); exit(1); }
    g_ctrl = (volatile uint32_t *)mmap(NULL, CTRL_MAP_SZ, PROT_READ | PROT_WRITE,
                                       MAP_SHARED, fd, 0);
    if (g_ctrl == MAP_FAILED) { perror("mmap uio"); exit(1); }
    close(fd);
}
```

問題：`"generic-uio"` 描述的是「要交給哪個 driver」，不是「這是什麼硬體」，而 device tree 只該描述硬體；要靠開機參數配合；kernel 對這顆硬體一無所知，不檢查、不配置資源，也不知道誰在用它。節點該有哪些屬性，也沒有寫在任何地方。

用了什麼技術

- `struct platform_driver`：SoC（把 CPU 與周邊做在同一顆晶片上）周邊的 driver 型別。這類周邊不在 PCI、USB 這種能自動列舉的匯流排上，要靠 device tree 告訴 kernel 它們在哪裡。
- `struct of_device_id` 比對表（`of_match_table`）與 `MODULE_DEVICE_TABLE(of, ...)`：列出認得的 `compatible` 字串，並寫進 module 的資訊，讓系統知道哪個 module 該被自動載入（第 14 章）。
- `probe()` / `remove()` 與 `devm_*`（device-managed）系列：資源跟著裝置走，driver 離開或 probe 失敗時自動釋放。
- **binding**：用 dt-schema 格式（JSON Schema 寫成的 YAML）描述節點，`wjhuang,twn-fuse.yaml`。

改善了什麼

- 硬體由自己的名字 `"wjhuang,twn-fuse-1.1"` 認領；字串尾端的版本號讓有中斷的新硬體和沒有中斷的舊硬體各自對到正確的處理方式。
- `probe()` 時就檢查硬體、配置記憶體、申請中斷、建立 `/dev/twn_fuse`，結果寫進 `dmesg`，不必等 app 跑起來才發現問題。
- 節點的規格寫成檔案，並拿 PetaLinux 實際產生的 `system.dtb` 來比對；比對時發現 PetaLinux 多產生了 `interrupt-names`，事先沒有人知道。

1.1 實際例子

比對表和 driver 宣告。比對表有兩筆：`-1.1` 是有中斷的硬體（第 6 章），`-1.0` 是 v1.0.0 那顆沒有中斷的硬體，最後一筆空的表示結束。`struct platform_driver` 把 probe、remove、比對表與 sysfs 屬性（第 8 章）綁在一起；`suppress_bind_attrs` 的理由在第 7 章：

linux/driver/twn_fuse/twn_fuse.c : 666--670

```
666 static const struct of_device_id twn_of_match[] = {
667     { .compatible = "wjhuang,twn-fuse-1.1" },       /* with the interrupt */
668     { .compatible = "wjhuang,twn-fuse-1.0" },
669     { }
670 };
```

linux/driver/twn_fuse/twn_fuse.c : 682--696

```
682 static struct platform_driver twn_driver = {
683     .probe  = twn_probe,
684     .remove = twn_remove,
685     .driver = {
686         .name           = "twn_fuse",
687         .of_match_table = twn_of_match,
688         .dev_groups     = twn_groups,
689         /*
690          * No sysfs "unbind": it would free the buffer under a process that
691          * still has it mapped. Unloading goes through rmmod, which the
692          * kernel refuses while the device is open or mapped.
693          */
694         .suppress_bind_attrs = true,
695     },
696 };
```

最後一行 `module_platform_driver(twn_driver);` 展開後就是 module 的 `init` 與 `exit`：載入時 `platform_driver_register()`，卸載時 `platform_driver_unregister()`。

probe：接管硬體的五個步驟。`probe()` 收到一個 `struct platform_device`，裡面已經有由 `reg` 轉成的位址範圍，以及指向 device tree 節點的指標（中斷要等第 4 步才從節點解析出來）。每一步失敗就回傳錯誤碼，kernel 放棄綁定並收回 `devm_` 配過的東西：

linux/driver/twn_fuse/twn_fuse.c : 578--649

```
578 static int twn_probe(struct platform_device *pdev)
579 {
580     struct device *dev = &pdev->dev;
581     struct twn_dev *td; /* 硬體的所有狀態: register 位址、buffer、mutex、interrupt 編號、/dev/twn_fuse
        */
582     u32 nblk;
583     int ret;
584 
585     td = devm_kzalloc(dev, sizeof(*td), GFP_KERNEL);
586     if (!td)
587         return -ENOMEM;
588     td->dev = dev;
589 
590     /* 1. map the node's "reg" range: 64 KB of registers at 0xA000_0000 */
591     td->regs = devm_platform_ioremap_resource(pdev, 0);
592     if (IS_ERR(td->regs))
593         return PTR_ERR(td->regs);
594 
595     /* 2. is this our hardware? (a debug run can leave NBLK changed, so only warn) */
596     nblk = rd(td, R_NBLK);
597     if (nblk != NBLK_RESET)
598         dev_warn(dev, "NBLK reads %u, expected %u: another bitstream, or a debug run changed it\n",
599              nblk, NBLK_RESET);
600 
601     /* 3. the DMA buffer; the PL's address registers are 40 bits wide */
```

```
602     ret = dma_set_mask_and_coherent(dev, DMA_BIT_MASK(40));
603     if (ret)
604         return ret;
605     td->arena_size = (size_t)arena_mb << 20;
606     td->arena = dmam_alloc_coherent(dev, td->arena_size, &td->arena_dma, GFP_KERNEL);
607     if (!td->arena)
608         return -ENOMEM;
609     /* the app lays out 64 KB-aligned regions (head input slots 64 KB apart) */
610     if (td->arena_dma & (SZ_64K - 1)) {
611         dev_err(dev, "DMA buffer at %pad is not 64 KB aligned\n", &td->arena_dma);
612         return -EINVAL;
613     }
614 
615     mutex_init(&td->bb_lock);
616     mutex_init(&td->hd_lock);
617     init_completion(&td->bb_done);
618     init_completion(&td->hd_done);
619     platform_set_drvdata(pdev, td);
620 
621     /* 4. the interrupt, if the device tree gives one (hardware 1.1); else poll */
622     td->irq = use_irq ? platform_get_irq_optional(pdev, 0) : -ENXIO;
623     if (td->irq == -EPROBE_DEFER)   /* the interrupt controller is not ready yet */
624         return td->irq;
625     if (td->irq > 0) {
626         ret = devm_request_irq(dev, td->irq, twn_isr, 0, dev_name(dev), td);
627         if (ret)
628             return ret;
629         wr(td, R_IRQ, IRQ_BB | IRQ_HD);
630     }
631 
632     /* 5. the character device /dev/twn_fuse, last: from here on it can be opened */
633     td->misc.minor  = MISC_DYNAMIC_MINOR;
634     td->misc.name   = "twn_fuse";
635     td->misc.fops   = &twn_fops;
636     td->misc.parent = dev;
637     ret = misc_register(&td->misc);
638     if (ret) {
639         if (td->irq > 0)
640             wr(td, R_IRQ, 0);
641         return ret;
642     }
643 
644     twn_debugfs_init(td);
645     dev_info(dev, "NBLK %u, %zu KB DMA buffer at %pad, /dev/%s, %s\n",
646          nblk, td->arena_size >> 10, &td->arena_dma, td->misc.name,
647          td->irq > 0 ? "interrupt" : "polling");
648     return 0;
649 }
```

第 1 步把暫存器映射進 kernel、第 2 步讀 NBLK 確認是這顆硬體（第 2 章），第 3 步配 DMA buffer（第

3 章），第 4 步申請中斷（第 6 章），第 5 步註冊 /dev/twn_fuse（第 5 章），最後建立 debugfs（第 8 章）。devm_kzalloc 配的 struct twn_dev 用 platform_set_drvdata 掛在裝置上，之後 remove() 與 sysfs 用 platform_get_drvdata / dev_get_drvdata 拿回來。

remove：移除 debugfs 與裝置檔、關掉中斷致能，其餘交給 devm_。

linux/driver/twn_fuse/twn_fuse.c : 651--664

```
651 static int twn_remove(struct platform_device *pdev)
652 {
653     struct twn_dev *td = platform_get_drvdata(pdev);
654 
655     /*
656      * Everything else (mapping, buffer, interrupt, memory) was allocated with
657      * devm_ and is released by the driver core; stop the interrupt source first.
658      */
659     debugfs_remove_recursive(td->dbg);
660     misc_deregister(&td->misc);
661     if (td->irq > 0)
662         wr(td, R_IRQ, 0);
663     return 0;
664 }
```

板上：module 帶著比對字串與參數。modinfo 印出的 alias 就是 MODULE_DEVICE_TABLE 產生的，兩個版本各兩筆；parm 是三個 module parameter：

results/raw/linux_driver_v11_20261027/H10_driver_state.log : 1-11（開機組 H）

```
1  == 2026-10-27 03:01 UTC  6.1.30-xilinx-v2023.2  BOOT.BIN 5d226057efc8760b  system.dtb bc4255b1c54ae27e  twn_fuse.ko
       aec4477e2c520faf
2  # modinfo /lib/modules/6.1.30-xilinx-v2023.2/extra/twn_fuse.ko | grep -E '^(license|alias|vermagic|parm)'
3  license:        Dual MIT/GPL
4  alias:          of:N*T*Cwjhuang,twn-fuse-1.0C*
5  alias:          of:N*T*Cwjhuang,twn-fuse-1.0
6  alias:          of:N*T*Cwjhuang,twn-fuse-1.1C*
7  alias:          of:N*T*Cwjhuang,twn-fuse-1.1
8  vermagic:       6.1.30-xilinx-v2023.2 SMP mod_unload aarch64
9  parm:           arena_mb:size of the DMA buffer in MB (default 8) (uint)
10 parm:           use_irq:wait on the interrupt when the device tree gives one (default 1); 0 polls (bool)
11 parm:           map_device:map the buffer to user space as Device memory, like /dev/mem (A/B measurement only) (bool)
```

板上：開機後已經綁好。發佈映像的 system-user.dtsi 把節點改成 driver 認得的字串（iommus 是第 11 章的 SMMU）：

linux/project-spec/meta-user/recipes-bsp/device-tree/files/system-user.dtsi : 20--24

```
20 &fuse_0 {
21     compatible = "wjhuang,twn-fuse-1.1";
22     status = "okay";
23     iommus = <&smmu 0xe80>, <&smmu 0x12c0>, <&smmu 0x1300>, <&smmu 0x1740>;
24 };
```

開機第 4.8 秒 probe 成功，裝置的 `driver` 連結指向 `twn_fuse`。開機參數裡還留著 `uio_pdrv_genirq.of_id=generic-uio`，但已經沒有節點寫 `generic-uio`，UIO 不會接管任何東西：

results/raw/linux_driver_v11_20261027/H3_boot_state.log : 1-13（開機組 H）

```
== 2026-10-26 20:39 UTC  6.1.30-xilinx-v2023.2  BOOT.BIN 5d226057efc8760b  system.dtb bc4255b1c54ae27e  twn_fuse.ko
     aec4477e2c520faf
# cat /proc/cmdline
earlycon console=ttyPS0,115200 clk_ignore_unused root=/dev/ram0 rw uio_pdrv_genirq.of_id=generic-uio
# lsmod | grep twn_fuse
twn_fuse               20480  0
# modinfo -n twn_fuse; grep -h twn_fuse /etc/modules-load.d/* 2>/dev/null
/lib/modules/6.1.30-xilinx-v2023.2/extra/twn_fuse.ko
twn_fuse
# readlink /sys/bus/platform/devices/a0000000.twn_fuse/driver
../../../../bus/platform/drivers/twn_fuse
# dmesg | grep 'twn_fuse a0000000'
[    4.795531] twn_fuse a0000000.twn_fuse: Adding to iommu group 15
[    4.819475] twn_fuse a0000000.twn_fuse: NBLK 17, 8192 KB DMA buffer at 0x000000ffff800000, /dev/twn_fuse, interrupt
```

binding：節點的規格。一定要有 `compatible` 和 `reg`；用 `if / then / else` 規定 `-1.1` 一定要有中斷、`-1.0` 不准有；`additionalProperties: false` 表示沒列出的屬性一律不准：

linux/driver/twn_fuse/wjhuang,twn-fuse.yaml : 52--70

```
required:
  - compatible
  - reg

allOf:
  - if:
      properties:
        compatible:
          contains:
            const: wjhuang,twn-fuse-1.1
    then:
      required:
        - interrupts
    else:
      properties:
        interrupts: false
        interrupt-names: false

additionalProperties: false
```

`test/binding_check.py` 把 PetaLinux 產生的 `system.dtb` 反組譯，找出加速器的節點，逐一檢查屬性都有被描述，也檢查 binding 的 `compatible` 和 driver 的比對表一致：

results/raw/linux_driver_v11_20261027/S3_binding_check.log : 1-3,23-45 (PC): 節點原文與每一項檢查

```
== 2026-10-26 19:14 UTC  host (WSL Ubuntu 22.04)  system.dtb bc4255b1c54ae27e (PetaLinux, XSA 253ca13d2e977247)
# python3 linux/driver/twn_fuse/test/binding_check.py <kernel-source> build/linux_v11/sd/system.dtb
PASS binding has $id
PASS system.dtb: one node with a wjhuang,twn-fuse compatible
     node twn_fuse@a0000000:
       clock-names = "ap_clk";
       clocks = <0x04 0x47>;
       compatible = "wjhuang,twn-fuse-1.1";
       interrupt-names = "interrupt";
       interrupt-parent = <0x05>;
       interrupts = <0x00 0x59 0x04>;
       reg = <0x00 0xa0000000 0x00 0x10000>;
       status = "okay";
       iommus = <0x11 0xe80 0x11 0x12c0 0x11 0x1300 0x11 0x1740>;
       phandle = <0x72>;
PASS dtb node: property clock-names is described
PASS dtb node: property clocks is described
PASS dtb node: property compatible is described
PASS dtb node: property interrupt-names is described
PASS dtb node: property interrupts is described
PASS dtb node: property iommus is described
PASS dtb node: property reg is described
PASS dtb node: has required property compatible
PASS dtb node: has required property reg
PASS dtb node: version 1.1 has an interrupt
BINDING CHECK: PASS
```

1.2 原理

從開機到 probe。U-Boot (開機載入程式) 把 `system.dtb` 放進記憶體交給 kernel；kernel 把它展開成一棵節點樹，`amba_pl` 底下每個節點建成一個 `platform_device`，名字是「位址.節點名」(`a0000000.twn_fuse`)，`reg` 變成記憶體資源。`interrupts` 這時還留在節點上：driver 呼叫 `platform_get_irq_optional()` 時，kernel 才用 `of_irq_get()` 把它翻成 Linux 的中斷號碼；中斷控制器還沒準備好，就回 `-EPROBE_DEFER` (第 6 章)。driver 註冊時，platform bus 拿它和每個還沒有 driver 的裝置比對，比對函式是 `drivers/base/platform.c` 的 `platform_match()`，順序是：裝置設了 `driver_override` 就只比 driver 名字；否則比 device tree 的 `compatible` 與 `of_match_table`；接著是 ACPI、`id_table`，最後比裝置名稱。比對成功就呼叫 `probe()`，回傳 0 才算綁定成功。

device tree 的檔案從哪裡來。PetaLinux 每次建置都從 Vivado 匯出的 XSA 重新產生最上層的 `system-top.dts`：它 include 晶片本身的 `zynqmp.dtsi`、時脈的 `zynqmp-clk-ccf.dtsi`、block design 裡 PL IP 的 `pl.dtsi`、PS 設定的 `pcw.dtsi`，最後才 include 我們維護的 `system-user.dtsi` (在 `linux/project-spec/` 底下)，所以後者可以改寫前面的內容。`.dtsi` 是被 include 的片段，格式和 `.dts` 相同，就像 C 的 `.h`。`dtc` (device tree compiler) 把它們編成二進位的 `system.dtb`，kernel 只讀這個檔。PetaLinux 為加速器產生的節點寫的是 `compatible = "xlnx,twn-fuse-1.1"`，沒有 driver 認得；

`system-user.dtsi` 用 `&fuse_0`（修改既有節點）把它改成 driver 的字串，並加上 `iommus`。發佈時 PetaLinux 產生的這幾個檔案，以及反編譯回文字的 `system.dtb`，存在 `linux/device-tree/`，可以和 `system-user.dtsi` 對照著看。

`compatible` 的格式與版本號。格式是 `"製造商,型號"`，逗號前是 vendor prefix（代表廠商或作者）。Linux 在 `vendor-prefixes.yaml` 登記合法的前綴；`wjhuang` 不在表上，自己用沒問題，送進 mainline（Linus Torvalds 維護的官方 kernel 原始碼樹）才需要登記。結尾的版本號在這個專案真的用上了：加了中斷的 bitstream 是 `-1.1`；舊的 driver 只認得 `-1.0`，就不會去等一個它不知道的中斷。

`devm_` 為什麼好用。每一筆 `devm_*` 配置都登記在裝置上，driver 離開或 `probe()` 中途失敗時，kernel 按相反順序全部釋放。所以 `probe()` 每一步失敗只要 `return` 錯誤碼，不用寫一串 `goto` 收拾。

binding 檢查的是什麼。device tree 是資料，不是程式：寫錯一個屬性名稱，`dtc` 照樣編過，開機後 driver 才在 `probe()` 裡找不到東西。`dtc` 不看 binding，要另外跑 `make dtbs_check`（或本專案的 `binding_check.py`）才能在開機前抓到。`status`、`phandle`、`interrupt-parent` 是每個節點都能有的通用屬性，由 dt-schema 的核心規格描述，個別 binding 不必列。kernel 官方的 `make dt_binding_check` 會逐條驗證屬性的格式，這次沒有跑，原因與補法在第 13 章。

一個沒處理的限制：PL 的時脈。節點有 `clocks = <&zynqmp_clk 71>`（PL 的 240 MHz 時脈），但 `probe()` 沒有取用它。現在能運作，推測是因為開機參數有 `clk_ignore_unused`，kernel 不會關掉「沒有 driver 宣告在用」的時脈（沒有試過拿掉它；時脈 driver 從 PMU firmware 拿到的旗標也可能讓這個時脈本來就不被關）。若 PL 時脈被關掉，之後讀暫存器會讓 AXI 匯流排卡住。正確做法是在 `probe()` 用 `devm_clk_get_enabled()` 宣告使用；這次沒有做。

1.3 面試會問

面試會問（完整版，約 1 分鐘）：compatible 字串怎麼讓 kernel 找到你的 driver？你怎麼確定節點寫對了？

開機時 kernel 把 device tree 展開，PL 底下每個節點變成一個 platform device，`reg` 變成它的記憶體資源，中斷等 probe 時才從節點解析。我的 driver 有一張 `of_match_table`，module 載入時向 platform bus 註冊，bus 用 `platform_match()` 比對：先看 `driver_override`，再比 compatible。對上了就呼叫我的 `probe()`：ioremap 暫存器、配 DMA buffer、申請中斷、註冊 `/dev/twn_fuse`，全部用 `devm_`，失敗時自動收回。compatible 帶版本號，加了中斷的硬體是 `-1.1`。節點的規格我寫成 dt-schema 的 binding，規定 `-1.1` 一定要有中斷，並拿 PetaLinux 實際產生的 dtb 去比對，因此發現它多產生了 `interrupt-names`。

1.4 關鍵函式速查

名稱	位置	做什麼
`twn_of_match`	`twn_fuse.c`: 666	比對表：`-1.1` 與 `-1.0`
`twn_driver`	`twn_fuse.c`: 682	把 probe、remove、比對表、sysfs 屬性綁在一起
`twn_probe()`	`twn_fuse.c`: 578	接管硬體的五個步驟
`twn_remove()`	`twn_fuse.c`: 651	移除 debugfs 與裝置檔、關中斷致能，其餘交給 `devm_`
`&fuse_0`	`system-user.dtsi`: 20	發佈映像的節點：compatible 與 `iommus`
`required`、`allOf`	`wjhuang,twn-fuse.yaml`: 52	binding 的必要屬性與依版本的規則
`platform_match()`	kernel `drivers/base/platform.c`	比對順序：override、compatible、ACPI、id table、名稱

2 MMIO：ioremap、readl / writel 與 barrier

暫存器的實體位址 `0xA000_0000` 要先經 `ioremap` 換成 kernel 虛擬位址，之後用 kernel 提供、而且自帶順序保證（barrier）的 `readl` / `writel` 讀寫。

原本怎麼做（v1.0.0 的 UIO 版本）

app 把 `/dev/uio4` `mmap` 進來（第 1 章 `fuse_hal_init()`），得到一個 `volatile` 指標 `g_ctrl`，直接用陣列下標讀寫暫存器（第 78–79 行）。app 還得自己處理順序：先把輸入寫進 DDR、再寫 START，兩者是不同的記憶體區段，CPU 不保證先後，所以 v1.0.0 手寫了一個 barrier（第 93 行的 `dsb sy`）：

sw/app/pl_hal_fuse.c @ v1.0.0 (3bdd485) : 78--82

```
78 static inline void   wr(uint32_t off, uint32_t v) { g_ctrl[off >> 2] = v; }
79 static inline uint32_t rd(uint32_t off)           { return g_ctrl[off >> 2]; }
80 static inline void   wr64(uint32_t off, uint64_t v) {
81     wr(off, (uint32_t)v); wr(off + 4, (uint32_t)(v >> 32));
82 }
```

sw/app/pl_hal_fuse.c @ v1.0.0 (3bdd485) : 91--97

```
91 static inline void fuse_wmb(void) {
92 #if defined(__aarch64__)
93     __asm__ __volatile__("dsb sy" ::: "memory");
94 #else
95     __asm__ __volatile__("" ::: "memory");
96 #endif
97 }
```

問題：任何能開 `/dev/uio4` 的程式都能寫任何暫存器；barrier 要每個寫 app 的人自己記得加，漏了，PL 可能讀到還沒寫完的輸入；kernel 也不知道這段位址被誰佔用。

用了什麼技術

- `devm_platform_ioremap_resource(pdev, 0)`：取出節點 `reg` 的第 0 段、向 kernel 登記「這段我用了」、建立映射，一次做完，driver 離開時自動解除。
- `void __iomem *`：標記「這是 I/O 位址」的指標型別，靜態檢查工具 sparse 會擋下直接解指標的寫法。
- `readl()` / `writel()`：kernel 在 `<linux/io.h>` 提供的 32-bit 暫存器讀寫，內含 barrier；同族還有 8、16、64-bit 的 `readb` / `readw` / `readq`。
- `lower_32_bits()` / `upper_32_bits()`：把 64-bit 位址拆成兩個 32-bit 暫存器。

改善了什麼

- 正常使用路徑上，app 不再映射暫存器（第 4 章用 `/proc/<pid>/maps` 確認）；`/dev/mem` 這條旁路由第 10 章的 kernel 選項限制。
- 位址範圍登記在 `/proc/iomem`，名字是 `a0000000.twn_fuse`，別的 driver 無法再登記同一段。
- 順序保證由 `writel` / `readl` 負責，driver 裡沒有任何手寫的 barrier。

2.1 實際例子

暫存器表。位址和位元定義照 `rtl/fuse/twn_csr.v`。`R_CTRL` 的第 1 位元讀和寫意思不同：讀到的是 busy，寫進去是「清除 done」。`R_IRQ` 是 v1.1 加的中斷致能（第 6 章）：

linux/driver/twn_fuse/twn_fuse.c : 35--57

```
35 /* ---- registers (rtl/fuse/twn_csr.v) ---- */
36 #define R_CTRL          0x0000  /* R: [0] start [1] busy [2] done   W: [0] start [1] clear done */
37 #define R_NBLK          0x0008
38 #define R_ACT           0x000c  /* 40-bit addresses: low word here, high word at +4 */
39 #define R_W             0x0014
40 #define R_IN            0x001c
41 #define R_OUT           0x0024
42 #define R_STATUS        0x002c  /* [0] table valid [1] table checked [10:7] error [15:11] block */
43 #define R_CHECK         0x0030
44 #define R_CYCLES        0x0034  /* six counters, 0x0034..0x0048 */
45 #define R_PIX_NZ        0x004c  /* debug: sticky pixel-phase bits */
46 #define R_CAP           0x0050  /* debug: the frame's first project beat, 4 words */
47 #define R_BB2HD         0x0060
48 #define R_IRQ           0x0064  /* 1.1: [0] backbone done -> irq [1] head done -> irq */
49 #define R_HD_CTRL       0x0100  /* R and W: [0] start [1] busy [2] done (write 1 to clear) */
50 #define R_HD_COMMIT     0x0104
51 #define R_HD_RQ_STAT    0x0108
52 #define R_HD_IN         0x0110
53 #define R_HD_W          0x0118
54 #define R_HD_OUT        0x0120
55 #define R_HD_BEATS      0x0128  /* IN, W, OUT beats, then SHAPE, FLAGS, CLAMP, every 4 bytes */
56 #define R_HD_RQ         0x0400
```

```
57 #define R_DESC          0x1000
```

三個存取函式。除了兩處把位址直接交給 `readl_poll_timeout` 的輪詢（第 6 章），以及 debugfs 的暫存器傾印（`debugfs_create_regset32()` 自己用 `readl` 讀，第 8 章），driver 都透過這三個函式碰暫存器。`wr_addr` 把「buffer 的 DMA 位址加 offset」拆成高低兩個 word；硬體只用高位 word 的低 8 位元，所以位址是 40 bits：

linux/driver/twn_fuse/twn_fuse.c : 105--108

```
105 static u32 rd(struct twn_dev *td, u32 off)
106 {
107     return readl(td->regs + off);
108 }
```

linux/driver/twn_fuse/twn_fuse.c : 110--113

```
110 static void wr(struct twn_dev *td, u32 off, u32 v)
111 {
112     writel(v, td->regs + off);
113 }
```

linux/driver/twn_fuse/twn_fuse.c : 116--122

```
116 static void wr_addr(struct twn_dev *td, u32 reg, u64 off)
117 {
118     u64 a = td->arena_dma + off;
119 
120     wr(td, reg, lower_32_bits(a));
121     wr(td, reg + 4, upper_32_bits(a));
122 }
```

probe 的第 1、2 步。先映射，再讀 NBLK：

linux/driver/twn_fuse/twn_fuse.c : 590--593

```
590     /* 1. map the node's "reg" range: 64 KB of registers at 0xA000_0000 */
591     td->regs = devm_platform_ioremap_resource(pdev, 0);
592     if (IS_ERR(td->regs))
593         return PTR_ERR(td->regs);
```

linux/driver/twn_fuse/twn_fuse.c : 595--599

```
595     /* 2. is this our hardware? (a debug run can leave NBLK changed, so only warn) */
596     nblk = rd(td, R_NBLK);
597     if (nblk != NBLK_RESET)
598         dev_warn(dev, "NBLK reads %u, expected %u: another bitstream, or a debug run changed it\n",
599              nblk, NBLK_RESET);
```

NBLK 的 reset 值是 17。這顆 RTL 沒有 ID 暫存器，NBLK 的 reset 值是唯一的指紋。讀錯只印警告、不拒絕綁定，因為除錯時可以改寫它（第 8 章的 debugfs `nblk`），上一次除錯後再載入 driver 就會讀到別的值。

板上：位址範圍有了主人，probe 讀到 17。

results/raw/linux_driver_v11_20261027/H10_driver_state.log : 1,14-18,23-24 (開機組 H)

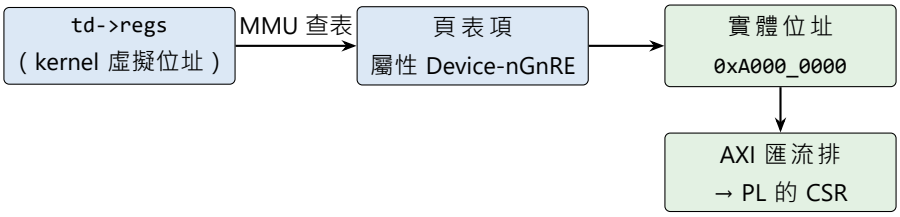
```
== 2026-10-27 03:01 UTC  6.1.30-xilinx-v2023.2  BOOT.BIN 5d226057efc8760b  system.dtb bc4255b1c54ae27e  twn_fuse.ko
     aec4477e2c520faf

# grep -i a0000000 /proc/iomem
a0000000-a000ffff : a0000000.twn_fuse twn_fuse@a0000000
# ls -l /dev/twn_fuse; cat /sys/class/misc/twn_fuse/dev
crw-------    1 root     root       10, 125 Oct 26 21:18 /dev/twn_fuse
10:125

# dmesg | grep 'twn_fuse a0000000' | tail -1
[    4.833261] twn_fuse a0000000.twn_fuse: NBLK 17, 8192 KB DMA buffer at 0x000000ffff800000, /dev/twn_fuse, interrupt
```

2.2 原理

為什麼不能直接用指標存取 `0xA000_0000`。開了 MMU (把虛擬位址翻譯成實體位址的硬體) 之後，程式裡的每個指標都是虛擬位址。arm64 Linux 的虛擬位址分兩半：低的一半屬於 user process，高的一半屬於 kernel。`0xA0000000` 當成虛擬位址落在 user 那一半，查的是當下 process 的頁表，跟實體位址 `0xA000_0000` 沒有關係：沒有映射就是 page fault，kernel 程式出這種錯會 oops (印出錯誤現場並終止目前的工作)；剛好有映射，則會靜默寫到某塊 user 記憶體上，更危險。kernel 自己的頁表一開始只對應 DDR，周邊的實體位址不在裡面，所以一定要先 `ioremap`。



`ioremap` 在 kernel 的頁表加一筆，屬性是 Device 記憶體；arm64 的 Linux 6.1 用 `PROT_DEVICE_nGnRE`。nG (non-Gathering)：不把多次存取合併；nR (non-Reordering)：對同一個裝置的存取不重排；E (Early write acknowledgement)：允許匯流排中途就回報寫完。UIO 的 `mmap` 給的是更嚴格的 nE。Device 記憶體也不經過 cache、不會被預先讀取，所以讀一次 done 位元就真的去硬體讀一次。

barrier 是什麼。程式寫的順序不等於實際發生的順序：compiler 只要單一執行緒看起來一樣就可以重排；ARM 是弱記憶體順序 (weakly ordered) 的架構，CPU 會亂序執行，store 先進 write buffer (CPU 送出寫入前暫放的小緩衝區)，寫 DDR 和寫 PL 暫存器走不同的路徑，誰先到沒有保證。對 CPU 自己看不出差別，但 PL 是另一個觀察者。barrier 是一道「順序」指令：寫在它前面的存取，一定比後面的先被別人看到。它不搬資料、不清 cache，也不是鎖。

`readl / writel` 裡面多了什麼。Linux 6.1 的通用實作 (`include/asm-generic/io.h`)，省略追蹤與位元組順序轉換 (在 little-endian 的 arm64 上不做事)：

```
writel(value, addr):  __io_bw();  __raw_writel(value, addr);   /* __io_bw = dma_wmb() = dmb oshst */
readl(addr):          v = __raw_readl(addr);  __io_ar(v);      /* __io_ar = dma_rmb() = dmb oshld ... */
```

在這個 driver 裡它們各解決一個實際問題：

- **寫 START 之前**：app 剛把輸入影像寫進 DDR 的 buffer。`writel` 先做 `dma_wmb()`，保證這些寫入比 START 先被 PL 看到；否則 PL 可能讀到上一幀或寫到一半的影像，而且不是每次都發生，很難查。
- **讀到 done 之後**：`readl` 讀完再做 `dma_rmb()`，保證之後讀 buffer 裡的輸出，不會被 CPU 提前到讀 done 之前，才不會讀到 PL 還沒寫完的結果。

ARM 有兩種 barrier 指令：`dsb`（Data Synchronization Barrier）等前面所有存取**真的完成**才往下執行；`dmb`（Data Memory Barrier）只保證**順序**。後綴 `osh`（outer shareable）指「CPU 和會存取記憶體的裝置」這個範圍，`st` / `ld` 表示只管寫或讀。v1.0.0 手寫的 `dsb sy` 是最重的一種；`writel` 的 `dmb oshst` 只排好寫入順序，已經足夠。`readl_relaxed` / `writel_relaxed` 則拿掉這些 barrier，用在確定不需要順序的地方。

為什麼要「登記」位址範圍。`devm_platform_ioremap_resource` 內部先呼叫 `request_mem_region`，在 `/proc/iomem` 記下這段位址屬於誰，另一個 driver 再要同一段就會失敗。它本身擋不住 `/dev/mem`；那要靠第 10 章的 `IO_STRICT_DEVMEM`，而它正是以這份登記為依據。

2.3 面試會問

面試會問（完整版，約 1 分鐘）：為什麼要先 ioremap？readl / writel 跟直接解指標差在哪？

開了 MMU 之後指標都是虛擬位址。`0xA0000000` 當成虛擬位址屬於 user 那一半，跟周邊的實體位址無關，kernel 的頁表裡也沒有周邊，直接存取會 page fault。`ioremap` 建一筆頁表，把一段 kernel 虛擬位址對到實體位址，屬性是 Device-nGnRE：不走 cache、不合併、不重排。存取一律用 kernel 的 `readl` / `writel`：它們保證每次真的讀寫到硬體，而且帶 barrier。`writel` 寫入前有 `dma_wmb`，所以 app 先寫進 DDR 的輸入一定比 START 先讓 PL 看到；`readl` 讀到 done 之後有 `dma_rmb`，之後讀輸出不會被提前。我的 driver 沒有手寫任何 barrier，全部靠這兩個函式。

2.4 關鍵函式速查

名稱	位置	做什麼
暫存器表	`twn_fuse.c`: 35	位址與位元，照 `twn_csr.v`
`rd()` / `wr()`	`twn_fuse.c`: 105	`readl` / `writel` 的包裝
`wr_addr()`	`twn_fuse.c`: 116	寫入 40-bit 位址：DMA 位址加 offset
probe 第 1、2 步	`twn_fuse.c`: 590	ioremap、讀 NBLK
`readl` / `writel`	kernel `include/asm-generic/io.h`	讀後 `dma_rmb()`、寫前 `dma_wmb()`

3 DMA buffer：配置、cache 與記憶體屬性

PL 讀寫的 DDR buffer 由 driver 透過 DMA API 配置，kernel 同時給出 CPU 用的位址與 PL 用的 DMA 位址，並讓 CPU 以 Normal non-cacheable 屬性存取它；在同一組開機檔上，這個屬性就是 driver 版本比 UIO 版本快約 1.1% 的原因。

原本怎麼做（v1.0.0 的 UIO 版本）

device tree 保留一塊固定的 256 MB，標成 no-map（kernel 完全不碰）：

linux/project-spec/meta-user/recipes-bsp/device-tree/files/system-user.dtsi @ v1.0.0 (3bdd485) : 3--12

```
    reserved-memory {
        #address-cells = <2>;
        #size-cells = <2>;
        ranges;
        pl_arena: pl_arena@60000000 {
            no-map;
            reg = <0x0 0x60000000 0x0 0x10000000>;   /* 256 MB @ 0x6000_0000, uncached */
        };
    };
};
```

app 再經 `/dev/mem`（第 148 行），用寫死的實體位址 `0x6000_0000`（第 140 行的 `FUSE_ARENA_PA_DEFAULT`）把整塊 `mmap` 進來：

sw/app/pl_hal_fuse.c @ v1.0.0 (3bdd485) : 138--156

```
void *fuse_arena_init(uint64_t *phys, uint64_t size) {
    const char *pas = getenv("PL_ARENA_PA"), *szs = getenv("PL_ARENA_SZ");
    g_arena_pa = pas ? strtoull(pas, 0, 16) : FUSE_ARENA_PA_DEFAULT;
    g_arena_sz = szs ? strtoull(szs, 0, 16) : FUSE_ARENA_SZ_DEFAULT;
    if (size > g_arena_sz) {
        fprintf(stderr, "arena too small: need %llu have %llu\n",
                (unsigned long long)size, (unsigned long long)g_arena_sz);
        exit(1);
    }
    if (!g_ctrl) fuse_hal_init();
    int fd = open("/dev/mem", O_RDWR | O_SYNC);
    if (fd < 0) { perror("open /dev/mem"); exit(1); }
    g_arena_v = (uint8_t *)mmap(NULL, g_arena_sz, PROT_READ | PROT_WRITE, MAP_SHARED,
                                fd, (off_t)g_arena_pa);
    if (g_arena_v == MAP_FAILED) { perror("mmap /dev/mem arena"); exit(1); }
    close(fd);
    *phys = g_arena_pa;
    return g_arena_v;
}
```

問題：位址寫死在程式與 device tree 兩個地方；256 MB 只用到約 6 MB，其餘 kernel 也不能用；`/dev/mem` 能讀寫**任何**實體位址（第 10 章）；映射出來的屬性是 Device-nGnRnE，寫入不能合併、不能非對齊存取，連 `memset` 都不能用。

用了什麼技術

- `dma_set_mask_and_coherent(dev, DMA_BIT_MASK(40))`：告訴 kernel 這個裝置能定址 40 bits。
- `dmam_alloc_coherent()`：配一塊 CPU 與裝置「一致」(coherent) 的記憶體，回傳 CPU 位址與 `dma_addr_t`；`dmam_` 表示 driver 離開時自動歸還。
- CMA (Contiguous Memory Allocator，連續記憶體配置器)：開機時劃出的一塊區域 (這裡 256 MB)，平時借給可以搬走的一般用途，driver 要大塊實體連續的記憶體時再清出來。
- `dma_addr_t`：裝置看到的位址。沒有 IOMMU 時等於實體位址；有 SMMU 時是 I/O 虛擬位址 (IOVA，第 11 章)，程式不該假設兩者相同。

改善了什麼

- 位址不再寫死：每次載入由 kernel 決定，driver 再寫進硬體。driver 只配 8 MB (module 參數 `arena_mb`)，app 需要 6 MB；256 MB 保留區拿掉了 (第 10 章)。
- 卸載時自動歸還：沒有 SMMU 時連續載入 10 次、每次 64 MB (共 640 MB，CMA 只有 256 MB)，10 次都成功。
- CPU 端屬性從 Device 變成 Normal non-cacheable，寫入可以合併：CPU 寫 buffer 快 3 到 5 倍，pipeline 的「寫輸入」從 0.14–0.15 ms 縮到 0.09–0.10 ms (每次執行印出的最後一幀，各 3 次)，fps 高 1.2% (開機組 D，只差這個屬性的兩組)。可以 `memset` 這一點是依 ARM 規格推得，app 沒有改用，也沒有在板上測。

3.1 實際例子

probe 的第 3 步。先設 DMA mask，再配 buffer，最後檢查對齊：

linux/driver/twn_fuse/twn_fuse.c : 601--613

```
601     /* 3. the DMA buffer; the PL's address registers are 40 bits wide */
602     ret = dma_set_mask_and_coherent(dev, DMA_BIT_MASK(40));
603     if (ret)
604         return ret;
605     td->arena_size = (size_t)arena_mb << 20;
606     td->arena = dmam_alloc_coherent(dev, td->arena_size, &td->arena_dma, GFP_KERNEL);
607     if (!td->arena)
608         return -ENOMEM;
609     /* the app lays out 64 KB-aligned regions (head input slots 64 KB apart) */
610     if (td->arena_dma & (SZ_64K - 1)) {
611         dev_err(dev, "DMA buffer at %pad is not 64 KB aligned\n", &td->arena_dma);
612         return -EINVAL;
613     }
```

`DMA_BIT_MASK(40)` 對應 CSR 的 40-bit 位址暫存器；kernel 配記憶體時保證位址在這個範圍內。`dmam_`

`alloc_coherent` 的回傳值 `td->arena` 是 CPU 位址（kernel 用），`&td->arena_dma` 收到 DMA 位址（寫進 PL 的暫存器）。64 KB 對齊是因為 app 的配置以 64 KB 為單位（head 的兩個輸入 slot 相隔 `0x10000`）。

8 MB 裡放什麼。buffer 的配置由 app 決定，driver 只管「給一塊、檢查 offset 不越界」。`FUSE_ARENA_NEED` 是 6 MB，後面還有一個 `_Static_assert`，配置長大超過 6 MB 就編譯失敗：

sw/app/mbv2_fuse_ps.c : 39--45

```
39 #define OFF_WEIGHTS   0x00000000u                    /* fuse_weights.bin, 64-B aligned */
40 #define OFF_ACT       0x00100000u                    /* the slot region          */
41 #define OFF_IN        (OFF_ACT + FUSE_N_SLOTS * FUSE_SLOT_STRIDE)
42 #define OFF_OUT       (OFF_IN + 0x00100000u)
43 /* the whole map ends at OFF_HD_OUT + 2 * HD_OUT_SLOT_BYTES (asserted below the
44  * head's regions); the kernel driver's DMA buffer must be at least this big */
45 #define FUSE_ARENA_NEED 0x00600000u
```

依序是：backbone 權重、兩個 activation slot（17 個 block 輪流使用）、輸入影像、backbone 輸出；`0x400000` 起是 head 的權重、兩個輸入 slot、三層中間結果與兩個輸出 slot。

板上：buffer 從哪裡來，取決於有沒有 SMMU。關掉 SMMU 時（開機組 I），DMA 位址 `0x6a10_0000` 落在 CMA（從 `0x6a00_0000` 起 256 MB）裡，`CmaFree` 少了 8 MB 多；連續 10 次各要 64 MB，超過 CMA 的總量也都成功，表示每次卸載都真的歸還了：

results/raw/linux_driver_v11_20261027/I1_driver_state_nosmmu.log : 1,19-24,56-60（開機組 I）

```
1  == 2026-10-27 03:08 UTC  6.1.30-xilinx-v2023.2  BOOT.BIN 5d226057efc8760b  system.dtb 6ceadb90b64b6a98  twn_fuse.ko
       aec4477e2c520faf
19 # dmesg | grep -i 'cma: reserved'; grep -E 'CmaTotal|CmaFree' /proc/meminfo
20 [    0.000000] cma: Reserved 256 MiB at 0x000000006a000000
21 CmaTotal:         262144 kB
22 CmaFree:          253376 kB
23 # dmesg | grep 'twn_fuse a0000000' | tail -1
24 [    4.765513] twn_fuse a0000000.twn_fuse: NBLK 17, 8192 KB DMA buffer at 0x000000006a100000, /dev/twn_fuse, interrupt
56 # ok=0; for i in 1 2 3 4 5 6 7 8 9 10; do rmmod twn_fuse && insmod
       /lib/modules/6.1.30-xilinx-v2023.2/extra/twn_fuse.ko arena_mb=64 && [ -e /dev/twn_fuse ] && ok=$((ok+1)); done;
       echo loaded $ok of 10
57 loaded 10 of 10
58 # dmesg | grep 'twn_fuse a0000000' | tail -2
59 [  122.870108] twn_fuse a0000000.twn_fuse: NBLK 17, 65536 KB DMA buffer at 0x000000006a100000, /dev/twn_fuse, interrupt
60 [  122.930084] twn_fuse a0000000.twn_fuse: NBLK 17, 65536 KB DMA buffer at 0x000000006a100000, /dev/twn_fuse, interrupt
```

發佈映像打開了 SMMU（開機組 H）：DMA 位址變成 IOVA `0xff_ff80_0000`，`CmaFree` 幾乎沒少，因為 SMMU 可以把分散的實體頁排成連續的 IOVA，DMA API 改從一般記憶體配頁（本章「原理」）：

results/raw/linux_driver_v11_20261027/H10_driver_state.log : 19-24（開機組 H）

```
19 # dmesg | grep -i 'cma: reserved'; grep -E 'CmaTotal|CmaFree' /proc/meminfo
20 [    0.000000] cma: Reserved 256 MiB at 0x000000006a000000
21 CmaTotal:         262144 kB
22 CmaFree:          261632 kB
23 # dmesg | grep 'twn_fuse a0000000' | tail -1
24 [    4.833261] twn_fuse a0000000.twn_fuse: NBLK 17, 8192 KB DMA buffer at 0x000000ffff800000, /dev/twn_fuse, interrupt
```

踩過的坑

一開始用 `/proc/meminfo` 的 `CmaFree` 檢查有沒有漏記憶體，卸載後數字卻沒有回升，看起來像漏掉了。其實不是：CMA 沒人用時會借給 page cache（kernel 快取檔案內容的記憶體）這類可搬移的用途，`CmaFree` 會隨讀檔變化，不能當證據。後來改用「載入總量超過 CMA」的方法，才得到能下結論的證據。

量測：記憶體屬性值多少。driver v1.0 剛完成時比 UIO 版本快約 0.8%（開機組 A，第 13 章），但當時一起改了三件事：等硬體的地方、buffer 位置、CPU 端屬性。在 v1.0.0 的開機檔上（開機組 D）做四組實驗，每組 pipeline 跑 6 次、分兩輪且順序相反：

組別	等硬體的地方	buffer 位置	CPU 端屬性	跟上一組的差別
`uio`	app	`0x6000_0000`	Device-nGnRnE	起點（v1.0.0）
`uio_68`	app	`0x6800_0000`	Device-nGnRnE	buffer 位置
`drv_dev`	driver	CMA	Device-nGnRnE	等硬體的地方與 buffer 來源（一起換）
`drv_nc`	driver	CMA	Normal non-cacheable	只有 CPU 端屬性

`drv_dev` 靠 driver 的 module 參數 `map_device=1`：`twn_mmap()` 改用 `/dev/mem` 那種 Device 屬性映射（第 4 章）。

`results/raw/linux_driver_v11_20261027/D1_ab_measure.log : 1-2,27-31`（開機組 D），第 27–31 行：每組 6 次的平均

```
== 2026-10-26 17:36 UTC  6.1.30-xilinx-v2023.2  BOOT.BIN 690d87cc5f4b6897  system.dtb 18fff66fcfb7798b  twn_fuse.ko 5f77eacd80e62272
# sh test/ab_measure.sh 2>/dev/null || sh ab_measure.sh
== mean of 6 runs per arm
uio      228.94 fps (n=6)
uio_68   229.09 fps (n=6)
drv_dev  228.83 fps (n=6)
drv_nc   231.53 fps (n=6)
```

前三組都在 228.8 到 229.1 fps，只有 `drv_nc` 跳到 231.53 fps。直接量 CPU 存取 buffer 的速度（`twn_bench` 連續寫、讀 1 MB），寫入差很多，讀取一樣：

`results/raw/linux_driver_v11_20261027/D2_bench.log : 11-12,20`（開機組 D）：第 11 行是 Normal non-cacheable，第 20 行是 `map_device=1`

```
CPU access to the buffer: 1-byte stores 316 MB/s, 8-byte stores 4250 MB/s, 8-byte loads 81 MB/s
# rmmod twn_fuse; insmod ./twn_fuse.ko map_device=1
CPU access to the buffer: 1-byte stores 103 MB/s, 8-byte stores 853 MB/s, 8-byte loads 81 MB/s
```

pipeline 開 `PIPE_TIMING=1`，每次執行（120 幀）印出最後一幀每一步的時間：只有 `write-in`（CPU 把輸入影像寫進 buffer）變了，從 0.14–0.15 ms 變成 0.09–0.10 ms，和這三次執行平均的每幀週期差（約 0.04 ms）相當：

`results/raw/linux_driver_v11_20261027/D6_stage_times.log : 3-21`（開機組 D）：前三次是 Normal non-cacheable，後三次是 `map_device=1`

```
# FUSE_BB2HD=1 FUSE_HEAD_PL=1 PIPE_TIMING=1 ./mbv2_fuse_pipe_drv . 120 0 2>&1 | grep -a -E 'producer split|fps ==='
[pipe_timing] producer split (last frame): write-in=0.10  stage-boundary=0.00  backbone PL=4.14  head PL=3.09 ms  (the head overlaps the backbone; it is not additive)
=== fused cross-frame pipeline: 120 frames in 519.6 ms = 4.33 ms/frame = 230.94 fps ===
# FUSE_BB2HD=1 FUSE_HEAD_PL=1 PIPE_TIMING=1 ./mbv2_fuse_pipe_drv . 120 0 2>&1 | grep -a -E 'producer split|fps ==='
[pipe_timing] producer split (last frame): write-in=0.09  stage-boundary=0.00  backbone PL=4.15  head PL=3.07 ms  (the head overlaps the backbone; it is not additive)
=== fused cross-frame pipeline: 120 frames in 518.1 ms = 4.32 ms/frame = 231.60 fps ===
# FUSE_BB2HD=1 FUSE_HEAD_PL=1 PIPE_TIMING=1 ./mbv2_fuse_pipe_drv . 120 0 2>&1 | grep -a -E 'producer split|fps ==='
[pipe_timing] producer split (last frame): write-in=0.09  stage-boundary=0.00  backbone PL=4.15  head PL=3.13 ms  (the head overlaps the backbone; it is not additive)
=== fused cross-frame pipeline: 120 frames in 517.6 ms = 4.31 ms/frame = 231.83 fps ===
# rmmod twn_fuse; insmod ./twn_fuse.ko map_device=1
# FUSE_BB2HD=1 FUSE_HEAD_PL=1 PIPE_TIMING=1 ./mbv2_fuse_pipe_drv . 120 0 2>&1 | grep -a -E 'producer split|fps ==='
[pipe_timing] producer split (last frame): write-in=0.15  stage-boundary=0.00  backbone PL=4.19  head PL=3.10 ms  (the head overlaps the backbone; it is not additive)
=== fused cross-frame pipeline: 120 frames in 525.2 ms = 4.38 ms/frame = 228.47 fps ===
# FUSE_BB2HD=1 FUSE_HEAD_PL=1 PIPE_TIMING=1 ./mbv2_fuse_pipe_drv . 120 0 2>&1 | grep -a -E 'producer split|fps ==='
[pipe_timing] producer split (last frame): write-in=0.14  stage-boundary=0.00  backbone PL=4.18  head PL=3.09 ms  (the head overlaps the backbone; it is not additive)
=== fused cross-frame pipeline: 120 frames in 522.9 ms = 4.36 ms/frame = 229.51 fps ===
# FUSE_BB2HD=1 FUSE_HEAD_PL=1 PIPE_TIMING=1 ./mbv2_fuse_pipe_drv . 120 0 2>&1 | grep -a -E 'producer split|fps ==='
[pipe_timing] producer split (last frame): write-in=0.14  stage-boundary=0.00  backbone PL=4.15  head PL=3.07 ms  (the head overlaps the backbone; it is not additive)
=== fused cross-frame pipeline: 120 frames in 522.3 ms = 4.35 ms/frame = 229.74 fps ===
```

3.2 原理

為什麼會有 cache 問題。CPU 讀寫 DDR 會經過 cache；PL 經 HP0–HP3 port（PL 存取 DDR 的四個高速 AXI 埠）直接到 DDR 控制器，**不會**去查 CPU 的 cache。buffer 若是 cacheable：CPU 寫的輸入可能還停在 cache，PL 讀到舊資料；PL 寫進 DDR 的輸出，CPU 可能命中 cache 裡上一幀的內容。處理方法有三種：

做法	怎麼做	代價
coherent buffer（這次用的）	`dma_alloc_coherent`：CPU 端映射成 non-cacheable，讀寫直接到 DDR	CPU 存取比 cacheable 慢；app 只寫輸入、讀輸出，量不大
streaming DMA	一般 cacheable 記憶體；PL 讀之前 `dma_sync_single_for_device`（寫回 cache），PL 寫完 `dma_sync_single_for_cpu`（作廢 cache）	每幀都要做 cache 維護，app 也要配合
硬體一致性	改用 HPC port，PL 的存取經過 CCI（會查 CPU cache 的互連）；AXI master 送 cacheable、shareable 的屬性；節點加 `dma-coherent`	要改 RTL 與 block design、重建 bitstream

kernel 不知道 HP port 是什麼，它看 device tree：節點沒有 `dma-coherent`，就把裝置當成 non-coherent，`dma_alloc_coherent` 給出 non-cacheable 映射。

`dma_alloc_coherent` 實際做了什麼。沒有 SMMU 時：從 CMA 拿實體連續的頁（一般的配置器一次最多給 4 MB，而且久了記憶體會零碎，不保證拿得到）；把這些頁在 cache 裡的內容寫回並作廢；另外映射一個 Normal non-cacheable 的 kernel 位址回傳；DMA 位址等於實體位址。有 SMMU 時：PL 看到的是 IOVA，實體頁不必連續，DMA API 從一般記憶體配頁，再由 SMMU 排成一段連續的 IOVA，所以不佔 CMA。driver 的程式兩種情況完全一樣。

為什麼不用 `kmalloc`。它的記憶體在 kernel 的線性映射（linear map：kernel 頁表裡把整個 DDR 一對一對應進來的那一段，cacheable）裡，要給 PL 就得自己做 streaming DMA；一次最多 4 MB；也不給 `dma_addr_t`，而有 SMMU 時實體位址根本不是 PL 該用的位址。

兩種「不經過 cache」的記憶體。arm64 頁表的每一頁都帶屬性。這裡牽涉兩種，CPU 存取都不經過 cache，差別在寫入：

	Device-nGnRnE	Normal non-cacheable
誰會得到	`/dev/mem` 映射 no-map 區、`pgprot_noncached()`	`dma_mmap_coherent()`、`pgprot_dmacoherent()`
寫入	每個 store 都是一筆獨立的匯流排交易，不合併（nG）；回應必須由終點發出（nE）	連續的小 store 可以在 write buffer 裡合併成一筆大的寫入（write gathering）
讀取	每次都到 DDR	每次都到 DDR
對齊	不能非對齊存取	可以

所以寫入差 3 到 5 倍、讀取一樣，和量到的一致。讀取 81 MB/s 表示每 8 bytes 約 100 ns，大約是一趟 DDR 讀取的延遲，兩種屬性都要付。只有 `write-in` 反映到 fps，因為寫輸入在 producer 執行緒的關鍵路徑上（寫完才能啟動 PL）；讀 heatmap 在另一個執行緒，和 PL 重疊。

實驗的限制。`uio_68` 只換位置、fps 不變，排除了「buffer 在 DDR 哪裡」；但 `uio_68→drv_dev` 一次換了兩件事，要分開還需要一組「driver 版、buffer 在保留區」，這次沒有做，所以只能說「輪詢位置與 buffer 來源合起來看不出影響」。

3.3 面試會問

面試會問（完整版，約 1 分鐘）：為什麼用 dma_alloc_coherent？HP port 不是 coherent，cache 怎麼處理？driver 為什麼比較快？

`dma_alloc_coherent` 一次給我 CPU 位址和裝置用的 DMA 位址。不用 `kmalloc`，因為它是 cacheable、最大 4 MB、也不給 DMA 位址；不用 `/dev/mem`，因為它能讀寫整個實體記憶體。HP port 不查 CPU 的 cache，節點沒有 `dma-coherent`，kernel 就把 buffer 映射成 non-cacheable。沒有 SMMU 時它從 CMA 拿連續的頁，有 SMMU 時從一般記憶體配、由 SMMU 排成連續的 IOVA。driver 版本比 UIO 版本快約 1%（同一組開機檔上 1.1%），我做四組 A/B 實驗，把記憶體屬性單獨拆出來，只差這一項的兩組差 1.2%：原因是 CPU 端屬性從 Device 變成 Normal non-cacheable，寫入可以合併，CPU 寫 buffer 快 3 到 5 倍，寫輸入從 0.14–0.15 降到 0.09–0.10 ms。

3.4 關鍵函式速查

名稱	位置	做什麼
probe 第 3 步	`twn_fuse.c: 601`	40-bit mask、配 buffer、檢查對齊
`arena_mb`	`twn_fuse.c: 76`	buffer 大小（MB），預設 8
buffer 配置	`mbv2_fuse_ps.c: 39`	app 在 8 MB 裡的區段
`run()`	`test/ab_measure.sh: 18`	A/B 實驗：切 driver、跑 pipeline 三次
第 3 部分	`test/twn_bench.c: 96`	CPU 寫、讀 buffer 的頻寬
`dmam_alloc_coherent`	kernel	coherent buffer，driver 離開時自動歸還

4 mmap：只把 buffer 給 user space

driver 的 `mmap` 只把 DMA buffer 那幾頁放進 app 的頁表，app 不必複製資料（zero-copy）就能讀寫 buffer，而暫存器從來不經由 driver 映射出去。

原本怎麼做（v1.0.0 的 UIO 版本）

v1.0.0 的 app 自己做兩次映射：暫存器（`/dev/uio4` 的 64 KB，第 1 章「原本怎麼做」的 `fuse_hal_init()`）和整塊實體記憶體（`/dev/mem` 從 `0x6000_0000` 起 256 MB，第 3 章「原本怎麼做」的 `fuse_arena_init()`）。板上 app 執行時，它的位址空間最後兩行就是這兩個映射；app 的任何一個錯誤指標，都可能寫到暫存器或那 256 MB 的任何地方：

results/raw/linux_driver_20261026/A4_uio_state.txt : 5-12（開機組 A）

```
# FUSE_BB2HD=1 FUSE_HEAD_PL=1 ./mbv2_fuse_pipe_uio . 3000 0 > /dev/null 2>&1 &
# sleep 2; pid=$(pidof mbv2_fuse_pipe_uio)
# grep -E "twn_fuse|uio|/dev/mem" /proc/$pid/maps
aaaab86d0000-aaaab8724000 r-xp 00000000 00:02 9647                       /home/root/mbv2_fuse_pipe_uio
aaaab873f000-aaaab8740000 r--p 0005f000 00:02 9647                       /home/root/mbv2_fuse_pipe_uio
aaaab8740000-aaaab8741000 rw-p 00060000 00:02 9647                       /home/root/mbv2_fuse_pipe_uio
ffff91e5d000-ffffa1e5d000 rw-s 60000000 00:05 3                          /dev/mem
ffffa1e5d000-ffffa1e6d000 rw-s 00000000 00:05 1465                       /dev/uio4
```

用了什麼技術

- `file_operations.mmap`：app 呼叫 `mmap()` 時，kernel 先建立一段位址範圍，再呼叫 driver 的這個函式，由 driver 決定這段範圍對應到哪些實體頁、用什麼屬性。
- `struct vm_area_struct`（VMA）：描述這段範圍，包括起點 `vm_start`、終點 `vm_end`，以及 app 要求的 offset `vm_pgoff`（以頁為單位）。
- `dma_mmap_coherent()`：把 `dma_alloc_coherent` 配的 buffer 映射到 user space，屬性和 kernel 端一樣是 non-cacheable。

改善了什麼

- app 的位址空間裡只剩一段 8 MB 的 `/dev/twn_fuse`，沒有暫存器，也沒有 `/dev/mem`。
- 要求超過 buffer 大小，或 offset 不是 0，都會被拒絕（`-EINVAL`）。

- 仍然是 zero-copy：輸入影像與 heatmap 不必經過 `read()` / `write()` 複製；CPU 端是 Normal non-cacheable，寫入可以合併（第 3 章）。

4.1 實際例子

driver 端：只映射 buffer。先檢查 offset 與長度，再交給 DMA API。中間的 `map_device` 分支只給第 3 章的 A/B 實驗用：它改用 `remap_pfn_range()` 加 `pgprot_noncached()`，得到和 `/dev/mem` 一樣的 Device 屬性；有 SMMU 時 buffer 沒有固定的實體位址可以這樣映射，所以拒絕：

linux/driver/twn_fuse/twn_fuse.c : 345--365

```
345 static int twn_mmap(struct file *f, struct vm_area_struct *vma)
346 {
347     struct twn_dev *td = file_to_td(f);
348     unsigned long len = vma->vm_end - vma->vm_start;
349 
350     /* only the arena, from its start: the registers are never mapped */
351     if (vma->vm_pgoff || len > td->arena_size)
352         return -EINVAL;
353     if (map_device) {
354         /*
355          * A/B measurement only: Device-nGnRnE, the attribute /dev/mem gives.
356          * Valid only without an IOMMU, where the DMA address is the physical one.
357          */
358         if (device_iommu_mapped(td->dev))
359             return -EINVAL;
360         vma->vm_page_prot = pgprot_noncached(vma->vm_page_prot);
361         return remap_pfn_range(vma, vma->vm_start, PHYS_PFN(td->arena_dma), len,
362                                vma->vm_page_prot);
363     }
364     return dma_mmap_coherent(td->dev, vma, td->arena, td->arena_dma, len);
365 }
```

`file_to_td(f)` 從 `struct file` 找回 driver 的 `struct twn_dev`（第 5 章）。

app 端：問大小、映射、回報「基底位址 0」。

sw/app/pl_hal_fuse_drv.c : 59--72

```
59 void *fuse_arena_init(uint64_t *phys, uint64_t size) {
60     fuse_hal_init();
61     struct twn_status s = status();
62     if (size > s.arena_size) {
63         fprintf(stderr, "arena too small: need %llu, the driver has %llu "
64                         "(module parameter arena_mb)\n",
65                 (unsigned long long)size, (unsigned long long)s.arena_size);
66         exit(1);
67     }
68     void *v = mmap(NULL, s.arena_size, PROT_READ | PROT_WRITE, MAP_SHARED, g_fd, 0);
69     if (v == MAP_FAILED) { perror("mmap " TWN_DEV_PATH); exit(1); }
70     *phys = 0;                          /* addresses are offsets into the buffer */
71     return v;
72 }
```

`*phys = 0`：回報基底位址 0，app 用「基底加 offset」算出的位址就變成 offset，正好是 ioctl 要的東西（為什麼這樣 app 就不用改，見第 9 章）。

板上：app 的位址空間只剩 buffer。pipeline app 執行中，看它的 `/proc/<pid>/maps` 與開著的檔案：

results/raw/linux_driver_v11_20261027/H10_driver_state.log : 56-58（開機組 H）

```
56 # FUSE_BB2HD=1 FUSE_HEAD_PL=1 ./mbv2_fuse_pipe_drv . 3000 0 > /dev/null 2>&1 & sleep 2; pid=$(pidof
       mbv2_fuse_pipe_drv); grep -E 'twn_fuse|uio|/dev/mem' /proc/$pid/maps; ls -l /proc/$pid/fd | grep -E
       'twn|uio|mem'; wait
57 ffff952fd000-ffff95afd000 rw-s 00000000 00:05 668                      /dev/twn_fuse
58 lrwx------    1 root     root           64 Oct 27 03:01 3 -> /dev/twn_fuse
```

`0xffff95afd000 - 0xffff952fd000 = 0x800000`，正好 8 MB。和前面 v1.0.0 的輸出相比，`/dev/mem` 與 `/dev/uio4` 都不見了。越界的要求被拒絕：

results/raw/linux_driver_v11_20261027/H4_board_check.log : 10-12（開機組 H）

```
10 PASS mmap the whole buffer                          -> ok
11 PASS mmap one page more than the buffer             -> Invalid argument
12 PASS mmap at a non-zero offset                      -> Invalid argument
```

4.2 原理

`mmap` 做了什麼。app 呼叫 `mmap(NULL, len, ..., fd, offset)` 時，kernel 在 app 的位址空間找一段空的範圍，建立 VMA，記下長度與 offset（`vm_pgoff = offset / 4096`），再呼叫 driver 的 `.mmap` 填頁表。driver 回傳錯誤碼，整個 `mmap()` 就失敗。

`dma_mmap_coherent` 的內部。沒有 SMMU 時走 `kernel/dma/direct.c` 的 `dma_direct_mmap`：用 `dma_pgprot` 把屬性改成 non-cacheable（同一塊記憶體若被 kernel 和 user 用不同的 cache 屬性映射，CPU 可能看到不一致的資料），檢查範圍不超出 buffer，再用 `remap_pfn_range` 把這段連續的實體頁一次填進頁表。發佈映像打開了 SMMU，buffer 的實體頁不連續（第 3 章），改走 `drivers/iommu/dma-iommu.c` 的 `iommu_dma_mmap`，逐頁映射。對 driver 來說都是同一個呼叫。

app 為什麼碰不到暫存器。app 的位址空間裡有什麼，完全由 kernel 建的頁表決定。CSR 只在 kernel

裡做過 ioremap，driver 的 .mmap 只映射 buffer，所以 app 的頁表裡沒有任何一項指向 0xA000_0000。剩下的旁路是 root 自己開 /dev/mem：發佈映像用 STRICT_DEVMEM 與 IO_STRICT_DEVMEM 擋住 System RAM 與 driver 佔用中的暫存器，但有邊界（第 10 章）。

映射的生命週期。VMA 持有 struct file 的參考。就算 app 關掉 fd，只要映射還在，檔案就不算關閉，module 也就不能卸載（第 7 章的板上測試）。這保證 buffer 不會在 app 還在用的時候被釋放。

4.3 面試會問

面試會問（完整版，約 1 分鐘）：app 為什麼只看得到 buffer，看不到暫存器？為什麼用 mmap 不用 read / write？

app 的頁表完全由 kernel 決定。我的 driver 的 .mmap 只把 dma_alloc_coherent 配的 buffer 用 dma_mmap_coherent 映射給 app，並檢查長度不超過 buffer、offset 必須是 0；暫存器只在 kernel 裡 ioremap，從來沒有映射出去。板上 /proc/<pid>/maps 可以看到：UIO 版本有 256 MB 的 /dev/mem 和 64 KB 的 /dev/uio4，driver 版本只剩 8 MB 的 /dev/twn_fuse。資料走 mmap 是因為每幀幾百 KB，用 read / write 要經 copy_from_user / copy_to_user 多複製兩次；mmap 讓 app 和 PL 看同一塊記憶體。

4.4 關鍵函式速查

名稱	位置	做什麼
twn_mmap()	twn_fuse.c: 345	檢查 offset 與長度，映射 buffer
map_device 的檢查	twn_fuse.c: 358	有 SMMU 時拒絕 Device 映射
fuse_arena_init()	pl_hal_fuse_drv.c: 59	問大小、mmap、回報基底 0
dma_mmap_coherent	kernel	映射 coherent buffer，屬性與 kernel 端一致

5 file operations 與 ioctl：控制介面、copy_from_user 與 offset

`/dev/twn_fuse` 只實作 `open`、`release`、`mmap`、`unlocked_ioctl` 四個 file operation：資料走 mmap，指令走 ioctl，參數一律用 `copy_from_user` 取，DDR 位址只收 buffer 內的 offset。

原本怎麼做（v1.0.0 的 UIO 版本）

app 自己寫暫存器，寫進去的是實體位址（`g_arena_pa + OFF_IN` 這種算法）。`wr64()` 直接把它拆成兩個 word 寫進 PL（第 2 章「原本怎麼做」的 `pl_hal_fuse.c` 第 80 行）。PL 拿到什麼位址就讀寫什麼位址，不經過任何檢查；app 算錯一個位址，或被惡意修改，PL 就會改寫 kernel 或其他程式的記憶體。每個 app 也都要自己記住「先等 idle、清 done、寫位址、再寫 START」的順序。

用了什麼技術

- misc device：主裝置號固定是 10、次裝置號自動分配的字元裝置，一個 `misc_register()` 就出現 `/dev/twn_fuse`。
- `struct file_operations`：把 `open`、`mmap`、`ioctl` 等系統呼叫接到 driver 的函式。
- `_IO` / `_IOR` / `_IOW` / `_IOWR`：把方向、參數大小、類別、編號編進一個 32-bit 指令碼的巨集。
- uapi header `twn_fuse_ioctl.h`：driver 與 app 共用，只用 `__u32`、`__u64` 這種兩邊都有的固定寬度型別。
- `copy_from_user` / `copy_to_user` / `memdup_user`：在 user 與 kernel 記憶體之間安全地搬資料；`in_arena()`：driver 自己的範圍檢查。
- `void __user *`：標記「這是 user space 的指標」的型別，和第 2 章的 `__iomem` 一樣是給靜態檢查工具 sparse 看的，直接解這種指標會被它擋下。

改善了什麼

- 每個指令就是一段完整、順序固定的暫存器操作，app 不可能漏掉某一步。
- 寫進 PL 的位址一定落在 driver 的 buffer 裡；head 的傳輸長度已知，連「從起點到終點」整段都檢查。
- 錯誤用標準錯誤碼回報：越界回 `EINVAL`、壞指標回 `EFAULT`、不認得的指令回 `ENOTTY`。
- 介面本身幾乎不花時間：一次 ioctl 來回 1.62 µs；跑一幀時，系統呼叫、拿鎖、參數複製與讀回六個 cycle counter 合計 3.3 µs，不到幀時間的 0.1%。

5.1 實際例子

四個 file operation。沒有 `.read` / `.write`：app 呼叫 `read()` 會直接得到 `-EINVAL`。

linux/driver/twn_fuse/twn_fuse.c : 429--435

```
429 static const struct file_operations twn_fops = {
430     .owner          = THIS_MODULE,
431     .open           = twn_open,
432     .release        = twn_release,
433     .mmap           = twn_mmap,
434     .unlocked_ioctl = twn_ioctl,
435 };
```

open 用 `test_and_set_bit` 讓一次只有一個 process 能打開（第 7 章）；`file_to_td()` 用 `container_of`，從 `misc_register` 註冊的 `struct miscdevice` 反推回整個 `struct twn_dev`，所有 file operation 都靠它找到暫存器、buffer 與鎖：

linux/driver/twn_fuse/twn_fuse.c : 327--331

```
327 static struct twn_dev *file_to_td(struct file *f)
328 {
329     /* misc_open() stores the struct miscdevice in private_data */
330     return container_of(f->private_data, struct twn_dev, misc);
331 }
```

介面定義：九個指令。`0xBC` 是這個 driver 的類別（magic），Linux 6.1 的 `ioctl-number.rst` 裡沒有人登記；第三個參數是型別，巨集用 `sizeof` 把大小編進指令碼：

linux/driver/twn_fuse/twn_fuse_ioctl.h : 65--73

```
65 #define TWN_IOC_STATUS     _IOR(TWN_IOC_MAGIC, 0, struct twn_status)
66 #define TWN_IOC_SET_TABLE  _IOW(TWN_IOC_MAGIC, 1, struct twn_table)
67 #define TWN_IOC_SET_BASES  _IOW(TWN_IOC_MAGIC, 2, struct twn_bases)
68 #define TWN_IOC_BB_RUN     _IOWR(TWN_IOC_MAGIC, 3, struct twn_bb_run)
69 #define TWN_IOC_BB2HD      _IOWR(TWN_IOC_MAGIC, 4, __u32)  /* in: value to write, out: read back */
70 #define TWN_IOC_HD_SET_RQ  _IOW(TWN_IOC_MAGIC, 5, struct twn_hd_rq)
71 #define TWN_IOC_HD_COMMIT  _IOW(TWN_IOC_MAGIC, 6, struct twn_hd_desc)
72 #define TWN_IOC_HD_START   _IO(TWN_IOC_MAGIC, 7)
73 #define TWN_IOC_HD_WAIT    _IOR(TWN_IOC_MAGIC, 8, __u64)    /* out: wait -> done, nanoseconds */
```

linux/driver/twn_fuse/twn_fuse_ioctl.h : 45--50

```
45 struct twn_bb_run {
46     __u64 in_off;      /* input image */
47     __u64 out_off;     /* output pointer (with BB2HD on: the head's input slots) */
48     __u64 ns;          /* out: START -> done, nanoseconds */
49     __u32 cycles[6];   /* out: total, array, depthwise, memory stall, idle before, idle after */
50 };
```

最簡單的完整範例：設定 base。複製參數、檢查範圍、寫暫存器：

linux/driver/twn_fuse/twn_fuse.c : 196--207

```
196 static long twn_set_bases(struct twn_dev *td, void __user *arg)
197 {
198     struct twn_bases b;
199 
200     if (copy_from_user(&b, arg, sizeof(b)))
201         return -EFAULT;
202     if (!in_arena(td, b.w_off, 16) || !in_arena(td, b.act_off, 16))
203         return -EINVAL;
204     wr_addr(td, R_W, b.w_off);
205     wr_addr(td, R_ACT, b.act_off);
206     return 0;
207 }
```

範圍檢查。起點 16-byte 對齊（PL 一次搬一個 16-byte 的 beat）、起點在 buffer 內、長度不超出 buffer 結尾。寫成 `len <= size - off` 而不是 `off + len <= size`，是為了避免 `off + len` 在 64 bits 溢位、繞回一個很小的數而通過檢查：

linux/driver/twn_fuse/twn_fuse.c : 125--128

```
125 static bool in_arena(struct twn_dev *td, u64 off, u64 len)
126 {
127     return !(off & 0xf) && off <= td->arena_size && len <= td->arena_size - off;
128 }
```

head 的描述子：長度已知，整段都檢查。app 已經把形狀、旗標打包好，driver 只檢查三個位址範圍、寫進去、commit。operator 0 的輸入多保留 64 KB，因為 head 會在兩個輸入 slot 之間切換：

linux/driver/twn_fuse/twn_fuse.c : 274--300

```
274 static long twn_hd_commit(struct twn_dev *td, void __user *arg)
275 {
276     struct twn_hd_desc d;
277     u64 in_len;
278 
279     if (copy_from_user(&d, arg, sizeof(d)))
280         return -EFAULT;
281     if (d.op > 3)
282         return -EINVAL;
283     /* the head's transfer lengths are known, so the whole range is checked */
284     in_len = (u64)d.in_beats * 16 + (d.op == 0 ? HD_SLOT_OFF : 0);
285     if (!in_arena(td, d.in_off, in_len) ||
286         !in_arena(td, d.w_off, (u64)d.w_beats * 16) ||
287         !in_arena(td, d.out_off, (u64)d.out_beats * 16))
288         return -EINVAL;
289     wr_addr(td, R_HD_IN, d.in_off);
290     wr_addr(td, R_HD_W, d.w_off);
291     wr_addr(td, R_HD_OUT, d.out_off);
292     wr(td, R_HD_BEATS + 0x00, d.in_beats);
293     wr(td, R_HD_BEATS + 0x04, d.w_beats);
294     wr(td, R_HD_BEATS + 0x08, d.out_beats);
295     wr(td, R_HD_BEATS + 0x0c, d.shape);
```

```
    wr(td, R_HD_BEATS + 0x10, d.flags);
    wr(td, R_HD_BEATS + 0x14, d.clamp);
    wr(td, R_HD_COMMIT, d.op);      /* latches the twelve fields into operator d.op */
    return 0;
}
```

描述子表（17 個 block 各 16 個 word）用 `memdup_user` 一次複製進來（`twn_set_table()`，第 165 行），它等於 `kmalloc` 加 `copy_from_user`，用完要 `kfree`。

分派：一個 switch、兩把鎖。`STATUS` 只讀六個暫存器、不拿鎖；backbone 與 head 的指令各拿自己的 mutex（第 7 章）；不認得的指令回 `-ENOTTY`：

linux/driver/twn_fuse/twn_fuse.c : 367--427

```
static long twn_ioctl(struct file *f, unsigned int cmd, unsigned long arg)
{
    struct twn_dev *td = file_to_td(f);
    void __user *p = (void __user *)arg;
    struct mutex *lock;     /* the engine this command touches */
    long ret;

    /*
     * The two engines have separate registers, so each has its own lock and
     * the head can run while the backbone is being waited for.
     */
    switch (cmd) {
    case TWN_IOC_STATUS:
        return twn_status(td, p);       /* reads only */
    case TWN_IOC_SET_TABLE:
    case TWN_IOC_SET_BASES:
    case TWN_IOC_BB_RUN:
    case TWN_IOC_BB2HD:
        lock = &td->bb_lock;
        break;
    case TWN_IOC_HD_SET_RQ:
    case TWN_IOC_HD_COMMIT:
    case TWN_IOC_HD_START:
    case TWN_IOC_HD_WAIT:
        lock = &td->hd_lock;
        break;
    default:
        return -ENOTTY;
    }

    if (mutex_lock_interruptible(lock))
        return -ERESTARTSYS;
    switch (cmd) {
    case TWN_IOC_SET_TABLE:
        ret = twn_set_table(td, p);
        break;
    case TWN_IOC_SET_BASES:
        ret = twn_set_bases(td, p);
```

```
405         break;
406     case TWN_IOC_BB_RUN:
407         ret = twn_bb_run(td, p);
408         break;
409     case TWN_IOC_BB2HD:
410         ret = twn_bb2hd(td, p);
411         break;
412     case TWN_IOC_HD_SET_RQ:
413         ret = twn_hd_set_rq(td, p);
414         break;
415     case TWN_IOC_HD_COMMIT:
416         ret = twn_hd_commit(td, p);
417         break;
418     case TWN_IOC_HD_START:
419         ret = twn_hd_start(td);
420         break;
421     default: /* TWN_IOC_HD_WAIT */
422         ret = twn_hd_wait(td, p);
423         break;
424     }
425     mutex_unlock(lock);
426     return ret;
427 }
```

板上：該拒絕的都拒絕了。`Bad address` 就是 `EFAULT`：測試程式故意把參數指標設成 16（沒有映射的位址），kernel 回報錯誤而沒有當掉：

results/raw/linux_driver_v11_20261027/H4_board_check.log : 1,13-20（開機組 H）

```
1  == 2026-10-26 20:39 UTC  6.1.30-xilinx-v2023.2  BOOT.BIN 5d226057efc8760b  twn_fuse.ko aec4477e2c520faf  backends: drv
13 PASS SET_BASES, weights at the end of the buffer          -> Invalid argument
14 PASS SET_BASES, offset not 16-byte aligned                -> Invalid argument
15 PASS BB_RUN, input offset 1 TB                            -> Invalid argument
16 PASS HD_COMMIT, output runs one beat past the end         -> Invalid argument
17 PASS STATUS into an unmapped pointer                      -> Bad address
18 PASS SET_TABLE whose words pointer is unmapped            -> Bad address
19 PASS an ioctl number the driver does not know             -> Inappropriate ioctl for device
20 PASS SET_TABLE that the PL's check refuses                -> Invalid argument
```

板上：ioctl 要花多少時間。`twn_bench` 第 1 部分把不拿鎖的 `STATUS` 連打 100,000 次；第 2 部分同一幀跑 1000 次，比較三個時鐘：app 在 ioctl 前後讀的時間、driver 在 kernel 裡量的時間（拿到參數、檢查完之後開始計時，清完 done 停錶；介面標頭的註解寫成「START -> done」是簡寫）、PL 自己數的 cycle 除以 240 MHz：

results/raw/linux_driver_v11_20261027/E3_bench.log : 6-10（開機組 E）：用中斷時

```
6  # ./twn_bench 1000
7  ioctl STATUS: 100000 calls, 1.62 us per call
8  BB_RUN: 1000 frames, per frame: app 4146.3 us, driver 4143.0 us, PL cycles 4129.0 us
9          overhead: syscall+copy 3.3 us, wait (driver - PL cycles) 14.0 us
10         CPU time of this process: 10.7 us per frame (0.3 % of the frame time)
```

app 比 driver 多的 3.3 µs，是系統呼叫進出、拿鎖、`copy_from_user` / `copy_to_user`，以及讀回六個 cycle counter：driver 在 `twn_bb_run()` 裡先停錶、才讀 counter（第 6 章的程式）。同樣讀六個暫

存器的 STATUS 整趟只要 1.62 µs，所以這 6 次 MMIO 讀取最多約佔 3.3 µs 的一半，這次沒有拆開量。driver 比 PL 多的 14.0 µs 主要是「等」的成本，另含 START 前後的幾次暫存器存取與 block 之間的空檔，第 6 章比較輪詢與中斷時用的就是它。

5.2 原理

為什麼沒有 read / write。`read / write` 是位元組串流的介面，適合一段段流進流出的資料（序列埠、感測器 FIFO）。這顆加速器兩件事都不是：資料每幀幾百 KB，用 `read / write` 要多複製兩次，所以走 mmap；指令有結構（設表、設位址、啟動並等待），用 `ioctl` 加固定格式的 struct 最直接。kernel 提供的其他介面各有用途：

介面	適合	這個 driver
`read / write`	串流資料；可以用 `cat / echo` 操作	沒用
`ioctl`	對裝置下動作，參數是結構化的	九個指令
`mmap`	大量資料，零複製	8 MB buffer
sysfs	一個檔案一個值的屬性，給人和腳本；算正式的介面約定（ABI）	狀態與計數器（第 8 章）
debugfs	只給開發者的除錯介面，不保證穩定	暫存器傾印、短幀（第 8 章）

指令碼怎麼編。`_IOC` 把 32-bit 數字切成四段：bits 31–30 是方向，29–16 是參數大小，15–8 是類別，7–0 是編號。`TWN_IOC_BB_RUN = 0xC030BC03`：讀加寫、48 bytes、類別 `0xBC`、編號 3。struct 的大小一改，指令碼就不同，舊 app 送來的指令 driver 不認得而回 `ENOTTY`，不會讀錯資料；但只改欄位順序、大小不變時偵測不到。

為什麼一定要用 `copy_from_user`。ioctl 的 `arg` 是 app 位址空間裡的指標，kernel 不能直接解：位址可能沒有映射，直接解會 oops，`copy_from_user` 遇到這種 fault 由 kernel 的 exception table（記錄「出錯時跳到哪裡修復」的表）接手並回傳失敗，driver 據此回 `-EFAULT`；位址也可能故意指向 kernel，`copy_from_user` 先用 `access_ok` 確認整段都在 user 範圍。ARMv8.1 起有 PAN（Privileged Access Never），kernel 直接存取 user 記憶體會發生例外；這塊板子的 Cortex-A53 是 ARMv8.0，沒有 PAN，也沒有開軟體模擬，所以更要靠 `copy_from_user`。struct 裡要帶指標時（描述子表的 `words`），型別用 `__u64`，32 與 64 位元的 app 看到的版面才一樣。

為什麼只收 offset。PL 的 DMA 拿到什麼位址就存取什麼。如果 driver 接受 app 給的實體位址，等於讓任何能開 `/dev/twn_fuse` 的程式控制一個能讀寫全部記憶體的 DMA 引擎。只收 offset，檢查後加上 driver 自己的 DMA 位址，PL 收到的位址就一定在 buffer 裡。這層檢查管的是「driver 寫進暫存器的起點」：head 的三個位址連長度都檢查；backbone 只檢查起點，輸出寫多長取決於描述子表，driver 不逐項解讀；描述子表本身由 PL 解讀，欄位理論上仍可讓 PL 越界。PL 的每一筆存取都檢查，要靠 SMMU（第 11 章）。

介面沒有實作的部分。沒有 `.compat_ioctl`：struct 都是固定寬度型別，32 位元的 app 要用時，填 kernel 現成的 `compat_ptr_ioctl` 即可，這次沒有做（app 是 64 位元）。沒有 `.poll`：需要等待的指令都會在 ioctl 裡睡，app 要同時做別的事就用執行緒，或把「啟動」和「等待」拆成兩個指令（head 的 `HD_START / HD_WAIT` 就是這樣）。

5.3 面試會問

面試會問（完整版，約 1 分鐘）：ioctl 和其他介面怎麼選？為什麼要 copy_from_user、只收 offset？

我依資料的性質選：大量資料用 mmap 零複製；對裝置的動作和結構化參數用 ioctl，指令碼用 `_IOWR` 把方向和大小編進去；給人看的狀態用 sysfs，除錯用 debugfs。所以我的 driver 只有 open、release、mmap、ioctl，沒有 read / write。ioctl 的參數是 user 指標，可能沒映射、也可能故意指向 kernel，所以一律 `copy_from_user`，失敗回 `-EFAULT`；A53 沒有 PAN，硬體不會幫我擋。位址只收 buffer 內的 offset，檢查對齊與範圍，連 offset 加長度的溢位都擋掉，再加上 DMA 位址寫進硬體，app 給不出任意實體位址。板上量到一次 ioctl 1.62 µs，跑一幀的系統呼叫、複製加讀回 counter 合計 3.3 µs，介面不是瓶頸。

5.4 關鍵函式速查

名稱	位置	做什麼
`twn_fops`	`twn_fuse.c`: 429	四個 file operation
九個指令碼	`twn_fuse_ioctl.h`: 65	driver 與 app 共用的介面
`in_arena()`	`twn_fuse.c`: 125	對齊、起點、長度，含溢位防護
`twn_set_bases()`	`twn_fuse.c`: 196	最簡單的完整 ioctl
`twn_set_table()`	`twn_fuse.c`: 165	`memdup_user` 複製描述子表
`twn_hd_commit()`	`twn_fuse.c`: 274	head 描述子：三段範圍完整檢查
`twn_ioctl()`	`twn_fuse.c`: 367	依指令分派、上鎖
第 1、2 部分	`test/twn_bench.c`: 67	ioctl 與一幀的時間

6 等待完成：從輪詢到中斷

driver 啟動一幀後要等 PL 做完：v1.1 在 RTL 加了一根 level 中斷線，driver 睡在 completion 上、由中斷處理函式（ISR）叫醒；沒有中斷的舊硬體則退回每 13–50 µs 讀一次 done 位元的輪詢，兩者都有 3 秒逾時。

原本怎麼做（v1.0.0 的 app 輪詢，driver v1.0 的 kernel 輪詢）

v1.0.0 的 app 在 user space 做完整個流程：清殘留的 done、等 busy 結束、寫位址、寫 START，再用 nanosleep 輪詢（第 263 行），逾時直接 `exit(1)`（第 243、271 行）：

sw/app/pl_hal_fuse.c @ v1.0.0 (3bdd485) : 219--279

```
219 uint64_t fuse_run_frame(uint64_t in_pa, uint64_t out_pa, int yield_wait) {
220     struct timespec t0, t1;
221     static int stale_warned = 0;
222     clock_gettime(CLOCK_MONOTONIC, &t0);
223 
224     /* Clear a leftover done before starting: ap_done is cleared only by
225      * writing CTRL[1], a START does not clear it, and a stale done would end
226      * the poll below at once. Warn once per process if one is found. If the core
227      * is still busy, wait for it first: a START written while busy is dropped,
228      * not queued. */
229     {
230         uint32_t c0 = rd(F_CTRL);
231         if ((c0 & CTRL_R_DONE) && !stale_warned) {
232             fprintf(stderr, "fuse_hal: WARNING -- CTRL reads %08x before START: a "
233                             "leftover ap_done from a run that never cleared it. "
234                             "Clearing it now (printed once per process).\n", c0);
235             stale_warned = 1;
236         }
237         while (c0 & CTRL_R_BUSY) {
238             clock_gettime(CLOCK_MONOTONIC, &t1);
239             double el = (t1.tv_sec - t0.tv_sec) + 1e-9 * (t1.tv_nsec - t0.tv_nsec);
240             if (el > 3.0) {
241                 fprintf(stderr, "fuse_hal: the core stayed busy for 3 s before START -- "
242                                 "CTRL %08x STATUS %08x\n", c0, rd(F_STATUS));
243                 exit(1);
244             }
245             c0 = rd(F_CTRL);
246         }
247         wr(F_CTRL, CTRL_CLR_DONE);
248     }
249 
250     wr64(F_IN_LO,  in_pa);
251     wr64(F_OUT_LO, out_pa);
252     /* the caller's arena stores must be complete before the PL reads them */
253     fuse_wmb();
254     wr(F_CTRL, CTRL_START);
```

```
255
256     /* Wait on ap_done (READ bit 2), not core_busy (READ bit 1). There is no
257      * interrupt: twn_fuse_top has no irq port; irq_en is only stored. */
258     for (;;) {
259         uint32_t c = rd(F_CTRL);
260         if (c & CTRL_R_DONE) break;
261         if (yield_wait) {
262             struct timespec ts = { 0, 50000 };      /* 50 us */
263             nanosleep(&ts, NULL);
264         }
265         clock_gettime(CLOCK_MONOTONIC, &t1);
266         double el = (t1.tv_sec - t0.tv_sec) + 1e-9 * (t1.tv_nsec - t0.tv_nsec);
267         if (el > 3.0) {
268             fprintf(stderr, "fuse_hal: frame did not finish in 3 s -- CTRL %08x "
269                             "STATUS %08x block %u cyc %u\n",
270                     c, rd(F_STATUS), (rd(F_STATUS) >> 2) & 0x1F, rd(F_CYC_TOTAL));
271             exit(1);
272         }
273     }
274     wr(F_CTRL, CTRL_CLR_DONE);
275
276     clock_gettime(CLOCK_MONOTONIC, &t1);
277     return (uint64_t)((t1.tv_sec - t0.tv_sec) * 1000000000ull
278                       + (t1.tv_nsec - t0.tv_nsec));
279 }
```

driver v1.0 把同樣的順序搬進 kernel，等待用 `readl_poll_timeout()`，但 bitstream 沒有中斷線（block design 把 PS 的中斷輸入接成常數 0），所以仍是輪詢。代價可以量：v1.1 的 driver 以 `use_irq=0` 載入時，輪詢的程式和 driver v1.0 相同（`readl_poll_timeout`，每次最多睡 50 µs），在開機組 E 量到 driver 比 PL 多等 39.3 µs，而且這個 process 每幀花掉 465.8 µs 的 CPU 時間（11%）。

用了什麼技術

- **RTL**：新暫存器 IRQ（offset `0x0064`）放兩個致能位元；中斷輸出是暫存器輸出、level high。
- **IP 封裝與 block design**：`ipx::infer_bus_interface` 把 `interrupt` 這個 port 宣告成 Vivado 的 interrupt 介面（`SENSITIVITY = LEVEL_HIGH`），block design 接到 `ps/pl_ps_irq0`；PetaLinux 讀 XSA（Vivado 匯出的硬體描述檔）後自動在節點寫 `interrupts = <0 89 4>`。
- **kernel**：`platform_get_irq_optional()`、`devm_request_irq()`、ISR 回傳 `irqreturn_t`；`struct completion` 的 `init_completion()`、`reinit_completion()`、`complete()`、`wait_for_completion_timeout()`；沒有中斷時的 `readl_poll_timeout()`；module parameter `use_irq`。

改善了什麼

- 啟動與等待的整套順序只有一份，在 kernel 裡；逾時回 `-ETIMEDOUT` 並在 `dmesg` 印出暫存器值，呼叫端可以自己處理，重新載入描述子表就能繼續用。
- **CPU 時間**：每幀從 465.8 µs 降到 10.7 µs；**等待**：driver 比 PL 多的時間從 39.3 µs 降到 14.0 µs（開機組 E，同一次開機、同一個程式，只差 `use_irq`）。
- **吞吐量**：同一次開機交替跑三輪，中斷 233.13 fps、輪詢 231.26 fps（+0.8%），三次中斷都比三次輪詢快。
- 1000 幀剛好 1000 次中斷，沒有多出來的空中斷；device tree 沒有中斷的舊 bitstream，driver 自動退回輪詢。

6.1 硬體：RTL 加一根線

每個 clock 都把「致能 AND done」重新算一次，存進暫存器 `irq`。`ap_done` 是 backbone 做完、`hd_done_l` 是 head 做完的黏著位元（設起來後一直保持，直到被清掉），所以 `irq` 在 done 設起來的下一個 cycle 拉高，在 done 被清掉的下一個 cycle 放下；`ifdef CSR_IRQ_BREAK_MASK` 是故意做壞的版本，只給模擬用：

rtl/fuse/twn_csr.v : 329--336

```
329             // level interrupt from the registered state: it rises a cycle after a
330             // done is latched and falls a cycle after the done is cleared
331 `ifdef CSR_IRQ_BREAK_MASK
332             // negative-test form (simulation only): the enables are ignored
333             irq <= ap_done | hd_done_l;
334 `else
335             irq <= (bb_irq_en & ap_done) | (hd_irq_en & hd_done_l);
336 `endif
```

致能放在自己的 IRQ 暫存器，不沿用 CTRL 的 bit 2，因為 CTRL 每一次寫入都會把 bit 2 一起寫掉。driver 啟動寫的是 `0x1`、清 done 寫的是 `0x2`，bit 2 都是 0，若致能放在 CTRL[2]，每次啟動都會把中斷關掉：

rtl/fuse/twn_csr.v : 403--407

```
403                         // IRQ write: [0] backbone done -> irq  [1] head done -> irq
404                         A_IRQ: begin
405                             bb_irq_en <= w_hold[0];
406                             hd_irq_en <= w_hold[1];
407                         end
```

rtl/fuse/twn_csr.v : 372--383

```
372                         A_CTRL: begin
373                             // a start is refused unless the descriptor table
374                             // has been checked and found valid
375                             if (w_hold[0] && tab_checked && d_valid) ap_start <= 1'b1;
376                             if (w_hold[1]) ap_done <= 1'b0;
377                             irq_en <= w_hold[2];
378 `ifdef CSR_IRQ_BREAK_CTRL
379                             // negative-test form (simulation only): the backbone enable
380                             // follows CTRL[2], so a START or clear-done write turns it off
381                             bb_irq_en <= w_hold[2];
382 `endif
383                         end
```

gate。單元 testbench 跑三次：原設計必須 PASS，兩個故意做壞的版本（忽略致能、致能放在 CTRL[2]）必須 FAIL，證明 testbench 抓得到這兩種錯：

results/raw/linux_driver_v11_20261027/S1_csr_irq_gate.log : 2,4-7,9-15（PC）: PC 上的 Vivado xsim

```
2     good | tb_csr_irq: PASS (18 checks)
4     mask | FAIL: 2 backbone done while masked: irq = 1, expected 0
5     mask | FAIL: 2 ap_done set, no irq: IRQ reads 00000014, expected 00000004
6     mask | FAIL: 6 backbone done, backbone source off: irq = 1, expected 0
7     mask | tb_csr_irq: 3 FAILURES of 18 checks
9     ctrl | FAIL: 5 enable survives CTRL writes: irq = 0, expected 1
10    ctrl | tb_csr_irq: 1 FAILURES of 18 checks
11
12 intact            = PASS   (must be PASS)
13 CSR_IRQ_BREAK_MASK = FAIL(3)   (must NOT be PASS)
14 CSR_IRQ_BREAK_CTRL = FAIL(1)   (must NOT be PASS)
15 CSR_IRQ_GATE_SELFTEST_PASS
```

head 路徑的整合 gate 用改過的 `twn_csr.v` 重跑，4 個 operator、434,176 bytes 逐位元相同：

results/raw/linux_driver_v11_20261027/S2_hd_int_gate.log : 86,626,631,634（PC）

```
86   TOTAL  bytes compared 434176   bytes wrong 0   never written 0
626 arm 1  intact                  = PASS   bit-exact operators 4/4   (the gate)
631 arm 6  BB2HD A_BB2HD = 1       = PASS   bit-exact operators 4/4   (a gate)
634 HD_INT_GATE_PASS -- the integrated head path is bit-exact
```

從 IP 到 device tree。封裝 IP 時把 port 宣告成 interrupt 介面，沒有這一步，Vivado 只把它當一條普通的線，PetaLinux 也不會寫出中斷；block design 再把它接到 PS：

vivado/package_fuse_ip.tcl : 110--124

```
110 # The interrupt output: declare it as an interrupt interface, level high, so the
111 # block design and the device-tree generator treat it as one.
112 if {[llength [ipx::get_bus_interfaces interrupt -of_objects $core]] == 0} {
113     ipx::infer_bus_interface interrupt xilinx.com:signal:interrupt_rtl:1.0 $core
114 }
115 set irqif [ipx::get_bus_interfaces interrupt -of_objects $core]
116 if {$irqif eq ""} { puts "FATAL: no interrupt interface on port 'interrupt'"; exit 1 }
117 set p [ipx::get_bus_parameters SENSITIVITY -of_objects $irqif]
118 if {$p eq ""} { set p [ipx::add_bus_parameter SENSITIVITY $irqif] }
```

```
set_property value LEVEL_HIGH $p
set rstif [ipx::get_bus_interfaces ap_rst_n -of_objects $core]
if {$rstif ne ""} {
    set p [ipx::get_bus_parameters POLARITY -of_objects $rstif]
    if {$p ne ""} { set_property value ACTIVE_LOW $p }
}
```

vivado/build_bd_fuse.tcl : 210--212

```
# --- interrupt: the accelerator's level-high irq (twn_csr, register IRQ 0x0064) to IRQ0 ---
# pl_ps_irq0[0] is GIC SPI 89; PetaLinux writes interrupts = <0 89 4> into the node.
connect_bd_net [get_bd_pins fuse_0/interrupt] [get_bd_pins ps/pl_ps_irq0]
```

PetaLinux 產生的節點（反組譯發佈的 `system.dtb`）：`interrupts = <0x00 0x59 0x04>` 的 0 表示 SPI（Shared Peripheral Interrupt，所有 CPU 共用的外部中斷），`0x59` = 89 是 SPI 編號，4 是 `IRQ_TYPE_LEVEL_HIGH`；GIC（ARM 的中斷控制器）內部的編號是 SPI 編號加 32，所以 `/proc/interrupts` 顯示 121：

results/raw/linux_driver_v11_20261027/S3_binding_check.log : 24-34（PC）：PC 上反組譯，節點原文

```
    node twn_fuse@a0000000:
      clock-names = "ap_clk";
      clocks = <0x04 0x47>;
      compatible = "wjhuang,twn-fuse-1.1";
      interrupt-names = "interrupt";
      interrupt-parent = <0x05>;
      interrupts = <0x00 0x59 0x04>;
      reg = <0x00 0xa0000000 0x00 0x10000>;
      status = "okay";
      iommus = <0x11 0xe80 0x11 0x12c0 0x11 0x1300 0x11 0x1740>;
      phandle = <0x72>;
```

改 RTL 就得重建 bitstream，而這個設計在 240 MHz 沒有時序餘裕；第 12 章講怎麼讓它收斂。

6.2 driver：ISR、completion 與輪詢的退路

probe 時申請中斷。device tree 有中斷、且 `use_irq` 沒被關掉，就用 `devm_request_irq()` 註冊 ISR，再寫 IRQ 暫存器打開兩個致能；沒有中斷就留著輪詢：

linux/driver/twn_fuse/twn_fuse.c : 621--630

```
    /* 4. the interrupt, if the device tree gives one (hardware 1.1); else poll */
    td->irq = use_irq ? platform_get_irq_optional(pdev, 0) : -ENXIO;
    if (td->irq == -EPROBE_DEFER)   /* the interrupt controller is not ready yet */
        return td->irq;
    if (td->irq > 0) {
        ret = devm_request_irq(dev, td->irq, twn_isr, 0, dev_name(dev), td);
        if (ret)
            return ret;
        wr(td, R_IRQ, IRQ_BB | IRQ_HD);
    }
```

`platform_get_irq_optional()` 找不到中斷時不在 kernel log 印錯誤，因為對舊 bitstream 這是正常

的。它回傳 `-EPROBE_DEFER`（中斷控制器還沒準備好）時，probe 直接回傳，kernel 稍後再呼叫一次 probe，不能當成「沒有中斷」而退回輪詢。之後 `misc_register()` 若失敗，probe 先關掉致能再回傳。

ISR。看哪一個 done 設起來，就清掉它、叫醒等它的人；兩個都看完之後讀回一次 IRQ 暫存器，確保清除已經到達 PL 才返回：

linux/driver/twn_fuse/twn_fuse.c : 473--497

```
473 static irqreturn_t twn_isr(int irq, void *data)
474 {
475     struct twn_dev *td = data;
476     irqreturn_t ret = IRQ_NONE;
477 
478     if (rd(td, R_CTRL) & CTRL_DONE) {
479         wr(td, R_CTRL, CTRL_CLR_DONE);
480         complete(&td->bb_done);
481         ret = IRQ_HANDLED;
482     }
483     if (rd(td, R_HD_CTRL) & HD_DONE) {
484         wr(td, R_HD_CTRL, HD_DONE);
485         complete(&td->hd_done);
486         ret = IRQ_HANDLED;
487     }
488     if (ret == IRQ_HANDLED) {
489         td->irq_count++;
490         /*
491          * The clears above are posted writes; read the device back so they
492          * have landed, and the line has dropped, before the handler returns.
493          */
494         rd(td, R_IRQ);
495     }
496     return ret;
497 }
```

等待。`wait_done()` 有中斷時睡在 completion 上，沒有時走 `wait_bit()` 輪詢；兩者都是 3 秒逾時：

linux/driver/twn_fuse/twn_fuse.c : 140--146

```
140 static int wait_done(struct twn_dev *td, struct completion *c, u32 reg, u32 bit)
141 {
142     if (td->irq > 0)
143         return wait_for_completion_timeout(c, usecs_to_jiffies(TIMEOUT_US)) ?
144                1 : -ETIMEDOUT;
145     return wait_bit(td, reg, bit, true);
146 }
```

linux/driver/twn_fuse/twn_fuse.c : 131--137

```
131 static int wait_bit(struct twn_dev *td, u32 reg, u32 bit, bool set)
132 {
133     u32 v;
134 
135     return readl_poll_timeout(td->regs + reg, v, !!(v & bit) == set,
136                   POLL_US, TIMEOUT_US);
137 }
```

啟動一幀。等 busy 變 0（PL 在忙時寫的 START 會被丟掉）；清 done（START 不清 done，殘留的 done 會讓等待立刻結束）；`reinit_completion()`；寫兩個位址，再寫 START（`writel` 的 barrier 保證 app 寫的輸入比 START 先被 PL 看到，第 2 章）；等 done，清 done，停錶，再讀六個 cycle counter，連同耗時交回 app：

linux/driver/twn_fuse/twn_fuse.c : 209--246

```
209 static long twn_bb_run(struct twn_dev *td, void __user *arg)
210 {
211     struct twn_bb_run r;
212     u64 t0;
213     int i, ret;
214 
215     if (copy_from_user(&r, arg, sizeof(r)))
216         return -EFAULT;
217     if (!in_arena(td, r.in_off, 16) || !in_arena(td, r.out_off, 16))
218         return -EINVAL;
219     t0 = ktime_get_ns();
220 
221     /* a START written while busy is dropped, so wait for idle first */
222     ret = wait_bit(td, R_CTRL, CTRL_BUSY, false);
223     if (ret)
224         return ret;
225     wr(td, R_CTRL, CTRL_CLR_DONE);  /* a stale done would end the wait at once */
226     reinit_completion(&td->bb_done);
227     wr_addr(td, R_IN, r.in_off);
228     wr_addr(td, R_OUT, r.out_off);
229     /*
230      * writel() puts a barrier before the store: the app's writes into the
231      * arena reach DDR before the PL is started.
232      */
233     wr(td, R_CTRL, CTRL_START);
234 
235     ret = wait_done(td, &td->bb_done, R_CTRL, CTRL_DONE);
236     if (ret) {
237         dev_err(td->dev, "frame timed out: CTRL %08x STATUS %08x\n",
238             rd(td, R_CTRL), rd(td, R_STATUS));
239         return ret;
240     }
241     wr(td, R_CTRL, CTRL_CLR_DONE);
```

```
    r.ns = ktime_get_ns() - t0;
    for (i = 0; i < 6; i++)
        r.cycles[i] = rd(td, R_CYCLES + 4 * i);
    return copy_to_user(arg, &r, sizeof(r)) ? -EFAULT : 0;
}
```

head 的兩個指令也一樣：`twn_hd_start()` 在 START 之前 `reinit_completion(&td->hd_done)`，`twn_hd_wait()` 呼叫 `wait_done()`。啟動與等待拆成兩個指令，app 才能在兩者之間讓 backbone 跑下一幀（第 7 章）。

6.3 板上

driver 開機時就用中斷，`/proc/interrupts` 有一條 GIC 121、Level：

results/raw/linux_driver_v11_20261027/H3_boot_state.log : 11-15（開機組 H）

```
# dmesg | grep 'twn_fuse a0000000'
[    4.795531] twn_fuse a0000000.twn_fuse: Adding to iommu group 15
[    4.819475] twn_fuse a0000000.twn_fuse: NBLK 17, 8192 KB DMA buffer at 0x000000ffff800000, /dev/twn_fuse, interrupt
# grep twn_fuse /proc/interrupts
 55:          0          0     GICv2 121 Level     a0000000.twn_fuse
```

中斷與輪詢比一比。同一次開機、同一個 `twn_bench`，先用中斷，再用 `use_irq=0` 重新載入改成輪詢：

results/raw/linux_driver_v11_20261027/E3_bench.log : 6-10,14-23（開機組 E）：前一段是中斷，後一段是輪詢

```
# ./twn_bench 1000
ioctl STATUS: 100000 calls, 1.62 us per call
BB_RUN: 1000 frames, per frame: app 4146.3 us, driver 4143.0 us, PL cycles 4129.0 us
        overhead: syscall+copy 3.3 us, wait (driver - PL cycles) 14.0 us
        CPU time of this process: 10.7 us per frame (0.3 % of the frame time)

# rmmod twn_fuse; insmod /lib/modules/6.1.30-xilinx-v2023.2/extra/twn_fuse.ko use_irq=0
# dmesg | grep 'twn_fuse a0000000' | tail -1
[   63.318609] twn_fuse a0000000.twn_fuse: NBLK 17, 8192 KB DMA buffer at 0x000000ffff800000, /dev/twn_fuse, polling
# FUSE_HEAD_PL=1 ./mbv2_fuse_test_drv . 2>&1 | grep -a -c 'final max|diff|=0'
4
# ./twn_bench 1000
ioctl STATUS: 100000 calls, 1.62 us per call
BB_RUN: 1000 frames, per frame: app 4171.9 us, driver 4168.3 us, PL cycles 4129.0 us
        overhead: syscall+copy 3.6 us, wait (driver - PL cycles) 39.3 us
        CPU time of this process: 465.8 us per frame (11.2 % of the frame time)
```

輪詢時 process 每 13–50 µs 醒來一次，用中斷時一幀只醒一次。剩下的 14.0 µs 裡有 START 前的幾次暫存器存取、中斷延遲、ISR、喚醒與排程、done 之後清 done 的那次寫入，以及 PL 在 block 之間的空檔（`cycles_total` 不算空檔）；讀六個 cycle counter 發生在 driver 停錶之後，算在 app 那 3.3 µs 裡（第 5 章）。推測空檔佔其中一部分，但沒有計數器可以拆開，沒有量。

吞吐量：同一次開機交替量。每一輪先載入中斷版、再載入輪詢版，各跑一次 120 幀的 pipeline，共三輪：

results/raw/linux_driver_v11_20261027/H7_irq_vs_poll_fps.log : 1-5,7,9,11,13,15,17,19,21,23,25（開機組 H）：交替的中斷與輪詢

```
== 2026-10-26 20:40 UTC  6.1.30-xilinx-v2023.2  BOOT.BIN 5d226057efc8760b  system.dtb bc4255b1c54ae27e  twn_fuse.ko aec4477e2c520faf
# rmmod twn_fuse; insmod /lib/modules/6.1.30-xilinx-v2023.2/extra/twn_fuse.ko ; dmesg | grep 'twn_fuse a0000000' | tail -1 | grep -o -E 'interrupt|polling'
interrupt
# FUSE_BB2HD=1 FUSE_HEAD_PL=1 ./mbv2_fuse_pipe_drv . 120 0 2>&1 | grep -a 'fps ==='
=== fused cross-frame pipeline: 120 frames in 513.2 ms = 4.28 ms/frame = 233.84 fps ===
polling
=== fused cross-frame pipeline: 120 frames in 518.7 ms = 4.32 ms/frame = 231.33 fps ===
interrupt
=== fused cross-frame pipeline: 120 frames in 515.4 ms = 4.30 ms/frame = 232.83 fps ===
polling
=== fused cross-frame pipeline: 120 frames in 518.9 ms = 4.32 ms/frame = 231.27 fps ===
interrupt
=== fused cross-frame pipeline: 120 frames in 515.7 ms = 4.30 ms/frame = 232.71 fps ===
polling
=== fused cross-frame pipeline: 120 frames in 519.1 ms = 4.33 ms/frame = 231.19 fps ===
```

每一次中斷都比每一次輪詢快，兩組都隨時間略降（233.84→232.71、231.33→231.19）。每一輪都是先中斷後輪詢，這個固定順序不能抵銷漂移；但輪詢三次之間只差 0.14 fps，遠小於兩組約 1.87 fps 的差，結論不受影響。換算成每幀週期，中斷平均 4.290 ms、輪詢 4.324 ms，短了約 35 µs（開機組 H，pipeline）。開機組 E 的 `twn_bench`（只跑 backbone、1000 幀）量到 backbone 的等待少了 25.3 µs；兩者是不同的開機與程式，推測其餘來自 head 的等待也改成中斷（沒有分開量）。

每一幀剛好一次中斷。跑 1000 幀前後，ISR 自己的計數和 kernel 的計數（`/proc/interrupts`，ISR 回傳 `IRQ_NONE` 的呼叫也算在內）都剛好加 1000：

results/raw/linux_driver_v11_20261027/E6_irqcount.log : 4-11（開機組 E）

```
# cat /sys/kernel/debug/twn_fuse/irq_count; grep twn_fuse /proc/interrupts
37
 55:       1841          0     GICv2 121 Level     a0000000.twn_fuse
# ./twn_bench 1000 | grep BB_RUN
BB_RUN: 1000 frames, per frame: app 4147.0 us, driver 4143.7 us, PL cycles 4129.0 us
# cat /sys/kernel/debug/twn_fuse/irq_count; grep twn_fuse /proc/interrupts
1037
 55:       2841          0     GICv2 121 Level     a0000000.twn_fuse
```

逾時在 3 秒回來。測試程式先載入一張 PL 會拒絕的描述子表，PL 因此忽略 START，done 永遠不會來；dmesg 同時印出 driver 的訊息，之後重新載入正確的表，fixture 照樣逐位元相同：

results/raw/linux_driver_v11_20261027/H4_board_check.log : 20-27（開機組 H）

```
PASS SET_TABLE that the PL's check refuses                -> Invalid argument
PASS BB_RUN with no valid table                           -> Connection timed out
     returned after 3.11 s
PASS: 0 check(s) failed
[  255.276885] twn_fuse a0000000.twn_fuse: descriptor table refused: error 1 at block 0
[  258.388942] twn_fuse a0000000.twn_fuse: frame timed out: CTRL 00000000 STATUS 000100c2
  mbv2_fuse_test_drv FUSE_HEAD_PL=1: 4/4 fixtures bit-exact
  mbv2_fuse_test_drv FUSE_HEAD_PL=0: 4/4 fixtures bit-exact
```

6.4 原理

輪詢的退路怎麼睡。`readl_poll_timeout(addr, val, cond, 50, 3000000)` 反覆「讀暫存器、檢查條件、`usleep_range(13, 50)`」，直到成立或逾時；逾時後再讀一次，避免「睡過頭的那一刻其實已經完成」被誤判。不用 `msleep(1)`：這個 kernel 的 `HZ=250`，`msleep` 至少睡一個 4 ms 的 jiffy，等於白多等一幀；`usleep_range` 用 hrtimer（高解析度計時器）。也不 busy-wait：kernel 是 `PREEMPT_NONE`，系統呼叫裡空轉 4 ms 會佔住一顆核心。`usleep_range` 的上限是硬到期時間，CPU 閒著時多半睡滿 50 µs，一幀大約醒 80 次（估計，沒有量），每次要排程進出、設定下一個 hrtimer、讀一次暫存器，這些算在 process 的 CPU 時間裡，累積起來每幀約 0.45 ms；計時器中斷本身多半在 CPU 閒置時發生，算在 idle（下一段）。

CPU 時間怎麼量。`twn_bench` 用 `getrusage(RUSAGE_SELF)`，只算這個 process 的 user 與 kernel 時間。這顆 kernel 沒開 `IRQ_TIME_ACCOUNTING`，CPU 閒置時進來的中斷（ISR 與 hrtimer）算在 idle，所以 10.7 µs 不含 ISR 本身的成本。

為什麼用 level，ISR 為什麼一定要清 done。level 中斷是「線拉高的期間一直成立」。GIC 在 ISR 結束（EOI）後重新看這條線，done 沒清，線還是高，ISR 會立刻再被呼叫，形成中斷風暴（interrupt storm）。level 的好處是不會漏：兩個 done 幾乎同時設起來，線會一直高到兩個都被處理。

為什麼清完還要讀一次。`writel()` 是 posted write：CPU 送出就繼續往下走，不等寫入真的到達 PL。ISR 若馬上返回，GIC 可能還看到舊的高電位，多進一次 ISR。讀同一個裝置的暫存器會等前面的寫入完成，讀回來時，清除的寫入已經到了 PL。這是「完成」的問題，barrier 只管「順序」，解決不了（第 2 章）。

`reinit_completion()` 為什麼放在 START 之前。completion 是一個計數加一個等待佇列：`complete()` 把計數加一並叫醒等待者，`wait_for_completion_timeout()` 在計數大於 0 時減一並返回。若歸零放在 START 之後，PL 很快做完時，ISR 的 `complete()` 可能先發生、接著被歸零，等待的人就睡到 3 秒逾時。放在 START 之前，PL 還沒開始，不可能有中斷搶先。

ISR 為什麼可以在硬體中斷裡做完。它只做幾次暫存器讀寫和 `complete()`，都不會睡，可以在 interrupt context（CPU 被中斷打斷、還沒回到任何 process 時執行的程式，不能睡眠）執行，不需要 threaded IRQ（把處理函式放到 kernel 執行緒、可以睡眠的處理方式）。

初始化、申請、打開致能的順序。probe 先 `init_completion()`，再 `devm_request_irq()`，最後打開致能；remove 先關掉致能，再由 `devm` 釋放中斷。對這條沒有共用的線，順序反過來其實也不會出現

「送到 CPU 卻沒人處理」的中斷：GIC 端的線在 `request_irq()` 裡才啟動，最後一個 handler 釋放時就關掉。這個順序真正保護的是兩件事：ISR 一被呼叫就會用到 completion，所以它必須先初始化；若中斷線和別的裝置共用，沒人清的 level 中斷會被 kernel 判成「nobody cared」而把整條線關掉。

6.5 面試會問

面試會問（完整版，約 1 分鐘）：你怎麼把輪詢改成中斷？從硬體講到軟體。

硬體上，我在控制暫存器模組加了一個暫存器輸出的 level-high 中斷，等於「致能 AND done」；致能放在獨立的暫存器，因為 CTRL 每次寫入都會覆蓋原本的 bit 2，driver 一啟動就會把中斷關掉，這個錯誤我在 testbench 裡故意做出來、確認抓得到。IP 封裝時宣告成 interrupt 介面，block design 接到 `pl_ps_irq0`，PetaLinux 就在 device tree 寫出 SPI 89、level high。driver 在 probe 用 `devm_request_irq()` 註冊 ISR；ISR 清 done、`complete()`，返回前讀回一次確保清除已經到達；等待的一邊用 `wait_for_completion_timeout()`，`reinit_completion()` 放在 START 之前避免漏掉中斷。在同一次開機上只切換 `use_irq` 比較：每幀 CPU 時間從 465.8 µs 降到 10.7 µs；另一次開機交替量三輪，pipeline 從 231.26 變 233.13 fps。device tree 沒有中斷時自動退回 `readl_poll_timeout` 輪詢。

6.6 關鍵函式速查

名稱	位置	做什麼
`irq` 暫存器	`rtl/fuse/twn_csr.v`: 335	致能 AND done，level high
interrupt 介面	`vivado/package_fuse_ip.tcl`: 110	`SENSITIVITY = LEVEL_HIGH`
接到 PS	`vivado/build_bd_fuse.tcl`: 210	`pl_ps_irq0`
`use_irq`	`twn_fuse.c`: 80	可以關掉中斷，退回輪詢
probe 的中斷段	`twn_fuse.c`: 621	申請中斷、打開致能
`twn_isr()`	`twn_fuse.c`: 473	清 done、`complete()`，返回前讀回
`wait_done()`	`twn_fuse.c`: 140	completion 或輪詢，3 秒逾時
`wait_bit()`	`twn_fuse.c`: 131	`readl_poll_timeout`，50 µs、3 s
`twn_bb_run()`	`twn_fuse.c`: 209	等閒置、清 done、寫位址、START、等 done

7 並行控制：兩把 mutex 與獨佔開啟

同一時間只有一次 open() 會成功；在開著的這一份裡，backbone 和 head 各用一把 mutex，同一個引擎的暫存器操作不會被兩個 thread 交錯執行。

原本怎麼做（v1.0.0 的 UIO 版本）

v1.0.0 完全沒有鎖，也沒有獨佔：/dev/uio4 誰都能開、開幾次都可以（第 1 章「原本怎麼做」的 fuse_hal_init()）。兩個 app 一起跑，就會互相改寫對方的描述子表和位址；兩個引擎該怎麼安排，也完全靠 app 自己遵守順序。

用了什麼技術

- struct mutex、mutex_lock_interruptible()：可以睡眠的鎖；等鎖時被訊號（例如 Ctrl-C）打斷就回傳 -EINTR。
- test_and_set_bit() / clear_bit()：不可分割（atomic）的位元操作，用來實作「只能開一次」。
- container_of()：已知某個欄位的位址，算出包含它的 struct 的位址（第 5 章的 file_to_td()）。
- .owner = THIS_MODULE：檔案開著時 module 不能卸載；suppress_bind_attrs：不提供 sysfs 的 bind / unbind。

改善了什麼

- 第二次 open() 得到 -EBUSY；裝置開著時、以及關掉之後只剩映射時，rmmod 都被拒絕（板上測過）。
- 同一個引擎的一整段暫存器操作（BB_RUN 的「等閒置、清 done、寫位址、START、等 done」）不會被另一個 thread 插進來。
- 兩個引擎各一把鎖：兩個 thread 分別驅動兩個引擎時互不阻擋。這是設計上的保證；現在的 app 由同一個 thread 依序驅動兩者，量不出差別（見「踩過的坑」）。

7.1 實際例子

獨佔開啟。in_use 的第 0 位元就是「有人開著」。test_and_set_bit 一次完成「讀舊值、設成 1」，兩個 process 同時開也只有一個會讀到 0：

linux/driver/twn_fuse/twn_fuse.c : 333--337

```
333 static int twn_open(struct inode *inode, struct file *f)
334 {
335     /* one process owns the accelerator at a time */
336     return test_and_set_bit(0, &file_to_td(f)->in_use) ? -EBUSY : 0;
337 }
```

driver 的狀態。兩把鎖、兩個 completion（第 6 章）、`in_use`，以及內嵌的 `struct miscdevice`（`file_to_td()` 就是從它反推回整個 struct）：

linux/driver/twn_fuse/twn_fuse.c : 88--103

```
88  struct twn_dev {
89      struct device *dev;
90      void __iomem *regs;     /* the control registers, mapped by ioremap */
91      void *arena;            /* the DMA buffer, CPU address */
92      dma_addr_t arena_dma;   /* the DMA buffer, the address the PL uses */
93      size_t arena_size;
94      struct mutex bb_lock;   /* backbone registers */
95      struct mutex hd_lock;   /* head registers */
96      unsigned long in_use;   /* bit 0 set while a process has the device open */
97      struct miscdevice misc; /* /dev/twn_fuse */
98      int irq;                /* > 0: the interrupt is used; otherwise the driver polls */
99      struct completion bb_done, hd_done;     /* completed by the interrupt handler */
100     u32 irq_count;          /* interrupts handled (debugfs) */
101     struct dentry *dbg;     /* /sys/kernel/debug/twn_fuse */
102     struct debugfs_regset32 regset;
103 };
```

分派時選鎖。`twn_ioctl()`（第 5 章）先判斷指令屬於哪個引擎、拿哪把鎖，再呼叫對應的函式；`STATUS` 只讀暫存器、不拿鎖，backbone 跑到一半時也能查狀態。

app 怎麼讓兩個引擎重疊。pipeline 的 producer thread 每一輪依序做三件事：啟動 head（處理上一幀）、跑 backbone（這一幀）、等 head：

sw/app/mbv2_fuse_ps.c : 500--540

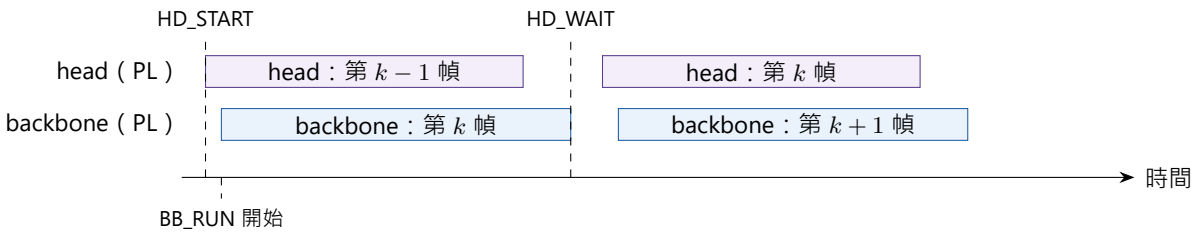
```
500 void mbv2_fuse_frame_pl(const int8_t *input_i8, int head_frame,
501                        fuse_counters_t *counters) {
502     struct timespec a, b, c;
503     uint64_t hd_ns = 0;
504     if (!g_ready) { fprintf(stderr, "mbv2_fuse: init first\n"); exit(1); }
505 
506     clock_gettime(CLOCK_MONOTONIC, &a);
507     mbv2_fuse_write_input(input_i8, g_arena_v + OFF_IN);
508     clock_gettime(CLOCK_MONOTONIC, &b);
509 
510     if (g_bb2hd) {
511         /* A sequence starts with head_frame < 0: reset the parity so backbone k
512          * writes head input slot k & 1 and head k uses box k & 1, the box
513          * mbv2_fuse_head_readout() indexes by. The boundary is already in the
514          * head's slot, so nothing is copied and op64 needs no re-commit. */
515         if (head_frame < 0) fuse_bb2hd_restart();
516         else                fuse_bb2hd_set(1);
517         if (head_frame >= 0 && fuse_hd_start() != 0) exit(1);
518     } else if (head_frame >= 0) {
519         /* the boundary the last backbone left in act slot 0, before this
520          * frame's backbone overwrites it */
521         hd_stage_boundary();
```

```
        hd_point_output(head_frame);
        if (fuse_hd_start() != 0) exit(1);
    }
    clock_gettime(CLOCK_MONOTONIC, &c);

    /* OUT_PTR is used by the fabric only with BB2HD on, as the base of the
     * head's two input slots */
    uint64_t pl_ns = fuse_run_frame(g_arena_pa + OFF_IN,
                                    g_arena_pa + (g_bb2hd ? OFF_HD_IN : OFF_OUT), 1);

    if (head_frame >= 0 && fuse_hd_wait(1, &hd_ns) != 0) exit(1);
    if (head_frame >= 0 && g_bb2hd) hd_check_box(head_frame);

    g_pl_split.t_in_ms    = (b.tv_sec - a.tv_sec) * 1e3 + (b.tv_nsec - a.tv_nsec) / 1e6;
    g_pl_split.t_stage_ms = (c.tv_sec - b.tv_sec) * 1e3 + (c.tv_nsec - b.tv_nsec) / 1e6;
    g_pl_split.t_pl_ms    = pl_ns / 1e6;
    g_pl_split.t_hd_ms    = hd_ns / 1e6;
    if (counters) fuse_read_counters(counters);
}
```



兩個引擎在硬體上同時跑；CPU 這邊是同一個 thread 依序發出三個 ioctl。

板上：重複開啟與卸載都被擋下。「只剩映射」是程式先 mmap、再關掉 fd、只留下映射時試 rmmod；之後連續卸載重載五次，每次 fixture 都逐位元相同：

results/raw/linux_driver_v11_20261027/H4_board_check.log : 1,7,28-35 (開機組 H)

```
== 2026-10-26 20:39 UTC  6.1.30-xilinx-v2023.2  BOOT.BIN 5d226057efc8760b  twn_fuse.ko aec4477e2c520faf  backends: drv
PASS second open of the device (one owner at a time)      -> Device or resource busy
== 3. unload while open, while only mapped, then five unload/reload cycles
  rmmod while open: refused (rmmod: ERROR: Module twn_fuse is in use)
  rmmod while only mapped: refused (rmmod: ERROR: Module twn_fuse is in use)
  cycle 1: 4/4 fixtures bit-exact
  cycle 2: 4/4 fixtures bit-exact
  cycle 3: 4/4 fixtures bit-exact
  cycle 4: 4/4 fixtures bit-exact
  cycle 5: 4/4 fixtures bit-exact
```

踩過的坑

sysfs 的 unbind 會繞過所有保護。一開始 `bind.sh` 用 `/sys/bus/platform/drivers/twn_fuse/unbind` 把裝置交還給 UIO。後來發現 unbind 會直接呼叫 `remove()`，`devm_` 的 buffer 隨即被釋放，就算還有 app 映射著也一樣，app 之後寫進去的資料會落在已經還給系統的記憶體上（讀

程式時發現的風險，沒有實際發生）。修法是 `.suppress_bind_attrs = true`（第 1 章的 `twn_driver`），拿掉 `bind` / `unbind`；裝置要離開 driver 只剩 `rmmod`，而 `rmmod` 在檔案開著或 buffer 映射著時會被拒絕。

我先前說錯的一件事。規劃時我說「整個裝置只用一把鎖，重疊會消失，fps 會掉」。這對這個 app 不成立：`HD_START` 一下就返回、鎖跟著釋放，硬體已經在跑；接著 `BB_RUN` 拿鎖、等 backbone；最後 `HD_WAIT`。三個呼叫來自同一個 thread、依序執行，只用一把鎖硬體照樣重疊。兩把鎖真正的作用，是兩個 thread 分別驅動兩個引擎時，等其中一個不會擋住另一個。

7.2 原理

為什麼用 mutex，不用 spinlock。拿著鎖時 driver 會睡：等中斷時睡在 completion 上，輪詢時睡在 `usleep_range` 裡，一等就是好幾 ms。spinlock 拿著時不能睡（等鎖的 CPU 會一直空轉），所以只能用 mutex：等不到鎖的 thread 被排程器換下去，把 CPU 讓給別人。ISR 不拿這兩把鎖，它只清 done 與 `complete()`，所以不會有「中斷裡等 mutex」的問題。

`_interruptible` 與 `-ERESTARTSYS`。一般的 `mutex_lock` 等鎖時連 `kill` 都叫不醒。`mutex_lock_interruptible` 被訊號打斷時回傳 `-EINTR`，driver 改回傳 `-ERESTARTSYS`：這是 kernel 內部的特殊值，返回 user space 前，kernel 依訊號處理函式有沒有設 `SA_RESTART`，決定自動重做這個 ioctl，還是讓 app 收到 `EINTR`。

為什麼一次只開一次。描述子表、base 位址、BB2HD 模式都是整顆加速器共用的狀態，兩個 process 同時用就會互相蓋掉。讓整個裝置同一時間只有一次 `open()` 成功，比替每個開檔維護一份狀態簡單，也符合「一顆加速器同一時間只跑一個網路」的用法。精確地說，它擋的是第二次 `open()`：`fork()` 出來的子 process、`dup()` 出來的 fd 共用同一個開啟的檔案，不會再經過 `twn_open()`。

`rmmod` 為什麼會被拒絕。`misc_open` 取得 driver 的 `file_operations` 時，因為 `.owner = THIS_MODULE`，會把 module 的參考計數加一，檔案完全關閉時才減回去；`mmap` 建立的 VMA 也持有這個檔案。所以只要有人開著裝置或映射著 buffer，`rmmod` 回報 `Module twn_fuse is in use`。

7.3 面試會問

面試會問（完整版，約 1 分鐘）：你的 driver 怎麼處理並行？為什麼用 mutex 不用 spinlock？

兩層。第一層是獨佔開啟：open 用 `test_and_set_bit`，同一時間只有一次 open 會成功，因為描述子表和 base 位址是整顆加速器共用的狀態。第二層是 thread：backbone 和 head 各一把 mutex，同一個引擎的一整段暫存器操作不會被插隊，兩個引擎互不阻擋；只讀的 `STATUS` 不拿鎖。用 mutex 是因為拿著鎖時我會睡在 completion 或 `usleep_range` 上好幾 ms，spinlock 不能睡。我也拿掉了 sysfs 的 unbind，因為它會在 buffer 還被映射時釋放 buffer；卸載只剩 `rmmod`，板上測過裝置開著、或只剩映射時都被拒絕。

7.4 關鍵函式速查

名稱	位置	做什麼
`struct twn_dev`	`twn_fuse.c`: 88	兩把鎖、兩個 completion、`in_use`、`miscdevice`
`twn_open()`	`twn_fuse.c`: 333	獨佔開啟，第二次回 `-EBUSY`
`twn_ioctl()`	`twn_fuse.c`: 367	選鎖、上鎖、分派、解鎖
`mbv2_fuse_frame_pl()`	`mbv2_fuse_ps.c`: 500	head start → backbone run → head wait

8 看得見硬體：sysfs 與 debugfs

給人和腳本看的狀態（六個 cycle counter、STATUS、NBLK）放 sysfs，一個檔案一個值、cat 就能看；只給開發者的除錯工具（暫存器傾印、短幀、中斷計數、SMMU 測試）放 debugfs，不讓 app 依賴。

原本怎麼做（v1.0.0 的 UIO 版本，與 driver v1.0）

v1.0.0 的 counter 只能由 app 在跑完一幀後經 `/dev/uio4` 讀；沒有 app 在跑時，想看硬體狀態得另外寫程式或用 `devmem`。UIO 版本的除錯功能（只跑前 n 個 block 的「短幀」、像素相位的觀察位元、陣列的第一個輸出）都是 app 直接寫暫存器。driver v1.0 不讓 app 碰暫存器，這三個函式就失效了：

sw/app/pl_hal_fuse_drv.c @ driver v1.0 (7615324) : 91--97

```
91 int fuse_set_nblk(uint32_t n) {
92     (void)n;
93     fprintf(stderr, "fuse_drv: fuse_set_nblk is a debug aid of the UIO backend\n");
94     return -1;
95 }
96 uint32_t fuse_read_pix_nz(void) { return 0; }
97 void fuse_read_arr_cap(uint32_t out[4]) { memset(out, 0, 4 * sizeof out[0]); }
```

`fuse_set_nblk()` 至少印訊息、回傳 -1；另外兩個安靜地回傳 0，呼叫的人分不出「硬體讀到 0」和「功能不存在」，比直接失敗更容易誤導。

用了什麼技術

- sysfs：`DEVICE_ATTR_RO(name)` 宣告唯讀屬性，讀檔時呼叫 `name_show()`；`sysfs_emit()` 保證輸出不超過一頁；`ATTRIBUTE_GROUPS()` 加 `driver.dev_groups`，讓 driver core 在綁定後自動建檔、解除前移除。
- debugfs：掛在 `/sys/kernel/debug`，只有 root 能讀寫，格式不保證穩定。`debugfs_create_regset32()` 一次印出一張暫存器表；`DEFINE_DEBUGFS_ATTRIBUTE()` 加 `debugfs_create_file_unsafe()` 做可寫的數字檔；`debugfs_create_u32()` 把一個變數開成唯讀檔；`debugfs_remove_recursive()` 整個目錄一起移除。

改善了什麼

- driver 綁定期間，任何時候都能 `cat` 硬體狀態，不用開裝置；檔案的建立與移除交給 driver core，和 probe / remove 沒有先後的競爭。
- 短幀回來了，而且比 UIO 安全：寫 `nblk` 檢查 1 到 17（寫 99 被拒絕），並拿 backbone 的鎖，不會在一幀進行到一半時改掉 NBLK。
- 多了兩個工具：`irq_count`（ISR 處理了幾次，第 6 章用它驗證沒有空中斷）與 `dma_fault`（故意越界讀取，第 11 章測 SMMU）。

8.1 實際例子

sysfs：一個巨集產生一個屬性。`TWN_REG_ATTR(name, off, fmt)` 展開成一個 `name_show()`（讀一個暫存器、照格式印出）加上 `DEVICE_ATTR_RO(name)`；用它宣告八個屬性，列成表交給第 1 章 `twn_driver` 的 `.dev_groups`：

linux/driver/twn_fuse/twn_fuse.c : 439--447

```
439 #define TWN_REG_ATTR(name, off, fmt)                          \
440 static ssize_t name##_show(struct device *dev, struct device_attribute *a,  \
441                char *buf)                          \
442 {                                       \
443     struct twn_dev *td = dev_get_drvdata(dev);              \
444                                         \
445     return sysfs_emit(buf, fmt "\n", rd(td, off));              \
446 }                                       \
447 static DEVICE_ATTR_RO(name)
```

linux/driver/twn_fuse/twn_fuse.c : 449--456

```
449 TWN_REG_ATTR(cycles_total,     R_CYCLES + 0x00, "%u");
450 TWN_REG_ATTR(cycles_array,     R_CYCLES + 0x04, "%u");
451 TWN_REG_ATTR(cycles_dw,        R_CYCLES + 0x08, "%u");
452 TWN_REG_ATTR(cycles_stall,     R_CYCLES + 0x0c, "%u");
453 TWN_REG_ATTR(cycles_idle_pre,  R_CYCLES + 0x10, "%u");
454 TWN_REG_ATTR(cycles_idle_post, R_CYCLES + 0x14, "%u");
455 TWN_REG_ATTR(status,           R_STATUS,        "0x%08x");
456 TWN_REG_ATTR(nblk,             R_NBLK,          "%u");
```

linux/driver/twn_fuse/twn_fuse.c : 458--468

```
458 static struct attribute *twn_attrs[] = {
459     &dev_attr_cycles_total.attr,
460     &dev_attr_cycles_array.attr,
461     &dev_attr_cycles_dw.attr,
462     &dev_attr_cycles_stall.attr,
463     &dev_attr_cycles_idle_pre.attr,
464     &dev_attr_cycles_idle_post.attr,
465     &dev_attr_status.attr,
466     &dev_attr_nblk.attr,
467     NULL,
468 };
```

板上：跑完一幀後讀出來。

results/raw/linux_driver_v11_20261027/H10_driver_state.log : 25-34（開機組 H）

```
25 # FUSE_HEAD_PL=1 ./mbv2_fuse_test_drv . > /dev/null 2>&1
26 # for a in nblk status cycles_total cycles_array cycles_dw cycles_stall cycles_idle_pre cycles_idle_post; do printf
       '%-18s %s\n' $a "$(cat /sys/bus/platform/devices/a0000000.twn_fuse/$a)"; done
27 nblk               17
28 status             0x00010043
29 cycles_total       990937
30 cycles_array       976076
31 cycles_dw          472744
32 cycles_stall       167
33 cycles_idle_pre    2204
34 cycles_idle_post   6584
```

PL 時脈 240 MHz。`cycles_total` 是 17 個 block 執行期間的 cycle 總和（block 之間的空檔不算），990,937 cycles 約 4.13 ms，是 START 到 done 的下限；`cycles_array` 佔 98.5%，三值運算陣列幾乎一直在工作；`cycles_stall` 只有 167，記憶體幾乎沒有讓運算停下來。`status` 的欄位：

位元	名稱	這次的值與意思
0、1	`d_valid`、`tab_checked`	1、1：描述子表通過檢查、檢查已經跑過
6:2	`cur_block`	16：最後執行的 block（0 到 16）
10:7、15:11	`err_code`、`err_block`	0、0：表格沒有錯
16	`ovr_sticky`	1：見下段

STATUS bit 16 是 ring overrun 的黏著旗標：PL 裡某個列緩衝區在滿的時候被寫入，就會設成 1（只

記錄、不停機），只有 PL reset 會清掉。它是 bitstream 本身在正常執行時就會設起來的，和 driver 無關：v1.0.0 時期回退到原本的 dtb、driver 完全沒載入，只用 UIO 版本跑 4 張 fixture，跑之前是 0、跑完變成 `0x00010043`，輸出照樣逐位元正確：

results/raw/linux_driver_20261026/C1_rollback_to_uio_dtb.txt : 10-21（開機組 C）

```
# lsmod | grep -c "^twn_fuse"
0
# devmem 0xA000002C 32
0x00000000
# FUSE_HEAD_PL=1 ./mbv2_fuse_test_uio . 2>&1 | grep -aE "^fixture|BIT-EXACT"
fixture 0: PASS | final max|diff|=0 | kpts 14/14 | backbone 4.44 ms  head 5.07 ms  frame 9.51 ms
fixture 1: PASS | final max|diff|=0 | kpts 14/14 | backbone 4.36 ms  head 4.59 ms  frame 8.95 ms
fixture 2: PASS | final max|diff|=0 | kpts 14/14 | backbone 4.37 ms  head 4.61 ms  frame 8.98 ms
fixture 3: PASS | final max|diff|=0 | kpts 14/14 | backbone 4.36 ms  head 4.58 ms  frame 8.95 ms
== FUSED BACKBONE BIT-EXACT vs golden (4 fixtures) ==
# devmem 0xA000002C 32
0x00010043
```

原因還沒查明，記錄在 `results/board_results.json`，留待之後追查。

debugfs：建立目錄與檔案。probe 的最後一步呼叫 `twn_debugfs_init()`；`regs` 背後是一張「名字、offset」的表，可以寫的 `nblk` 在寫入時檢查範圍、拿 backbone 的鎖：

linux/driver/twn_fuse/twn_fuse.c : 564--574

```
static void twn_debugfs_init(struct twn_dev *td)
{
    td->regset.regs  = twn_dbg_regs;
    td->regset.nregs = ARRAY_SIZE(twn_dbg_regs);
    td->regset.base  = td->regs;
    td->dbg = debugfs_create_dir("twn_fuse", NULL);
    debugfs_create_regset32("regs", 0444, td->dbg, &td->regset);
    debugfs_create_file_unsafe("nblk", 0644, td->dbg, td, &twn_nblk_fops);
    debugfs_create_u32("irq_count", 0444, td->dbg, &td->irq_count);
    debugfs_create_file_unsafe("dma_fault", 0200, td->dbg, td, &twn_fault_fops);
}
```

linux/driver/twn_fuse/twn_fuse.c : 508--518

```
static int twn_nblk_set(void *data, u64 val)
{
    struct twn_dev *td = data;

    if (val < 1 || val > NBLK_RESET)
        return -EINVAL;
    mutex_lock(&td->bb_lock);
    wr(td, R_NBLK, val);
    mutex_unlock(&td->bb_lock);
    return 0;
}
```

app 這一側。HAL 用 `fopen` / `fgets` 讀 `regs`、找出某個名字的那一行；設定短幀就是寫 `nblk` 再讀回確認：

sw/app/pl_hal_fuse_drv.c : 95--105

```c
static uint32_t dbg_reg(const char *name) {
    char line[128], key[32];
    unsigned int v = 0;
    FILE *f = fopen(TWN_DEBUGFS "regs", "r");
    if (!f) { perror("open " TWN_DEBUGFS "regs"); return 0; }
    while (fgets(line, sizeof line, f))
        if (sscanf(line, "%31s = %x", key, &v) == 2 && !strcmp(key, name)) { fclose(f); return v; }
    fclose(f);
    fprintf(stderr, "fuse_drv: no %s in " TWN_DEBUGFS "regs\n", name);
    return 0;
}
```

sw/app/pl_hal_fuse_drv.c : 107--113

```c
int fuse_set_nblk(uint32_t n) {
    FILE *f = fopen(TWN_DEBUGFS "nblk", "w");
    if (!f) { perror("open " TWN_DEBUGFS "nblk"); return -1; }
    int bad = fprintf(f, "%u\n", n) < 0;
    bad |= fclose(f) != 0;              /* the driver's range check reports here */
    return (bad || dbg_reg("NBLK") != n) ? -1 : 0;
}
```

板上。`IRQ = 0x00000003` 表示兩個中斷致能都打開了：

results/raw/linux_driver_v11_20261027/E4_debugfs.log : 4-25 (開機組 E)

```
# ls /sys/kernel/debug/twn_fuse
dma_fault
irq_count
nblk
regs
# cat /sys/kernel/debug/twn_fuse/regs
CTRL = 0x00000000
NBLK = 0x00000011
STATUS = 0x00010043
PIX_NZ = 0x00000000
CAP0 = 0x00000000
CAP1 = 0x00000000
CAP2 = 0x00000000
CAP3 = 0x00000000
BB2HD = 0x00000013
IRQ = 0x00000003
HD_CTRL = 0x00000000
HD_COMMIT = 0x000000f3
HD_RQ_STAT = 0x0000100f
# cat /sys/kernel/debug/twn_fuse/nblk /sys/kernel/debug/twn_fuse/irq_count
17
0
```

接著用短幀跑一次，再試一個不合法的值：

results/raw/linux_driver_v11_20261027/E4_debugfs.log : 26-36（開機組 E）

```
# FUSE_NBLK=5 FUSE_HEAD_PL=1 ./mbv2_fuse_test_drv . 2>&1 | grep -a -E 'SHORT FRAME|pix_nz|arr_cap' | head -3
== SHORT FRAME: 5 of 17 blocks. Fixture verdicts below are MEANINGLESS; read the dump. ==
    pix_nz: array .... | bank_wr .... | bank_rd ....   (0x000)
    arr_cap: pix0 00000000  pix1 00000000  pix2 00000000  pix3 00000000
# cat /sys/kernel/debug/twn_fuse/nblk
5
# echo 99 > /sys/kernel/debug/twn_fuse/nblk; echo exit $?
sh: line 1: echo: write error: Invalid argument
exit 1
# echo 17 > /sys/kernel/debug/twn_fuse/nblk; cat /sys/kernel/debug/twn_fuse/nblk
17
```

`PIX_NZ` 與 `CAP0`–`CAP3` 都是 0：這兩組觀察邏輯只在 RTL 用 `FUSE_OBS` 編譯時才會建出來，發佈的 bitstream 沒有，暫存器固定讀到 0。debugfs 只是讓它們讀得到。

8.2 原理

這些檔案的 read / write 是誰寫的。app 讀 `regs`、寫 `nblk`，底層確實是 `read()` / `write()` 系統呼叫，但那組 file operations 是 kernel 產生的：`regs` 由 `debugfs_create_regset32()` 處理，只用到 driver 給的名字與 offset 表；`nblk` 由 `DEFINE_DEBUGFS_ATTRIBUTE()` 展開，寫入時才呼叫 driver 的 `twn_nblk_set()`；sysfs 的屬性檔讀取時呼叫 driver 的 `show()`。「文字轉數字、處理 offset 與長度、`copy_to_user`」這些瑣事都由 kernel 做，driver 只寫取值與設值。這也是 `/dev/twn_fuse` 不需要自己的 `.read` / `.write` 的原因之一（第 5 章）。

為什麼用 `.dev_groups`，不在 `probe()` 裡自己建檔。driver core 在 `probe()` 成功之後才建立 `dev_groups` 的檔案，在呼叫 `remove()` 之前先移除，所以 `show()` 被呼叫時，`dev_get_drvdata()` 一定拿得到 probe 設好的資料。

為什麼除錯工具放 debugfs。各種介面怎麼選在第 5 章。sysfs 的檔案一旦給人用，改格式就是改了 ABI（給使用者的介面約定）；除錯工具不希望 app 依賴，所以放 debugfs：kernel 文件說它「不打算當成給 user space 的穩定 ABI」（`Documentation/filesystems/debugfs.rst`），同時也提醒最好還是當成要長期維護的介面來設計。

為什麼用 `_unsafe` 版本。`debugfs_create_file()` 每次讀寫都多包一層保護，防止檔案被移除後還被呼叫；`DEFINE_DEBUGFS_ATTRIBUTE()` 產生的 file operations 自己已經處理了這件事，所以搭配 `debugfs_create_file_unsafe()` 避免重複保護（`fs/debugfs/inode.c` 的註解）。

為什麼寫 `nblk` 要拿鎖。NBLK 決定 PL 這一幀要跑幾個 block，一幀進行中被改，PL 的 sequencer 可能在中途看到不同的值。`TWN_IOC_BB_RUN` 從頭到尾都拿著 `bb_lock`，所以只要沒有發生過逾時，拿到鎖就表示沒有幀在跑；逾時的 ioctl 會釋放鎖，但 RTL 沒有中止功能，PL 若其實還在跑，這個保證就不成立。

8.3 面試會問

面試會問（完整版，約 1 分鐘）：除了 ioctl，你怎麼讓人看到硬體狀態、怎麼提供除錯功能？

狀態用 sysfs：六個 cycle counter、STATUS、NBLK 各是一個唯讀檔，一個巨集產生 show 加 DEVICE_ATTR_RO，再用 ATTRIBUTE_GROUPS 交給 platform driver 的 .dev_groups，driver core 在 probe 成功後建檔、remove 前移除。板上跑完一幀，cat cycles_total 是約 99 萬個 cycle、4.13 ms，運算陣列忙碌 98.5%。除錯工具放 debugfs，不讓 app 依賴：debugfs_create_regset32() 一次印出暫存器；可寫的 nblk 用 DEFINE_DEBUGFS_ATTRIBUTE()，寫入時檢查範圍、拿 backbone 的 mutex，所以比 UIO 時代直接寫暫存器安全。這些檔案的 read / write 都由 kernel 提供，我只寫取值與設值的函式。

8.4 關鍵函式速查

名稱	位置	做什麼
TWN_REG_ATTR	twn_fuse.c: 439	一個巨集產生 show() 與 DEVICE_ATTR_RO
twn_attrs[]	twn_fuse.c: 458	八個 sysfs 屬性與 ATTRIBUTE_GROUPS(twn)
twn_debugfs_init()	twn_fuse.c: 564	建 debugfs 目錄與四個檔案
twn_nblk_set()	twn_fuse.c: 508	範圍檢查、拿鎖、寫 NBLK
dbg_reg()	pl_hal_fuse_drv.c: 95	app 從 regs 讀一個暫存器
fuse_set_nblk()	pl_hal_fuse_drv.c: 107	app 設定短幀

9 App 端：同一份 HAL 介面，以及影像怎麼進 buffer

app 的演算法與位址計算沒有改：pl_hal_fuse.h 有兩個實作，編譯時用 HAL=drv（預設）或 HAL=uio 選一個；driver 版本回報「buffer 基底位址是 0」，app 算出的位址就自動變成 offset。

原本怎麼做（v1.0.0 的 UIO 版本）

v1.0.0 的 Makefile.cross 只有一個實作可選（pl_hal_fuse.c）：

sw/app/Makefile.cross @ v1.0.0 (3bdd485) : 20--20

```
20 COMMON = mbv2_fuse_ps.c pl_hal_fuse.c mbv2_ps.c pl_hal_zynq.c
       ../../models/mbv2_pose/model_meta.c ../net/mbv2_pose.c
```

pl_hal_fuse.c 裡混著兩種程式：要碰暫存器的（UIO、/dev/mem），和完全不碰硬體的（head 描述子的編碼與檢查、印 counter）。app（mbv2_fuse_ps.c）只呼叫這份介面、不直接碰硬體，這個分層正是能換實作的原因。

用了什麼技術

- HAL（Hardware Abstraction Layer）：上層只依賴一份 header，底下可以換不同的實作；Makefile.cross 用 HAL 變數決定編哪一個 .c。
- 重構：不碰硬體的函式搬到 pl_hal_fuse_enc.c，兩個實作共用，內容一字不改。

- 「基底 0」：driver 版本的 `fuse_arena_init()` 回報實體位址 0。
- `_Static_assert`：編譯時期的檢查，buffer 配置超過 6 MB 就編譯失敗。

改善了什麼

- 兩個版本跑的是同一份 app，量到的差異只來自 HAL 底下那一層，比較才有意義（第 3、13 章）。
- UIO 版本保留下來，給 v1.0.0 的開機檔與 A/B 實驗用；v1.1 的映像只用 driver 版本。
- app 本身只改了一處：要求的 buffer 從整塊 256 MB 改成實際需要的 6 MB。

9.1 實際例子

在 Makefile 選實作。HAL 是 drv 就編 `pl_hal_fuse_drv.c`，否則編 `pl_hal_fuse.c`；輸出放在不同資料夾：

sw/app/Makefile.cross : 19--23

```
19 # HAL=drv (default since v1.1): the twn_fuse kernel driver, linux/driver/twn_fuse (pl_hal_fuse_drv.c)
20 # HAL=uio: registers through UIO, buffer through /dev/mem (pl_hal_fuse.c), for a v1.0.0 image
21 HAL     ?= drv
22 HAL_SRC  = $(if $(filter drv,$(HAL)),pl_hal_fuse_drv.c,pl_hal_fuse.c)
23 OUT     ?= $(if $(filter drv,$(HAL)),../build/arm-drv,../build/arm)
```

每個 HAL 函式都變成一個 ioctl。共用的錯誤處理，與跑一幀的函式：

sw/app/pl_hal_fuse_drv.c : 40--44

```
40 static int xioctl(unsigned long cmd, void *arg, const char *what) {
41     if (ioctl(g_fd, cmd, arg) == 0) return 0;
42     fprintf(stderr, "fuse_drv: %s: %s\n", what, strerror(errno));
43     return -1;
44 }
```

sw/app/pl_hal_fuse_drv.c : 120--126

```
120 uint64_t fuse_run_frame(uint64_t in_pa, uint64_t out_pa, int yield_wait) {
121     (void)yield_wait;                   /* the driver sleeps until done: interrupt or polling */
122     struct twn_bb_run r = { .in_off = in_pa, .out_off = out_pa };
123     if (xioctl(TWN_IOC_BB_RUN, &r, "BB_RUN")) exit(1);
124     memcpy(g_cycles, r.cycles, sizeof g_cycles);
125     return r.ns;
126 }
```

和 UIO 版本 61 行的 `fuse_run_frame()`（第 6 章）比，這裡只剩填參數、呼叫、存下 counter。注意其中的 `exit(1)`：`fuse_run_frame()` 的介面沒有錯誤回傳值，為了不改介面，driver 回的錯誤碼在這裡仍然變成結束程式；要讓 app 自己處理逾時，得連 `pl_hal_fuse.h` 一起改。

「基底 0」怎麼讓 app 不用改。app 算位址的方式一直是「基底加區段 offset」：

sw/app/mbv2_fuse_ps.c : 109--110

```
109     uint64_t pl_ns = fuse_run_frame(g_arena_pa + OFF_IN, g_arena_pa + OFF_OUT, 1);
110     clock_gettime(CLOCK_MONOTONIC, &c);
```

UIO 版本的 g_arena_pa 是 0x6000_0000，傳下去的是實體位址，HAL 直接寫進暫存器；driver 版本的 g_arena_pa 是 0（第 4 章），傳下去的就是 offset，HAL 原封不動交給 ioctl，driver 再加上真正的 DMA 位址。同一行 app 程式，在兩個實作底下都對。

板上：兩個版本，同樣的答案。v1.0.0 的開機檔上，同一份 app 分別連結兩個 HAL，四張合成 fixture 都逐位元相同：

results/raw/linux_driver_v11_20261027/D4_synthetic.log : 1,7-8,12,19-20,24（開機組 D）

```
1  == 2026-10-26 17:45 UTC  6.1.30-xilinx-v2023.2  BOOT.BIN 690d87cc5f4b6897  system.dtb 18fff66fcfb7798b  twn_fuse.ko
       5f77eacd80e62272
7  # FUSE_HEAD_PL=1 ./mbv2_fuse_test_uio syn 2>&1 | grep -a -E '^fixture|BIT-EXACT|MISMATCH'
8  fixture 0: PASS | final max|diff|=0 | kpts 14/14 | backbone 4.51 ms  head 5.10 ms  frame 9.61 ms
12 == FUSED BACKBONE BIT-EXACT vs golden (4 fixtures) ==
19 # FUSE_HEAD_PL=1 ./mbv2_fuse_test_drv syn 2>&1 | grep -a -E '^fixture|BIT-EXACT|MISMATCH'
20 fixture 0: PASS | final max|diff|=0 | kpts 14/14 | backbone 4.43 ms  head 5.12 ms  frame 9.55 ms
24 == FUSED BACKBONE BIT-EXACT vs golden (4 fixtures) ==
```

9.2 影像怎麼進 buffer：以攝影機 app 為例

sw/app/main_cam.c 從 USB 攝影機讀影像，經過五步才變成 PL 能吃的格式：

1. **從攝影機拿一幀**：攝影機由 kernel 的 uvcvideo driver 管，app 用 V4L2（Video4Linux2）的 ioctl 操作：VIDIOC_S_FMT 設 640×480 與格式（YUYV 或 MJPEG），VIDIOC_REQBUFS 請 kernel 配 capture buffer 並 mmap 進來，每幀 VIDIOC_DQBUF 取一格、用完 VIDIOC_QBUF 還回去。用法和 /dev/twn_fuse 一樣是「ioctl 下指令、mmap 給資料」；uvcvideo 的 .read 直接回 -EINVAL。
2. **解碼成 RGB**：YUYV 用固定小數點換算；MJPEG 用 libturbojpeg。
3. **裁切與縮放**：從畫面中間取正方形、略往下移，nearest 取樣成 256×256。
4. **量化成 int8**：訓練時的輸入是「像素 ÷ 255、減 ImageNet 平均、除標準差、除模型的輸入 scale、四捨五入」。這串浮點運算事先算成每個 channel 256 格的表（tools/gen_input_qlut.py 從模型的 model_meta.c 產生 input_qlut.h），執行時查表：

sw/app/main_cam.c : 323--340

```
323 static void prep_input(const uint8_t *rgb, int jw, int jh,
324                         int side, int ox, int oy, int8_t *inp)
325 {
326     for (int y = 0; y < 256; y++) {
327         int sy = oy + y * side / 256;
328         if (sy < 0) sy = 0; else if (sy >= jh) sy = jh - 1;
329         const uint8_t *row = rgb + (size_t)sy * jw * 3;
330         int8_t *d0 = inp + y * 256, *d1 = d0 + 256 * 256, *d2 = d1 + 256 * 256;
331         for (int x = 0; x < 256; x++) {
332             int sx = ox + x * side / 256;
333             if (sx < 0) sx = 0; else if (sx >= jw) sx = jw - 1;
```

```
            const uint8_t *s = row + (size_t)sx * 3;
            d0[x] = INPUT_QLUT[0][s[0]];      /* plane c uses channel c's table */
            d1[x] = INPUT_QLUT[1][s[1]];
            d2[x] = INPUT_QLUT[2][s[2]];
        }
    }
}
```

5. **換排列、寫進 buffer**：上一步得到的是 CHW（三張平面），PL 要的是 HWC（每個像素的 R、G、B 連在一起），每列補齊到 128 bytes 的倍數，直接寫進 mmap 來的 buffer（第 3 章的 `OFF_IN`）：

sw/app/mbv2_fuse_ps.c : 205--217

```
void mbv2_fuse_write_input(const int8_t *input_i8, uint8_t *dst) {
    enum { C = NET_IN_C, H = NET_IN_H, W = NET_IN_W };
    const size_t stride = FUSE_IN_ROW_STRIDE;
    int8_t *d = (int8_t *)dst;
    for (int y = 0; y < H; y++) {
        const int8_t *r0 = input_i8 + 0 * H * W + y * W;
        const int8_t *r1 = input_i8 + 1 * H * W + y * W;
        const int8_t *r2 = input_i8 + 2 * H * W + y * W;
        int8_t *o = d + (size_t)y * stride;
        for (int x = 0; x < W; x++) { o[0] = r0[x]; o[1] = r1[x]; o[2] = r2[x]; o += C; }
        for (size_t t = (size_t)C * W; t < stride; t++) o[t - (size_t)C * W] = 0;
    }
}
```

接著 `ioctl(TWN_IOC_BB_RUN)` 只傳這塊輸入的 offset；driver 寫 START 時，`writel` 的 barrier 保證剛寫的影像比 START 先被 PL 看到（第 2 章）。推論結束後，app 讀回 head 的 heatmap，找出 14 個關節點，畫在 `/dev/fb0`（framebuffer，又是一個 mmap 出來的記憶體）上。

PL 吃的是量化後的 256×256 int8，攝影機給的是 640×480 的 YUYV 或 JPEG，每個像素都要 CPU 處理一次，所以讓 PL 直接讀攝影機的 buffer（dma-buf 共用）在這裡沒有意義。查表這一步也沒有 fixture 驗證，因為 fixture 存的是量化好的 tensor；app 留了 `CAM_QSTAT=1` 印出量化後的範圍供人工檢查。攝影機 app 的 `--selftest`（把 fixture 走完同樣的推論並逐位元比對）只在 v1.0.0 的 UIO 版本上做過（`results/board_results.json` 的 `camera_app_selftest`）；driver 版本的攝影機 app 有編出來，但 v1.1 的板上檢查沒有跑它。

9.3 原理

為什麼在連結時選，不在執行時選。執行時選（例如函式指標表）要多一層間接呼叫，兩個實作也都會編進同一個執行檔。這裡兩個版本不會在同一個程式裡同時用到，用 Makefile 在連結時選最簡單：每個執行檔只含一個實作，函式名稱完全相同，app 看不出差別。

為什麼不乾脆讓 app 直接呼叫 ioctl。那得改動 app 裡每一處算位址、每一次啟動與等待，UIO 版本也保留不了。守住 HAL 這一層，差異都關在兩個 `.c` 裡，app 不用改，也就不會因為改 app 而引入新的錯誤。不碰硬體的編碼函式搬到共用檔、內容不改，兩個版本就不會悄悄產生不同的結果。

9.4 面試會問

面試會問（完整版，約 1 分鐘）：換成 driver 之後，上層的 app 改了多少？攝影機的影像怎麼送進去？

演算法和位址計算都沒改。app 本來就只透過 `pl_hal_fuse.h` 碰硬體，我替它寫了第二個實作，用 Makefile 的 `HAL=drv` 在連結時選。關鍵是 driver 版本回報 buffer 基底位址為 0：app 用「基底加 offset」算位址，基底是 0 時算出來的就是 offset，正好是 ioctl 要的。攝影機那邊：用 V4L2 的 ioctl 和 mmap 拿到一幀，解碼成 RGB、裁切成 256×256、每個 channel 查表量化成 int8，再轉成 PL 要的 HWC 排列直接寫進 mmap 來的 DMA buffer，最後一個 ioctl 只傳 offset。driver 寫 START 時 `writel` 的 barrier 保證影像比 START 先被 PL 看到。

9.5 關鍵函式速查

名稱	位置	做什麼
`HAL` 變數	`Makefile.cross`: 19	選 `pl_hal_fuse_drv.c` 或 `pl_hal_fuse.c`
`xioctl()`	`pl_hal_fuse_drv.c`: 40	呼叫 ioctl，失敗時印出是哪一個
`fuse_run_frame()`	`pl_hal_fuse_drv.c`: 120	一個 `TWN_IOC_BB_RUN`
`prep_input()`	`main_cam.c`: 323	裁切、縮放、查表量化
`mbv2_fuse_write_input()`	`mbv2_fuse_ps.c`: 205	CHW 轉 HWC，寫進 buffer
`pl_hal_fuse_enc.c`	`sw/app/`	兩個實作共用的編碼與檢查

10 隔離之一：限制 /dev/mem、拿掉保留區

driver 自己配 DMA buffer 之後，v1.0.0 為 `/dev/mem` 保留的 256 MB 就拿掉了；再打開 `STRICT_DEVMEM` 與 `IO_STRICT_DEVMEM`，`/dev/mem` 不能再映射 kernel 管理的記憶體（System RAM）與 driver 佔用中的暫存器。

原本怎麼做（v1.0.0 的 UIO 版本）

device tree 保留 256 MB 給 app 經 `/dev/mem` 映射（第 3 章），kernel 沒有限制 `/dev/mem`。v1.0.0 開機時的紀錄：保留區佔掉 256 MB（`MemTotal` 只有 1,773,764 kB）；`STRICT_DEVMEM` 沒開；任何 root 程式都能用 `devmem` 讀寫 DDR 和加速器的暫存器：

results/raw/linux_driver_v11_20261027/D5_boot_state.log : 26-43（開機組 D）

```
# grep -E 'MemTotal|CmaTotal|CmaFree' /proc/meminfo
MemTotal:        1773764 kB
CmaTotal:         262144 kB
CmaFree:          253376 kB
# ls /proc/device-tree/reserved-memory 2>/dev/null || echo no reserved-memory node
#address-cells
#size-cells
name
pl_arena@60000000
ranges
# zcat /proc/config.gz 2>/dev/null | grep -E 'STRICT_DEVMEM|ARM_SMMU=' || echo no /proc/config.gz
# CONFIG_STRICT_DEVMEM is not set
# devmem 0x60000000 32; echo exit $?
0x00000000
exit 0
# devmem 0xa0000008 32; echo exit $?
0x00000011
exit 0
```

用了什麼技術

- **拿掉 `reserved-memory`**：driver 的 buffer 由 DMA API 配，不再需要保留區。
- `CONFIG_STRICT_DEVMEM`：`/dev/mem` 不能存取 System RAM（`/proc/iomem` 裡標成 `System RAM` 的範圍）。
- `CONFIG_IO_STRICT_DEVMEM`：`/dev/mem` 也不能存取被 driver 佔用（busy）的 I/O 區間。
- kernel 設定片段（`.cfg`）加 `linux-xlnx_%.bbappend`：在 PetaLinux 預設的 kernel 設定上只改這幾個選項。

改善了什麼

- kernel 可用的記憶體多了約 255.5 MB：`MemTotal` 從 1,773,764 kB 變成 2,035,396 kB。兩次開機的 kernel 不同（v1.1 多開了 SMMU、`STRICT_DEVMEM` 等選項），這個差裡也混著兩顆 kernel 本身大小的變化。
- `/dev/mem` 映射 System RAM 與加速器暫存器都被拒絕（`Operation not permitted`）。
- driver 載入期間，user space 只能經過 driver 碰暫存器與 buffer，「只給 buffer、不給暫存器」（第 4 章）不能再被 `/dev/mem` 繞過。這個保證有邊界：凡是「不是 System RAM、也沒有 driver 佔用」的位址，`/dev/mem` 都還能映射，例如 root 卸載 driver 之後的暫存器、DDR 最上面 1 MB（下面實測）。
- 代價：v1.0.0 的 UIO 版本 app 在 v1.1 映像上不能跑，v1.1 的 app 一律用 driver。

10.1 實際例子

kernel 設定。PetaLinux 的 kernel recipe 把 `SRC_URI` 裡的 `.cfg` 片段合併進 kernel 設定。片段的註解說 `/dev/mem` 不能再映射「any RAM」，精確地說是 System RAM：

linux/project-spec/meta-user/recipes-kernel/linux/linux-xlnx/twn-v11.cfg : 1--11

```
# v1.1 kernel options, on top of the PetaLinux 2023.2 defaults.
#
# /dev/mem may reach only I/O ranges that no driver has claimed: with the twn_fuse driver loaded,
# neither the accelerator's registers nor any RAM can be mapped through /dev/mem any more.
CONFIG_STRICT_DEVMEM=y
CONFIG_IO_STRICT_DEVMEM=y
# The SMMU (MMU-500) driver, so that the accelerator's DMA is confined to its buffer (the
# release device tree enables the SMMU, see system-user.dtsi). A master the device tree does
# not list keeps working untranslated (bypass) instead of faulting; arm_smmu.disable_bypass=1 on
# the kernel command line makes such a master fault and report its stream ID.
CONFIG_ARM_SMMU=y
```

linux/project-spec/meta-user/recipes-kernel/linux/linux-xlnx_%.bbappend : 1--5

```
FILESEXTRAPATHS:prepend := "${THISDIR}/${PN}:"

# bsp.cfg is the empty fragment petalinux-create writes; twn-v11.cfg is ours (see that file)
SRC_URI:append = " file://bsp.cfg file://twn-v11.cfg"
KERNEL_FEATURES:append = " bsp.cfg twn-v11.cfg"
```

板上：發佈映像開機後。`reserved-memory` 節點不在了，兩個選項都開了，`devmem` 讀 DDR 與暫存器都被拒絕：

results/raw/linux_driver_v11_20261027/H3_boot_state.log : 16-31（開機組 H）

```
# grep -E 'MemTotal|CmaTotal|CmaFree' /proc/meminfo
MemTotal:        2035396 kB
CmaTotal:         262144 kB
CmaFree:          261632 kB
# ls /proc/device-tree/reserved-memory 2>/dev/null || echo no reserved-memory node
no reserved-memory node
# zcat /proc/config.gz 2>/dev/null | grep -E 'STRICT_DEVMEM|ARM_SMMU=' || echo no /proc/config.gz
CONFIG_ARM_SMMU=y
CONFIG_STRICT_DEVMEM=y
CONFIG_IO_STRICT_DEVMEM=y
# devmem 0x60000000 32; echo exit $?
devmem: mmap: Operation not permitted
exit 1
# devmem 0xa0000008 32; echo exit $?
devmem: mmap: Operation not permitted
exit 1
```

限制的邊界。`v11_check.sh devmem` 只讀不寫，依序試 System RAM 裡的一個位址、driver 載入時的暫存器、卸載後的暫存器、重新載入後的暫存器，最後是 System RAM 以外的 DDR：

results/raw/linux_driver_v11_20261027/H8_devmem.log : 1-20（開機組 H）

```
== 2026-10-26 20:42 UTC  6.1.30-xilinx-v2023.2  BOOT.BIN 5d226057efc8760b  system.dtb bc4255b1c54ae27e  twn_fuse.ko
     aec4477e2c520faf
# grep -E 'System RAM|twn_fuse' /proc/iomem
00000000-7fefffff : System RAM
a0000000-a000ffff : a0000000.twn_fuse twn_fuse@a0000000
# devmem 0x7fe00000 32; echo exit $?
devmem: mmap: Operation not permitted
exit 1
# devmem 0xa0000008 32; echo exit $?
devmem: mmap: Operation not permitted
exit 1
# rmmod twn_fuse; grep -c twn_fuse /proc/iomem; devmem 0xa0000008 32; echo exit $?
0
0x00000011
exit 0
# insmod /lib/modules/6.1.30-xilinx-v2023.2/extra/twn_fuse.ko; devmem 0xa0000008 32; echo exit $?
devmem: mmap: Operation not permitted
exit 1
# devmem 0x7ff00000 32; echo exit $?
0xFFFF0000
exit 0
```

System RAM 到 `0x7fef_ffff` 為止。System RAM 與 driver 佔用中的暫存器都被拒絕；`rmmod` 之後 `/proc/iomem` 裡沒有 `twn_fuse`，同一個暫存器讀得到（`0x11` = 17，NBLK 的 reset 值），重新載入後又被拒絕；`0x7ff0_0000` 不是 System RAM、也沒有 driver 佔用，所以讀得到。這最上面的 1 MB 不是閒置的記憶體：Vivado 建 block design 時就報告它保留給 PMU（平台管理單元）內部使用，PetaLinux 產生的 `/memory` 節點因此沒有涵蓋它。`devmem_is_allowed()` 不分讀寫，這裡只測了讀：

results/raw/linux_driver_v11_20261027/S8_pmu_ddr_reserved.log : 2-3（PC）：PC 上的 Vivado 建置訊息

```
# grep -n "PSU-0" vivado/build_bd_fuse_v11.log
559:INFO: [PSU-0] Address Range of DDR (0x7ff00000 to 0x7fffffff) is reserved by PMU for internal purpose.
```

10.2 原理

`/dev/mem` 為什麼危險。它把整個實體位址空間開給 root：kernel 的程式碼與資料、其他 process 的記憶體、所有裝置的暫存器。拿到 root 的程式能直接改 kernel，也能繞過 driver 去動硬體，driver 做的 offset 檢查（第 5 章）全被繞過。

兩個選項各擋什麼。`/dev/mem` 的 `mmap()` 對每一頁呼叫 `devmem_is_allowed()`（Linux 6.1 `lib/devmem_is_allowed.c`），參數 `pfn`（page frame number）是實體位址除以頁大小 4096：

lib/devmem_is_allowed.c（Linux 6.1，節錄）

```
int devmem_is_allowed(unsigned long pfn)
{
    if (iomem_is_exclusive(PFN_PHYS(pfn)))
        return 0;
    if (!page_is_ram(pfn))
        return 1;
    return 0;
}
```

`STRICT_DEVMEM` 讓這個函式生效：`page_is_ram()` 為真就拒絕，依據的是 kernel 管理的記憶體範圍（System RAM），不是「這是不是 DDR」。System RAM 是 `/memory` 節點的範圍，扣掉標成 `no-map` 的保留區，所以 `/memory` 以外、以及 `no-map` 保留區裡的 DDR 都不會被擋；v1.0.0 的 256 MB 保留區就是 `no-map`，就算當時開了 `STRICT_DEVMEM`，UIO 版本也還能經 `/dev/mem` 映射它。`IO_STRICT_DEVMEM` 讓 `iomem_is_exclusive()` 把 busy 的區間也算成獨佔：probe 用的 `devm_platform_ioremap_resource()` 會先佔用 `reg` 範圍（第 2 章），所以加速器的 64 KB 暫存器被擋下；driver 卸載時釋放這個區間，之後那段暫存器又能映射；重新載入、再次登記時，kernel 會撤銷 `/dev/mem` 對這段位址既有的映射（`kernel/resource.c` 的 `revoke_iomem()`）。沒有登記佔用的周邊也擋不住，例如中斷控制器 GIC 的 driver 用 `of_iomap()` 映射、沒有登記，root 仍能經 `/dev/mem` 映射它的暫存器（由原始碼推得，沒有在板上測）。要完全不給 `/dev/mem`，得關掉 `CONFIG_DEVMEM`；這個映像沒有關。

為什麼以前需要保留區，現在不需要。`/dev/mem` 只是把一個實體位址映射給 app，不會配置記憶體；要讓 kernel 不去用那塊記憶體，只能在 device tree 事先保留。driver 版本改用 DMA API，記憶體是 kernel 自己配給 driver 的，kernel 知道它被誰用，不需要保留。

少掉的 512 kB。256 MB 是 262,144 kB，`MemTotal` 只多了 261,632 kB。kernel 收回這塊記憶體時要為它建立管理結構，例如線性映射（第 3 章）的分頁表；以 4 KB 的頁、每項 8 bytes 計算，256 MB 的分頁表剛好 512 kB。這是推算，沒有逐項查證；而且比較的兩邊是不同設定的 kernel，kernel 映像本身的大小差也混在裡面，沒有拆開。

10.3 面試會問

面試會問（完整版，約 1 分鐘）：為什麼要限制 /dev/mem？限制之後 app 怎麼存取硬體？

`/dev/mem` 把整個實體位址空間開給 root，等於任何 root 程式都能改 kernel、繞過 driver 動硬體。v1.0.0 需要它，因為 app 要自己映射 DMA buffer，device tree 還得保留 256 MB。driver 版本用 DMA API 配 buffer、只把 buffer mmap 給 app，所以 app 不再需要 `/dev/mem`，保留區也拿掉了。我打開 `STRICT_DEVMEM` 擋 System RAM、`IO_STRICT_DEVMEM` 擋 driver 佔用的暫存器，板上 `devmem` 都回 `Operation not permitted`，kernel 多了大約 255.5 MB。邊界我也講得出來：不是 System RAM、也沒有 driver 佔用的位址都還能映射，板上實測卸載 driver 後的暫存器、DDR 最上面那 1 MB 的 PMU 保留區都讀得到，所以這是限制，不是把 `/dev/mem` 關掉。

10.4 關鍵函式速查

名稱	位置（meta-user/ 指 linux/project-spec/ meta-user/）	做什麼
twn-v11.cfg	meta-user/recipes-kernel/linux/linux-xlnx/twn-v11.cfg: 1	kernel 選項片段
linux-xlnx_%.bbappend	meta-user/recipes-kernel/linux/linux-xlnx_%.bbappend: 1	把片段加進 kernel recipe
devmem_is_allowed()	kernel 的 lib/devmem_is_allowed.c	/dev/mem 每一頁的檢查
iomem_is_exclusive()	kernel 的 kernel/resource.c	開了 IO 選項後，busy 的區間也算獨佔
v11_check.sh devmem	test/v11_check.sh: 143	限制的邊界（只讀）

11 隔離之二：SMMU 限制 PL 能碰的記憶體

v1.1 打開 SMMU（ARM 的 IOMMU）並登記加速器四個 HP 埠的 stream ID，讓帶著這些 ID 的 Non-secure 交易只能存取 driver 映射的那 8 MB；第一次打開沒有效果，因為 PL 送出的是 Secure 交易。

原本怎麼做（沒有 SMMU：v1.0.0 的開機檔，v1.1 的 driver）

driver 的 in_arena() 只能保證 driver 自己寫進暫存器的基底位址落在 buffer 內（第 5 章）。PL 執行時用「基底加上描述子裡的偏移」去存取 DDR，描述子由 app 提供，PL 也可能因為 RTL 的錯誤而越界。沒有 SMMU 時，越界的存取不會被任何東西擋下。v1.0.0 的開機檔上（開機組 D，沒有 SMMU），dma_fault（第 8 章）讓 PL 從 buffer 結尾之後讀輸入，結果「frame done」，沒有任何錯誤：

results/raw/linux_driver_v11_20261027/D3_debugfs.log : 68-71（開機組 D）

```
# echo 1 > /sys/kernel/debug/twn_fuse/dma_fault; echo exit $?
exit 0
# dmesg | grep -i -E 'context fault|dma_fault' | tail -2
[ 3547.884770] twn_fuse a0000000.twn_fuse: dma_fault: input at 0x0000000050100000 + 8192 KB: frame done, STATUS
     00010043
```

用了什麼技術

- **SMMU（MMU-500）**：替 CPU 以外的匯流排主控端（PS 的 DMA 周邊，以及 PL 經 HP 埠進來的交易）做位址轉換與權限檢查，概念和 CPU 的 MMU 相同。
- **stream ID**：每筆交易進 SMMU 時帶的來源編號。ZynqMP 的組成是「TBU 編號（bit 14:10）、埠的 master ID（bit 9:6）、AXI ID（bit 5:0）」。
- **device tree 的 iommus**：告訴 kernel 這個裝置用哪些 stream ID；kernel 就把裝置放進一個 IOMMU group（一組必須共用同一套轉換的裝置），並給它一個 IOMMU domain（一套轉換表）。
- **DMA API 與 IOMMU**：driver 的資料路徑不用改，dma_alloc_coherent() 在有 IOMMU 時

自動配一段 IOVA、建好映射，回傳的 `dma_addr_t` 就是 IOVA（第 3 章）。只有量測用的 `map_device` 路徑加了一個檢查：有 IOMMU 時 buffer 沒有固定的實體位址可以映射，driver 拒絕（第 4 章）。

- **AxPROT 與 TrustZone**：AXI 交易的 bit 1 標示 Secure 或 Non-secure；Linux 只設定 SMMU 的 Non-secure 那一側。
- **診斷參數 `arm_smmu.disable_bypass=1`**：沒登記的 stream ID 一律擋下並回報，用來找出真正的 stream ID。發佈的 kernel 沒有加這個參數，沒登記的 stream ID 直接通過（bypass），見「原理」最後一段。

改善了什麼

- PL 越界的存取被擋下：同一個越界讀取測試，SMMU 打開時 kernel 回報 context fault，關掉時只有「frame done」。前提是交易是 Non-secure、帶著登記過的 stream ID（「原理」最後一段）。
- 正確性與效能不受影響：同一組開機檔只換 `system.dtb`，SMMU 開與關的 4 張 fixture 都逐位元相同，pipeline 三次平均 232.58 對 232.17 fps，差距在兩次執行之間的正常變動之內。
- driver 的資料路徑不用改：DMA API 自己處理 IOVA 與頁表。
- v1.1 的發佈映像預設就打開 SMMU；PS 的 DMA 周邊也因此一起被放到 SMMU 後面（Ethernet 與 SD 卡在所有測試裡都用到，USB 與 DisplayPort 沒有測）。

11.1 第一次：打開了，卻沒有效果

`smmu_dtb.sh` 在 `system.dtb` 的副本上做兩件事：打開 SMMU 節點，並在加速器節點寫入 stream ID。第一次用的版本寫入的是我推算的三個 ID（`0xe80`、`0x1280`、`0x1680`）；下面印出的是量到 ID 之後的現行版本，註解與 `iommus` 的四個值都是量到的：

linux/driver/twn_fuse/smmu_dtb.sh : 9--25

```
# Stream ID = TBU number [14:10] | master ID of the port [9:6] | AXI ID [5:0]. Measured on the
# board (boot with arm_smmu.disable_bypass=1: the SMMU reports every stream ID it has no entry
# for): HP0 enters through TBU 3, HP1 and HP2 through TBU 4, HP3 through TBU 5, with master IDs
# 1010b, 1011b, 1100b, 1101b, and the masters drive AXI ID 0. The block design puts gmem0 on
# HP0, gmem1 on HP1, gmem2 on HP2 and gmem3 on HP3 (vivado/build_bd_fuse.tcl), so:
#   HP0 0x0e80   HP1 0x12c0   HP2 0x1300   HP3 0x1740
# A master whose stream ID is not listed is not translated (the v1.1 kernel leaves bypass on):
# it would use the buffer's I/O virtual address as a physical one, and the bit-exact check fails.
set -e
NODE=/amba_pl@0/twn_fuse@a0000000
SMMU=/axi/smmu@fd800000
[ $# = 2 ] || { echo "usage: $0 <in.dtb> <out.dtb>"; exit 2; }
cp "$1" "$2"
ph=$(fdtget -t x "$2" $SMMU phandle)
fdtput -t s "$2" $SMMU status okay
fdtput -t x "$2" $NODE iommus "$ph" e80 "$ph" 12c0 "$ph" 1300 "$ph" 1740
echo "$SMMU status: $(fdtget "$2" $SMMU status); $NODE iommus: $(fdtget -t x "$2" $NODE iommus)"
```

開機後 SMMU 正常啟動，加速器也被放進自己的 IOMMU group，buffer 位址變成 IOVA `0xff_ff80_0000`：

results/raw/linux_driver_v11_20261027/first_bitstream/F1_smmu_guessed_ids.log : 11-20 (第一版 v1.1 bitstream)

```
# ls /sys/kernel/iommu_groups/*/devices/ | grep -B1 twn_fuse
/sys/kernel/iommu_groups/15/devices/:
a0000000.twn_fuse
# dmesg | grep 'twn_fuse a0000000' | tail -1
[    4.872568] twn_fuse a0000000.twn_fuse: NBLK 17, 8192 KB DMA buffer at 0x000000ffff800000, /dev/twn_fuse, interrupt
# FUSE_HEAD_PL=1 ./mbv2_fuse_test_drv . 2>&1 | grep -a -E '^fixture|BIT-EXACT|MISMATCH'
fixture 0: FAIL | final max|diff|=15694 | kpts MISMATCH | backbone 4.40 ms  head 4.40 ms  frame 8.80 ms
fixture 1: FAIL | final max|diff|=15243 | kpts MISMATCH | backbone 4.26 ms  head 3.97 ms  frame 8.23 ms
fixture 2: FAIL | final max|diff|=15321 | kpts MISMATCH | backbone 4.26 ms  head 3.91 ms  frame 8.17 ms
fixture 3: FAIL | final max|diff|=14733 | kpts MISMATCH | backbone 4.25 ms  head 3.93 ms  frame 8.18 ms
```

但四張 fixture 全錯，而且越界讀取也沒有觸發任何 SMMU 錯誤。再用 `arm_smmu.disable_bypass=1` 開機 (沒登記的 stream ID 應該被擋下並回報)，kernel log 裡仍然一條 SMMU 錯誤都沒有。這次開機沒有保存紀錄，`first_bitstream/README.md` 只記下「沒有任何 fault」。兩個現象合起來的意思是：PL 的交易根本沒有經過 Linux 設定的那一套轉換。

原因：AXI master 送的是 Secure 交易。加速器的 AXI master 把 AxPROT 接成常數 `3'b000`，bit 1 是 0，代表 Secure。SMMU 對 Secure 與 Non-secure 交易各有一套設定；Linux 跑在 Non-secure，只能設定 Non-secure 那一套。Secure 那一套不是 Linux 設定的；從板上的結果看，它直接放行、不轉換。所以 PL 拿 IOVA 當成實體位址去存取，讀寫到錯的地方。

rtl/src/twn_axi_rd.v : 50--53

```
    assign arcache = 4'b0011;
    // [1] = 1: Non-secure. The Linux SMMU driver programs only the Non-secure stream
    // mapping; a Secure transaction (3'b000, before v1.1) is never translated or checked.
    assign arprot  = 3'b010;
```

修正是把 bit 1 設成 1 (`3'b010`：Non-secure、非特權、資料存取)。`twn_axi_rd.v` 與 `twn_axi_wr.v` 是加速器所有 AXI master 共用的模組，改這兩行就涵蓋四個 HP 埠。

11.2 第二次：找出真正的 stream ID

改成 Non-secure 交易之後，HP1、HP2、HP3 的 stream ID 仍然是我推算的，推算可能是錯的。這時 `arm_smmu.disable_bypass=1` 就派上用場：SMMU 遇到沒有登記的 stream ID 時會擋下交易，並在 global fault 的 `GFSYNR1` 欄位回報它的 stream ID（開機組 F）：

`results/raw/linux_driver_v11_20261027/F0_smmu_find_stream_ids.log : 2-3,5-8,11-12,17-22`（開機組 F）：只登記了推算的 0xe80、0x1280、0x1680

```
# cat /proc/cmdline
earlycon console=ttyPS0,115200 clk_ignore_unused root=/dev/ram0 rw uio_pdrv_genirq.of_id=generic-uio
     arm_smmu.disable_bypass=1
[   36.995236] arm-smmu fd800000.smmu: Unexpected global fault, this could be serious
[   37.002813] arm-smmu fd800000.smmu:  GFSR 0x80000002, GFSYNR0 0x00000000, GFSYNR1 0x000012c0, GFSYNR2 0x00000000
[   37.013154] arm-smmu fd800000.smmu: Unexpected global fault, this could be serious
[   37.020709] arm-smmu fd800000.smmu:  GFSR 0x80000002, GFSYNR0 0x00000000, GFSYNR1 0x00001300, GFSYNR2 0x00000000
[   37.052387] arm-smmu fd800000.smmu: Unexpected global fault, this could be serious
[   37.059943] arm-smmu fd800000.smmu:  GFSR 0x80000002, GFSYNR0 0x00000000, GFSYNR1 0x00001740, GFSYNR2 0x00000000
# dmesg | grep -o "GFSYNR1 0x[0-9a-f]*" | sort | uniq -c
      6 GFSYNR1 0x000012c0
      1 GFSYNR1 0x00001300
      3 GFSYNR1 0x00001740
# dmesg | grep -o "cbfrsynra=0x[0-9a-f]*" | sort | uniq -c
      1 cbfrsynra=0xe80
```

HP0 的交易已經經過 Non-secure 的轉換：第 22 行有一個 context fault 帶著 `cbfrsynra=0xe80`（這次開機做過哪些存取沒有記下，這一筆是哪一個存取造成的沒有查）。另外三個埠的 ID 沒有登記，SMMU 回報 global fault，沒登記的 ID 是 `0x12c0`、`0x1300`、`0x1740`。`uniq -c` 數到的 6、1、3 次加起來正好 10，等於 kernel 印這類訊息的速率上限（5 秒內最多 10 次），所以這些次數不代表各埠的流量，某個 ID 也可能被限流吃掉；`F1_smmu_measured_ids.log` 登記了四個 ID 之後一次 global fault 都沒有，補上了這個漏洞。拆開來看：TBU 4 的 master ID 1011、TBU 4 的 1100、TBU 5 的 1101，正好接在 HP0 的 1010 後面。所以 HP1 是 `0x12c0`、HP2 是 `0x1300`、HP3 是 `0x1740`；我原本以為 HP1 與 HP2 共用 `0x1280`、HP3 是 `0x1680`，兩個都錯了。

把四個 ID 都登記上（現行的 `smmu_dtb.sh`），同樣開著 `disable_bypass`：fixture 全部逐位元相同，越界讀取被擋下（context fault），global fault 一個都沒有，表示 PL 的每一筆交易都找得到登記：

results/raw/linux_driver_v11_20261027/F1_smmu_measured_ids.log : 16-21,30,37-38,51-54（開機組 F）：四個 ID 都登記

```
# FUSE_HEAD_PL=1 ./mbv2_fuse_test_drv . 2>&1 | grep -a -E '^fixture|BIT-EXACT|MISMATCH'
fixture 0: PASS | final max|diff|=0 | kpts 14/14 | backbone 4.41 ms  head 5.11 ms  frame 9.52 ms
fixture 1: PASS | final max|diff|=0 | kpts 14/14 | backbone 4.25 ms  head 4.49 ms  frame 8.75 ms
fixture 2: PASS | final max|diff|=0 | kpts 14/14 | backbone 4.30 ms  head 4.57 ms  frame 8.87 ms
fixture 3: PASS | final max|diff|=0 | kpts 14/14 | backbone 4.25 ms  head 4.54 ms  frame 8.79 ms
== FUSED BACKBONE BIT-EXACT vs golden (4 fixtures) ==
# echo 1 > /sys/kernel/debug/twn_fuse/dma_fault; echo exit $?
[   65.566206] arm-smmu fd800000.smmu: Unhandled context fault: fsr=0x402, iova=0x00000000, fsynr=0x80000,
     cbfrsynra=0xe80, cb=15
[   65.577735] twn_fuse a0000000.twn_fuse: dma_fault: input at 0x000000ffff800000 + 8192 KB: frame done, STATUS
     00010043
# cat /proc/cmdline
earlycon console=ttyPS0,115200 clk_ignore_unused root=/dev/ram0 rw uio_pdrv_genirq.of_id=generic-uio
     arm_smmu.disable_bypass=1
# dmesg | grep -c "global fault"
0
```

11.3 發佈映像：預設打開 SMMU

量到 stream ID 之後，`system-user.dtsi` 直接打開 SMMU 並登記這四個 ID（第 1 章的 `fuse_0` 節點），PetaLinux 產生的節點因此帶著 `iommus`。發佈映像開機後，加速器在 iommu group 15，DMA buffer 的位址是 IOVA；越界讀取被 SMMU 擋下、回報 context fault，`cbfrsynra=0xe80` 就是 HP0 的 stream ID：

results/raw/linux_driver_v11_20261027/H6_smmu.log : 11-15（開機組 H）

```
# ls /sys/kernel/iommu_groups/*/devices/ | grep -B1 twn_fuse
/sys/kernel/iommu_groups/15/devices/:
a0000000.twn_fuse
# dmesg | grep 'twn_fuse a0000000' | tail -1
[  266.202626] twn_fuse a0000000.twn_fuse: NBLK 17, 8192 KB DMA buffer at 0x000000ffff800000, /dev/twn_fuse, interrupt
```

results/raw/linux_driver_v11_20261027/H6_smmu.log : 30-31,37-38（開機組 H）

```
# echo 1 > /sys/kernel/debug/twn_fuse/dma_fault; echo exit $?
exit 0
[  269.471202] arm-smmu fd800000.smmu: Unhandled context fault: fsr=0x402, iova=0x00000000, fsynr=0x80000,
     cbfrsynra=0xe80, cb=15
[  269.482771] twn_fuse a0000000.twn_fuse: dma_fault: input at 0x000000ffff800000 + 8192 KB: frame done, STATUS
     00010043
```

同一個測試在 SMMU 關掉時（開機組 G）只說 frame done，buffer 位址是 CMA 裡的實體位址：

results/raw/linux_driver_v11_20261027/G4_debugfs.log : 37-40（開機組 G）：SMMU 關

```
# echo 1 > /sys/kernel/debug/twn_fuse/dma_fault; echo exit $?
exit 0
# dmesg | grep -i -E 'context fault|dma_fault' | tail -2
[   56.261995] twn_fuse a0000000.twn_fuse: dma_fault: input at 0x000000006a100000 + 8192 KB: frame done, STATUS
     00010043
```

context fault 裡的 `iova=0x00000000` 是因為 buffer 剛好在 2^{40} 下面，buffer 結尾之後的第一個位址是 2^{40}，寫進 40 bits 的位址暫存器就變成 0；不論是哪一個，都不在映射範圍內。擋下之後，下一個 app 照常執行，4 張 fixture 仍然正確（`H6_smmu.log` 第 39 行起）。

開與關的效能比較要用同一組開機檔、只換 `system.dtb`，這是開機組 E 與 G（發佈的開機檔加候選版 rootfs；它的 driver 和發佈版只差兩個 probe 錯誤路徑，不影響這裡量的東西）：

同一個映像，只差 `system.dtb`	E：SMMU 開	G：SMMU 關
pipeline，三次的平均（`E2/G2` 第 37–39 行）	232.58 fps	232.17 fps
每幀 CPU 時間，中斷（`E3/G3` 第 10 行）	10.7 µs	10.8 µs
等待延遲，中斷（第 9 行）	14.0 µs	14.3 µs
越界讀取（`E4/G4` 第 37–41 行）	context fault	沒有察覺

為什麼 SMMU 沒有拖慢 PL。每次轉換都要查頁表，但 SMMU 的 TBU 有自己的 TLB（轉換結果的快取）；PL 又是順序地串流讀寫同一塊 buffer，大部分的存取會落在剛查過的頁上。量到的吞吐量差距在正常變動之內；TLB 命中率本身沒有量，用了多大的頁也沒有查。buffer 的實體頁在 SMMU 開與關時從哪裡來，見第 3 章。

11.4 原理

SMMU 怎麼轉換。SMMU 的結構和 CPU 的 MMU 很像：每個 IOMMU domain 有一套頁表，裝置發出的位址（IOVA）經頁表轉成實體位址，沒有映射的位址會產生 context fault。kernel 的 DMA API 在有 IOMMU 時，從裝置的 DMA mask 往下配 IOVA（這裡的 mask 是 40 bits，所以 IOVA 是 `0xff_ff80_0000`，緊貼在 2^{40} 下面），再把 buffer 的實體頁映射上去。

stream ID 怎麼算。ZynqMP 的 SMMU 有 6 個 TBU（轉換單元），各自負責幾個匯流排埠。PL 經 HP0 進來的交易走 TBU 3，HP1、HP2 走 TBU 4，HP3 走 TBU 5；HP0 的 master ID 是 `1010`，HP1 到 HP3 依序是 `1011`、`1100`、`1101`（開機組 F 量到的）。AXI ID 由 RTL 決定，這個設計的 master 都送 0。所以 HP0 是 `(3 << 10) | (0b1010 << 6) = 0xe80`。

為什麼診斷時也看不到錯誤。`disable_bypass` 只影響 Non-secure 那一套設定裡「沒有登記的 stream ID」。Secure 交易根本不走這一套，所以不會被擋，也不會被回報。這種「完全沒有反應」比錯誤訊息更難查：要從「測試的結果和預期的差異，連錯誤都沒有」反推出交易沒有進到被設定的路徑。

SMMU 能保護什麼、不能保護什麼。對帶著登記過的 stream ID 的 Non-secure 交易，它保證 PL 只能存取 driver 為它映射的頁，描述子給錯的偏移、或 PL 的位址計算錯誤，都碰不到 kernel 和其他 process 的記憶體。但有兩種交易它擋不住：

- **沒有登記的 stream ID**：發佈的 kernel 沒有開 `ARM_SMMU_DISABLE_BYPASS_BY_DEFAULT`，命令列也沒有 `arm_smmu.disable_bypass=1`，所以 SMMU 對沒登記的 ID 直接放行。stream ID 的低 6 位元是 AXI ID，只要 PL 送出非 0 的 AXI ID（RTL 的錯誤，或換了另一顆 bitstream），交易就以實體位址直通整個 DDR。
- **Secure 交易**（AxPROT[1] = 0）：完全不經過 Linux 設定的轉換。這正是本章第一次失敗的情形，也是一種 RTL 錯誤。

要擋住第一種，可以在命令列加 `arm_smmu.disable_bypass=1`；發佈映像沒有這樣做，因為它會擋下

所有沒有登記 stream ID 的主控端，這個組合只在開機組 F 用過，沒有完整測過 PS 的各個周邊。第二種只能靠 RTL 本身正確。另外，SMMU 也不檢查 buffer 內部的存取是否合理（例如輸出寫到權重區），那仍然要靠 driver 的 offset 檢查。

11.5 面試會問

面試會問（完整版，約 1 分鐘）：你怎麼用 SMMU 隔離 FPGA 的 DMA？遇到什麼問題？

在 device tree 打開 SMMU，給加速器的節點寫 `iommus`，列出它 HP 埠的 stream ID；stream ID 是 TBU 編號、埠的 master ID 和 AXI ID 拼起來的。driver 的資料路徑不用改，DMA API 在有 IOMMU 時自動配 IOVA、建頁表，帶著登記過的 ID 的交易只能存取那 8 MB。第一次打開時 fixture 全錯，而且連 `disable_bypass` 都看不到任何錯誤，表示交易根本沒進 Linux 設定的那套轉換。原因是我的 AXI master 把 AxPROT 接成 0，送的是 Secure 交易，而 Linux 只設定 SMMU 的 Non-secure 那一側。把 AxPROT[1] 改成 1 之後，越界讀取會變成 context fault。後來發現另外三個埠的 stream ID 我推算錯了兩個，就用 `disable_bypass` 讓 SMMU 回報沒有登記的 ID，量出真正的值。開 SMMU 前後的吞吐量是 232.58 對 232.17 fps，看不出差別，所以發佈映像預設打開。它的限制我也會講：發佈的 kernel 對沒登記的 stream ID 是 bypass，PL 若送出非 0 的 AXI ID 或 Secure 交易，SMMU 擋不住；要更嚴格得加 `disable_bypass`，代價是每個 PS 周邊都得登記。

11.6 關鍵函式速查

名稱	位置	做什麼
`smmu_dtb.sh`	`linux/driver/twn_fuse/smmu_dtb.sh`: 9	打開 SMMU、寫入 `iommus`
`arprot`、`awprot`	`rtl/src/twn_axi_rd.v`: 53、`twn_axi_wr.v`	Non-secure 交易
`twn_fault_set()`	`twn_fuse.c`: 528	越界讀取的測試
`map_device` 的檢查	`twn_fuse.c`: 358	有 IOMMU 時拒絕 Device 映射

12 改了 RTL 就要重建：timing closure

v1.1 改了兩次 RTL（加中斷、把 AXI 交易改成 Non-secure），兩次都用 v1.0.0 發佈時的建置配方重建並在 240 MHz 收斂，第二次 route 後是 −0.032 ns，靠 route 之後的 `phys_opt_design`（strategy 內建一次，建置腳本再補兩次）補到 0.000 ns，和 v1.0.0 發佈時的情形一樣。

原本怎麼做（v1.0.0 的 bitstream）

v1.0.0 的 bitstream（`52b400785924efe0`）在 240 MHz 收斂得很勉強：第一次 route 後 WNS（Worst Negative Slack，最差路徑的時間餘裕；負的表示這條路徑來不及）是 −0.030 ns，建置腳本自動跑 post-route `phys_opt_design`，補到 0.000 ns，完全沒有餘裕。出處：`results/board_results.json` 的 `timing_bitstream_52b400785924efe0`，報告原文在 `results/raw/thesis_model_20261012/timing_impl_fuse_ternpose.rpt`。

這個設計的 LUT（look-up table，FPGA 裡實作組合邏輯的基本單元）用了 93%、block RAM 用了 97%，改一點點 RTL，place & route（placement 決定每個邏輯單元放在晶片的哪個位置，route 決定它們之間怎麼繞線）的結果就可能不同。

用了什麼技術

- **固定的建置配方**：`vivado/build.cmd` 裡的變數，與 timing 有關的是 implementation strategy（Vivado 預先組好的一整組 place & route 選項）`Performance_NetDelay_high`、HP3 拆出獨立的 interconnect（`FUSE_HP3_SPLIT`）、一個列緩衝區（`RB_C`，RTL 裡叫 RingC）做成三列深（`FUSE_CC_ROWS3`）、一組 block RAM 的位置限制（pblock，`FUSE_T1_PBLOCK`）；其餘是建出 head 引擎（`FUSE_HD_PROBE`）、套用開發板的 PS 設定（`BOARD_PS`）、跑 implementation 並寫出 bitstream（`FUSE_IMPL`、`FUSE_BIT`）。兩次都沒改。
- **post-route 恢復**：`build_bd_fuse.tcl` 在 route 之後若 WNS < 0，依序最多跑三次 `phys_opt_design`（`AggressiveExplore`、`AlternateFlowWithRetiming`、再一次 `AggressiveExplore`；directive 是同一個指令的不同最佳化模式），還不行就重新 route 再跑一次。
- **資料路徑檢查**：合成後與 route 後各查一次，25 個資料路徑實例都還在 netlist（合成產生的電路元件與連線清單）裡。
- **備案：重現加 ECO**：用同一個 commit 在另一個資料夾重建，確認 Vivado 到 placement 為止給出一模一樣的中間結果，準備在已經收斂的設計上只改那幾根常數線（ECO，engineering change order：直接修改已經 route 好的設計，不重跑整個流程）。這次沒有用到。

改善了什麼

- 最終的 bitstream（`11b4d88f583b8e23`）收斂：WNS 0.000 ns、WHS +0.008 ns，和 v1.0.0 一樣靠 post-route 恢復。
- 資源幾乎不變：LUT 44,157（v1.0.0 是 44,057，多 0.2%），block RAM tile（晶片上的記憶體區塊）145 個、DSP（硬體乘法器單元）217 個都相同。

- 可以從 repo 的原始碼重現：`build.cmd` 不需要任何手動步驟。

12.1 實際例子

建置配方。`build.cmd` 把配方寫成環境變數。兩次重建只設了 `FUSE_TAG`（`v11irq`、`v11`），讓輸出檔名不蓋掉 v1.0.0 版：

vivado/build.cmd : 26--34

```
set "FUSE_HD_PROBE=1"
set "BOARD_PS=1"
set "FUSE_IMPL=1"
set "FUSE_BIT=1"
set "TWN_FREQ=240"
set "BUILD_STRATEGY=Performance_NetDelay_high"
set "FUSE_HP3_SPLIT=1"
set "FUSE_CC_ROWS3=1"
set "FUSE_T1_PBLOCK=1"
```

post-route 恢復。第一次重建 route 完已經是正的，這段第一行就跳出；第二次重建跑了前兩次：

vivado/build_bd_fuse.tcl : 358--372

```
# The implementation strategy can end a few picoseconds below zero. Re-optimise the routed
# design in place (post-route phys_opt, then a re-route and one more pass) before giving up.
puts "POSTROUTE start: WNS $wns WHS [slack_hold]"
foreach d {AggressiveExplore AlternateFlowWithRetiming AggressiveExplore} {
    if {$wns >= 0} break
    phys_opt_design -directive $d
    set wns [slack_setup]
    puts "POSTROUTE phys_opt $d: WNS $wns WHS [slack_hold]"
}
if {$wns < 0} {
    route_design -directive AggressiveExplore -tns_cleanup
    phys_opt_design -directive AggressiveExplore
    set wns [slack_setup]
    puts "POSTROUTE re-route + phys_opt: WNS $wns WHS [slack_hold]"
}
```

最終那次建置的紀錄（只留 timing 與檢查的行；第 4 行確認 `FUSE_CC_ROWS3` 生效，第 13–14 行是 pblock 的結果，見後文）：

`results/raw/linux_driver_v11_20261027/S4_build_timing.log : 1,3-14`（PC）：PC 上的 Vivado 2023.2

```
1  == 2026-10-27 build of FUSE_TAG=v11 with vivado/build.cmd (Vivado 2023.2), bitstream 11b4d88f583b8e23, XSA
       253ca13d2e977247
3    VERDICT: all 25 datapath instances are present in the netlist.
4    CC_ROWS3 VERDICT: RB_C has 63 LUT (> 45, the depth-2 count) -- the define TOOK EFFECT
5  INFO: [Place 30-746] Post Placement Timing Summary WNS=-0.472. For the most accurate timing information please run
       report_timing.
6  INFO: [Physopt 32-619] Estimated Timing Summary | WNS=-0.116 | TNS=-69.719 |
7  INFO: [Route 35-20] Post Routing Timing Summary | WNS=-0.032 | TNS=-4.403 | WHS=0.008  | THS=0.000  |
8  INFO: [Physopt 32-668] Current Timing Summary | WNS=-0.015 | TNS=-2.572 | WHS=0.008 | THS=0.000 |
9  POSTROUTE start: WNS -0.015 WHS 0.008
10 POSTROUTE phys_opt AggressiveExplore: WNS -0.006 WHS 0.008
11 POSTROUTE phys_opt AlternateFlowWithRetiming: WNS 0.000 WHS 0.008
12 IMPL RESULT (FUSE): WNS = 0.000 ns @ 240 MHz  (TIMING MET)
13 T132 PBLOCK VERDICT: 18 RB_T1 block RAMs, columns X2:13 X3:4 X4:1
14 T132 PBLOCK VERDICT: 5 OUTSIDE X0..X2 -- the constraint DID NOT HOLD
```

由上往下讀：placement 後 −0.472 ns；placement 後的 `phys_opt` 拉到 −0.116 ns；route 後 −0.032 ns；strategy 自己在 route 後的 `phys_opt` 到 −0.015 ns；接著建置腳本的恢復：`AggressiveExplore` 到 −0.006 ns，`AlternateFlowWithRetiming` 到 0.000 ns。

三次建置比一比（都是 Vivado 2023.2、同一份配方、同一顆 xczu2cg-sfvc784-1-e）：

	v1.0.0（發佈）	v1.1 第一版	v1.1 最終
RTL 的差別		加中斷	再改 AxPROT[1]
placement 後的 WNS	公開紀錄沒有	−0.295 ns	−0.472 ns
route 後的 WNS	−0.030 ns	+0.037 ns	−0.032 ns
最後的 WNS / WHS	0.000 / 0.002 ns	+0.037 / 0.008 ns	0.000 / 0.008 ns
post-route 恢復	用到	沒用到	用到
CLB LUT	44,057	44,081	44,157
CLB Register	36,339	36,253	36,203
Block RAM tile / DSP	145 / 217	145 / 217	145 / 217

出處：v1.0.0 見「原本怎麼做」；v1.1 第一版是 `first_bitstream/S4_*`，最終是 `S4_*`（報告第 142 行是 WNS / WHS，第 10 行的 Design State 是報告產生時設計所在的步驟，從它看得出有沒有跑 post-route `phys_opt`）。表中的 CLB（configurable logic block）是放 LUT 與暫存器的基本區塊。AxPROT 是 AXI 交易附帶的保護屬性，第 11 章解釋為什麼要改它的 bit 1。

pblock 的變化。配方裡的 pblock 希望某一組 block RAM（`RB_T1`）放在第 0 到第 2 欄。第一版有 6 顆落在範圍外（`first_bitstream/S4_build_timing.log` 第 11 行），最終版有 5 顆（紀錄第 13–14 行）；v1.0.0 版沒有公開的紀錄。它在這個配方裡只是建議性的位置限制，落在外面不算錯；若之後重建 timing 收不了，這是第一個要看的地方。

備案：重現加 ECO。第二次重建 placement 後明顯比較差，當時不確定能不能收斂，所以同時在另一個資料夾，用 v1.1 第一版的 commit 重建。它的合成報告和第一版相同（日期、主機名稱、指令那幾行除外），placement 後 `phys_opt` 的第一個估計值也一模一樣（`WNS=-0.622`、`TNS=-443.497`；TNS 是所有違規路徑的負餘裕加總，`first_bitstream/S5_rebuild_determinism.log`）：到 placement 為

止，同樣的輸入給出同樣的結果。這次重建在 placement 中途就停了，所以「可以重現第一版那個 +0.037 ns 的設計」只是推測，沒有跑完驗證。計畫是重現之後，用 ECO 把 PS 端六根 AxPROT[1] 的 pin 從接地改接 VCC：這幾根線都是常數，沒有任何 timing 路徑，改了也不會動到 slack。最終的重建先收斂了，這個備案就停掉，沒有執行。

12.2 原理

只改常數，為什麼 timing 差這麼多。place & route 是一個從某個起點出發的最佳化。netlist 只要有一點不同，起點就不同，結果像換了一個亂數種子。AxPROT[1] 從 0 改成 1（`3'b000` → `3'b010`）看起來只是常數，但 interconnect 裡若有暫存器存這些位元（這是推論）：資料是 0、reset 也是 0 時，暫存器的輸出永遠是 0，合成時會被當成常數刪掉；資料改成 1 之後，reset 期間輸出 0、之後輸出 1，就不再是常數，必須保留。最終版比第一版多了 76 個 LUT，就是 netlist 確實變了的證據（哪些邏輯變了，沒有逐一比對）。

同一份配方，結果為什麼不一樣。v1.0.0 版原本的建置（`board_results.json` 的 `opt1_archived_20261008`）第一次 route 是 −0.030 ns、LUT 44,057；2026-10-11 從乾淨的 clone 重建（`rebuilt_from_clone_20261011`），第一次 route 也是 −0.030 ns、資源數相同（兩次的 bitstream 雜湊不同，相同的是 WNS 與資源數），表示同樣的輸入會得到同樣的結果；這次的 S5 到 placement 為止也是如此。但輸入一變，結果就在 −0.03 到 +0.04 ns 之間跳動。這個設計在 240 MHz 本來就沒有餘裕，所以 post-route 恢復是配方的必要部分，不是偶爾才需要的補救。

為什麼 ECO 不會影響 timing。timing 分析看的是暫存器到暫存器之間的路徑。接到常數的 pin 沒有起點暫存器，不在任何路徑上；把它從接地改接 VCC，只是換一條常數線，其他所有 cell 的位置與繞線都不變。

retiming 要再驗一次。最後一步用了 `AlternateFlowWithRetiming`，它會做 retiming：把暫存器搬過組合邏輯，讓前後兩段路徑的延遲比較平均。工具保證功能不變，但這是會改變電路結構的最佳化，所以上板後仍然重跑了全部的逐位元比對（第 13 章，開機組 E 與 H）。

12.3 面試會問

面試會問（完整版，約 1 分鐘）：改了 RTL 之後 timing 收不了，你會怎麼做？

先確定建置配方沒變，差異只來自 RTL。這個設計 LUT 用 93%、block RAM 用 97%，在 240 MHz 本來就沒什麼餘裕，所以我的建置腳本在 route 後若 WNS 小於 0，會自動依序跑幾種 `phys_opt_design`，還不行就重新 route。這次我改了兩次 RTL：加中斷那次 route 完就是 +0.037 ns；把 AxPROT[1] 從 0 改成 1 那次，只改常數，placement 卻差了 0.18 ns，route 完 −0.032 ns，靠 route 後的 `phys_opt_design`（strategy 內建一次、建置腳本再兩次）補到 0。當時我也準備了備案：先確認 Vivado 對同樣的輸入、到 placement 為止給出同樣的結果，打算重現上一版收斂的設計，再用 ECO 只改那幾根常數 pin，timing 一點都不會動。最後沒有用到，重現也沒有跑完。因為最後一步用了 retiming，上板後我重跑了全部的逐位元比對。

12.4 關鍵函式速查

名稱	位置	做什麼
建置配方	`vivado/build.cmd`: 26	strategy、時脈、pblock 等變數
post-route 恢復	`vivado/build_bd_fuse.tcl`: 358	WNS < 0 時跑 `phys_opt`，再不行重新 route
`FUSE_TAG`	`vivado/build.cmd`: 24	輸出檔名，不蓋掉 v1.0.0 版

13 板上驗證與靜態檢查：證明了什麼、沒證明什麼

`board_check.sh` 一條指令跑完四類檢查（輸出是否逐位元相同、該拒絕的有沒有拒絕、卸載重載是否正常、吞吐量），發佈映像上全部通過；v1.1 新增的機制各有 `v11_check.sh` 的一個模式；程式另外過了 `W=1` 與 `checkpatch`，官方的 `dt_binding_check` 與 `sparse` 沒有跑。

原本怎麼做（v1.0.0 的 UIO 版本）

v1.0.0 的板上檢查是 `board/deploy.sh` 的 `verify`（4 張 fixture）與 `speed`（吞吐量）：

board/deploy.sh @ v1.0.0 (3bdd485) : 193--198

```
193 verify)
194     for pl in 1 0; do
195         echo "===== FUSE_HEAD_PL=$pl ====="
196         T=900 bssh "cd /home/root && FUSE_HEAD_PL=$pl ./mbv2_fuse_test . 2>&1 | grep -aE
    'fixture|BIT-EXACT|FAIL'"
197     done
198     ;;
```

只檢查「算得對不對」。沒有錯誤路徑可以檢查，因為 UIO 版本根本不拒絕任何東西。

用了什麼技術

- bit-exact gate：4 張 fixture 的輸出和 golden 逐位元比對，head 在 PL 和在 CPU 兩種擺法都跑。公開 repo 用的是合成的 fixture（`models/mbv2_pose/fixtures_synthetic/`，第 14 章），MPII 的 fixture 只在作者的板子上跑。
- `twn_errtest`：14 項檢查，其中 12 項故意做錯一件事、確認 driver 回傳預期的錯誤碼，另外 2 項確認正常的操作會成功；參數 `timeout` 加跑逾時，`hold` 只留映射。
- 卸載測試：裝置開著時、以及只剩映射時，`rmmod` 都必須失敗；之後連續 5 次卸載、重載，每次都跑 4 張 fixture。
- 吞吐量：pipeline 跑 120 幀、3 次，每次都檢查關節點和 fixture 0 相同。
- 靜態檢查：`make W=1`（kernel 額外的編譯器警告）與 `scripts/checkpatch.pl --strict`（kernel 的程式風格檢查）。

改善了什麼

發佈的 driver 上的檢查（沒有註明的都是開機組 H）	結果	紀錄
4 張合成 fixture，head 在 PL / 在 CPU	4/4、4/4	`H4`
4 張 MPII fixture，同上	4/4、4/4	`H4m`
錯誤路徑 14 項（12 項拒絕、2 項成功）	14/14 符合預期	`H4` 第 7–23 行
被拒絕的表之後，下一個 app 再跑	4/4、4/4	`H4` 第 26–27 行
裝置開著時 / 只剩映射時 `rmmod`	都被拒絕	`H4` 第 29–30 行
5 次卸載重載，每次 4 張 fixture	5 次都 4/4	`H4` 第 31–35 行
SMMU 關（開機組 I）：10 次各要 64 MB，總量超過 CMA	10/10	`I1`
SMMU 開：10 次各要 64 MB 的重載	10/10	`H10`
pipeline 吞吐量（3 次）	232.22 / 232.39 / 232.21 fps	`H4` 第 37–39 行
`W=1`、`checkpatch --strict`	0 個警告	`S6`

13.1 實際例子

fixture 檢查。每一種擺法跑一次 app，數有幾行 `PASS | final max|diff|=0`，要剛好 4 行：

linux/driver/twn_fuse/test/board_check.sh : 21--27

```
fixtures() {   # <app> -> number of fixtures that passed, for each head placement
    for pl in 1 0; do
        n=$(FUSE_HEAD_PL=$pl ./$1 . 2>&1 | grep -ac '^fixture .: PASS | final max|diff|=0')
        echo "  $1 FUSE_HEAD_PL=$pl: $n/4 fixtures bit-exact"
        [ "$n" = 4 ] || fail=1
    done
}
```

錯誤路徑測試的寫法。每一項都是「做一件事，看回傳值和錯誤碼對不對」：

linux/driver/twn_fuse/test/twn_errtest.c : 27--34

```
static void check(const char *what, int r, int e, int want)
{
    int ok = want ? (r < 0 && e == want) : (r >= 0);

    printf("%-4s %-52s -> %s\n", ok ? "PASS" : "FAIL", what,
           r < 0 ? strerror(e) : "ok");
    fails += !ok;
}
```

linux/driver/twn_fuse/test/twn_errtest.c : 36--36

```
#define CALL(what, expr, want) do { errno = 0; int r_ = (expr); check(what, r_, errno, want); } while
    (0)
```

踩過的坑

`twn_errtest` 第一次跑，兩項 `mmap` 檢查顯示 `FAIL ... -> Success`，看起來像是 driver 沒拒絕。其實錯在測試程式：當時 `mmap()` 寫在 `CALL` 外面，`CALL` 先把 `errno` 設成 0 才檢查，於是 `mmap` 留下的 `EINVAL` 被蓋掉，印出來的「Success」其實是 `strerror(0)`。`mmap` 本身是失敗的，driver 確實拒絕了。修法是把 `mmap()` 放進 `CALL` 裡，讓 `errno` 在呼叫完的下一刻就被讀取。教訓是：測試說「沒拒絕」時，先檢查測試本身有沒有讀對東西，不要直接去改受測的程式。

發佈映像上的一次 board check。中間的錯誤路徑與卸載已經在第 4 到 7 章逐段列出，這裡只印開頭與吞吐量：

results/raw/linux_driver_v11_20261027/H4_board_check.log : 1-6（開機組 H）

```
== 2026-10-26 20:39 UTC  6.1.30-xilinx-v2023.2  BOOT.BIN 5d226057efc8760b  twn_fuse.ko aec4477e2c520faf  backends: drv
== 1. fixtures, both backends
a0000000.twn_fuse -> twn_fuse
  mbv2_fuse_test_drv FUSE_HEAD_PL=1: 4/4 fixtures bit-exact
  mbv2_fuse_test_drv FUSE_HEAD_PL=0: 4/4 fixtures bit-exact
== 2. refusals and timeout
```

results/raw/linux_driver_v11_20261027/H4_board_check.log : 36-40（開機組 H）

```
== 4. pipeline throughput (120 frames, FUSE_BB2HD=1)
  drv run 1: === fused cross-frame pipeline: 120 frames in 516.8 ms = 4.31 ms/frame = 232.22 fps ===
  drv run 2: === fused cross-frame pipeline: 120 frames in 516.4 ms = 4.30 ms/frame = 232.39 fps ===
  drv run 3: === fused cross-frame pipeline: 120 frames in 516.8 ms = 4.31 ms/frame = 232.21 fps ===
BOARD CHECK: PASS
```

v1.1 新增機制的檢查。`v11_check.sh` 的每個模式只測一件事，每條指令先印成一行 `\# ...` 再印輸出，紀錄因此可以逐行引用：

模式	測什麼	紀錄與章節
`ab`	四組 A/B 的吞吐量（要 UIO，只能在 v1.0.0 的開機檔上跑）	`D1`，第 3 章
`bench`	ioctl 成本、backbone 時間拆解、CPU 存取 buffer 的頻寬	`D2`、`E3`，第 3、5、6 章
`split`	pipeline 各階段的時間，兩種映射	`D6`，第 3 章
`irqcount`	1000 幀對應 1000 次中斷，沒有空中斷	`E6`，第 6 章
`irqfps`	同一次開機，中斷與輪詢交替量吞吐量	`H7`，第 6 章
`debugfs`	debugfs 檔案、短幀、不合法的值、越界讀取	`D3`、`E4`，第 8、11 章
`synthetic`	合成 fixture，兩種 head 擺法	`D4`、`E5`、`H5`，第 9 章
`boot`	開機自動載入、中斷、`/dev/mem`、記憶體、密碼期限	`H3`，第 10、14 章
`devmem`	`/dev/mem` 還碰得到什麼（只讀）	`H8`，第 10 章
`smmu`	IOMMU group、fixture、吞吐量、越界讀取	`H6`，第 11 章
`state`	載入中的 driver：module 資訊、綁定、buffer、sysfs、映射、10 次重載	`H10`、`I1`，第 1、3、8 章

driver 版本和 UIO 版本比快慢。driver v1.0 剛完成時（開機組 A：v1.0.0 的開機檔，driver 靠輪詢），同一次開機、同一份 app 分別連結兩個 HAL，board check 的吞吐量是 UIO 230.04 fps、driver 231.87 fps：

results/raw/linux_driver_20261026/A2_board_check_uio_dtb.log : 40-45（開機組 A）

```
  uio run 1: === fused cross-frame pipeline: 120 frames in 521.4 ms = 4.35 ms/frame = 230.13 fps ===
  uio run 2: === fused cross-frame pipeline: 120 frames in 521.3 ms = 4.34 ms/frame = 230.22 fps ===
  uio run 3: === fused cross-frame pipeline: 120 frames in 522.2 ms = 4.35 ms/frame = 229.78 fps ===
  drv run 1: === fused cross-frame pipeline: 120 frames in 517.1 ms = 4.31 ms/frame = 232.05 fps ===
  drv run 2: === fused cross-frame pipeline: 120 frames in 517.0 ms = 4.31 ms/frame = 232.13 fps ===
  drv run 3: === fused cross-frame pipeline: 120 frames in 518.5 ms = 4.32 ms/frame = 231.44 fps ===
```

當時 driver 一起改了三件事（等硬體的地方、buffer 位置、CPU 端的記憶體屬性），只能說「沒有變慢」。v1.1 的 A/B 實驗把記憶體屬性單獨拆出來，差距來自它；另外兩件事是一起換的，合起來看不出影響（第 3 章）。中斷對吞吐量的影響另外在同一次開機上量（第 6 章）。

靜態檢查。審稿時重跑發現 `twn_fuse_ioctl.h` 有 1 個 WARNING（檔案裡寫了自己的檔名）與 1 個 CHECK（一行超過 100 欄），修掉之後，對著發佈映像那一顆 kernel 的建置樹重跑，兩個檔案都是 0：

results/raw/linux_driver_v11_20261027/S6_static_checks.log : 1-6,10-13,16-20,22,24,28,30（PC）：第 3–5 行是發佈 kernel 的選項，第 18–19 行證明 W=1 真的傳到了編譯器

```
== 2026-10-26 21:21 UTC  kernel source 6.1.30 of the v1.1 PetaLinux project (the release Image's build)  twn_fuse.c
     1006f944b08a55f7  twn_fuse_ioctl.h 680689f72675286d
# grep -E 'CONFIG_(ARM_SMMU|STRICT_DEVMEM|IO_STRICT_DEVMEM)=' <kernel build tree>/.config   (the release kernel's
     options)
CONFIG_ARM_SMMU=y
CONFIG_STRICT_DEVMEM=y
CONFIG_IO_STRICT_DEVMEM=y
# make clean; make W=1          (W=1: the kernel's extra warnings; output below is complete)
  CC [M]  <repo>/linux/driver/twn_fuse/twn_fuse.o
  MODPOST <repo>/linux/driver/twn_fuse/Module.symvers
  CC [M]  <repo>/linux/driver/twn_fuse/twn_fuse.mod.o
  LD [M]  <repo>/linux/driver/twn_fuse/twn_fuse.ko
exit 0
# make clean; make W=1 V=1 | grep -o -E -- '-W(missing-declarations|missing-prototypes)' | sort -u    (W=1 reached the
     compiler)
-Wmissing-declarations
-Wmissing-prototypes
# scripts/checkpatch.pl --no-tree --strict -f twn_fuse.c twn_fuse_ioctl.h
twn_fuse.c
total: 0 errors, 0 warnings, 0 checks, 692 lines checked
twn_fuse_ioctl.h
total: 0 errors, 0 warnings, 0 checks, 75 lines checked
```

13.2 沒有做到的兩項，以及怎麼補

`make dt_binding_check`。kernel 官方的 binding 檢查會用 JSON Schema 逐條驗證每個屬性的格式（例如 `reg` 的 cell 數），需要 Python 套件 `dtschema`（`yamllint` 沒裝時只會略過 YAML 的格式檢查）。這次的工作環境不允許從網路安裝新套件，所以沒有跑，只有 `binding_check.py` 的部分檢查（第 1 章）。補的方法：

```
pip install dtschema yamllint     # yamllint is optional
```

```
cp linux/driver/twn_fuse/wjhuang,twn-fuse.yaml <kernel>/Documentation/devicetree/bindings/misc/
make -C <kernel> dt_binding_check DT_SCHEMA_FILES=misc/wjhuang,twn-fuse.yaml
```

要送進 mainline 還得登記 vendor prefix（第 1 章），這次沒有做。

sparse（`make C=2`）。kernel 專用的靜態分析工具，最常抓到的是 `__user` 指標被直接取值、`__iomem` 指標沒經過 `readl()` 就使用（第 5 章的 `__user` 與第 2 章的 `__iomem` 就是給它看的標記）、大小端型態混用。同樣需要另外安裝，所以沒有跑。補的方法：`apt install sparse` 之後在 driver 資料夾執行 `make C=2`（`C=1` 只檢查這次重新編譯的檔案，driver 已經編過時要先 `make clean`）。

13.3 原理

每一項證明了什麼、沒證明什麼。

檢查	證明了	沒有證明
fixture bit-exact	driver 版本對這 4 張輸入算出和 golden 完全相同的結果，兩種 head 擺法都是	其他輸入。證明的是這顆 v1.1 bitstream 加上 v1.1 driver 的組合；bitstream 和 v1.0.0 不同（加中斷、改 AXPROT，第 12 章）
14 項錯誤路徑	每一條檢查規則至少在一種錯誤上有效，而且回報的是預期的錯誤碼	所有可能的錯誤組合
`rmmod` 被拒、5 次重載	module 的參考計數有效（開著與只剩映射兩種）；重新 probe 之後硬體仍正常	長時間反覆載入（5 次，加上 `state` 的 10 次）
10 次 64 MB（SMMU 關）	每次卸載都把 buffer 還給 CMA（第 3 章）	SMMU 開著時從一般記憶體配的頁有沒有歸還（不經過 CMA，640 MB 也小於可用記憶體，量不出來）
吞吐量	driver 版本沒有變慢；中斷比輪詢快約 0.8%（第 6 章）	長時間執行的穩定性
`W=1`、`checkpatch`	沒有編譯器警告與風格問題	sparse 的位址空間檢查、binding 的格式檢查

量測時不碰板子。吞吐量都是在同一條 ssh 指令裡連續跑完的，量測期間沒有其他連線去查板子的狀態，避免查詢本身干擾量測。

合成 fixture 與 MPII fixture。合成的 fixture 為什麼也能當驗證，見第 14 章；同一個映像上 MPII fixture 也全部通過（`H4m`），兩組結果一致。

13.4 面試會問

面試會問（完整版，約 1 分鐘）：你怎麼證明 driver 是對的，而且沒有讓系統變慢？

正確性：4 張 fixture，head 放在 PL 和放在 CPU 兩種擺法，都和 golden 逐位元相同。錯誤路徑：14 項測試，其中 12 項故意給越界 offset、壞指標、重複開啟、PL 會拒絕的表，都回傳預期的錯誤碼，逾時在 3.11 秒回來，之後下一個 app 照常跑。生命週期：裝置開著、或只剩映射時，`rmmod` 都被拒絕；連續 5 次卸載重載都 4/4；SMMU 關掉時連續 10 次各配 64 MB，總量

超過 CMA，證明 buffer 有歸還。效能：driver v1.0 剛完成時比 UIO 版本快約 0.8%，但同時改了三件事分不出原因，後來用四組 A/B 實驗把記憶體屬性單獨拆出來，差距來自它。靜態檢查做了 `W=1` 和 `checkpatch --strict`；官方的 `dt_binding_check` 與 sparse 因為環境不能安裝工具沒有跑，我會直接說出來，並說明怎麼補。

13.5 關鍵函式速查

名稱	位置	做什麼
`board_check.sh`	`linux/driver/twn_fuse/test/`	四類檢查一次跑完，最後印 PASS / FAIL
`fixtures()`	`board_check.sh`: 21	兩種 head 擺法各數 4 行 PASS
`CALL / check()`	`twn_errtest.c`: 36	呼叫後立刻讀 `errno`，比對預期錯誤碼
`v11_check.sh`	`linux/driver/twn_fuse/test/`	v1.1 新增機制的板上檢查，一個模式一件事
`bind.sh`	`linux/driver/twn_fuse/`	用 `driver_override` 在 UIO 與 driver 之間切換，也可帶 module 參數重新載入；UIO 版本的 app 只能在 v1.0.0 的開機檔上跑（要有保留區與 `/dev/mem`）

14 進映像與發佈：recipe、預設密碼、合成 fixture

v1.1 用 PetaLinux（Yocto）的 module recipe 把 `twn_fuse.ko` 編進映像、開機自動載入；發佈的 root file system 帶著公開、但建置時就設成過期的預設密碼，另附四張不含人物的合成 fixture 與板上工具包，拿到板子的人不需要 PetaLinux、交叉編譯器或 MPII 資料集就能驗證加速器。

原本怎麼做（v1.0.0 的映像與 driver v1.0）

v1.0.0 的 `system-user.dtsi` 把節點交給 UIO（第 1 章「原本怎麼做」）。driver v1.0 不在映像裡。每次開機要把 `twn_fuse.ko` 複製到板上，再用 `bind.sh` 透過 `driver_override` 把裝置從 UIO 搶過來；driver 是在 PC 上對著另外複製出來的 kernel 編譯樹編的，要靠 vermagic（module 裡記錄的 kernel 版本與主要設定字串，載入時必須和執行中的 kernel 完全相同）一致才能載入。發佈方面，v1.0.0 沒有附 root file system（rootfs，開機後掛成根目錄的整個檔案系統），因為建好的映像裡有板子的 root 密碼雜湊（`/etc/shadow`）；也沒有附 fixture，因為四張 fixture 都是 MPII（一個人體姿態資料集）的人物照片，不能再散布。想在板子上驗證，得自己建 rootfs、自己下載 MPII 重新產生 fixture、自己交叉編譯 app。

用了什麼技術

- **Yocto module recipe**（`.bb` 檔）：`inherit module` 讓這個 recipe 依賴映像自己的 kernel，用同一棵 kernel 樹編譯 out-of-tree module（原始碼不在 kernel 原始碼樹裡、另外編譯的 module）。`KERNEL_MODULE_AUTOLOAD` 產生開機載入清單，`IMAGE_INSTALL:append` 把套件放進 rootfs。

- **用標籤參照改寫節點**：`system-user.dtsi` 裡的 `&fuse_0 {...}` 覆寫 PetaLinux 產生的節點的 `compatible`（第 1 章）。這是建置時的覆寫，不是執行期才套上的 device tree overlay（`.dtbo`）。
- **公開的預設密碼，一開始就過期**：`build_linux.sh` 沒有給 `BOARD_PW` 時用 `twn-v1.1`，雜湊用 `openssl passwd -6`（SHA-512 crypt）算好，經 Yocto 的 `EXTRA_USERS_PARAMS` 寫進映像；同一個變數裡的 `passwd-expire root` 讓第一次登入一定要先改密碼。
- **合成 fixture**：用公式畫出四張圖，標準答案由與加速器逐位元相同的整數參考模型算出。
- **板上工具包**（`board/make_kit.sh`）：driver 版的 app、測試程式、權重、合成 fixture 打成一個 tar 檔，附 `SHA256SUMS`。

改善了什麼

- 開機到可以用不需要任何手動步驟：開機第 4.8 秒 driver 就載入並綁上（第 1 章的 `H3_boot_state.log`）；module 和 kernel 在同一次建置裡用同一棵樹編出來，vermagic 一定相符。
- repo 裡只有一份 driver 原始碼：`build_linux.sh` 建置時把它複製進 recipe。
- 開機即用：把開機檔放上 SD 卡、開機、登入並改密碼、解開工具包，`sh board_check.sh` 就能得到 `BOARD CHECK: PASS`（發佈的映像與工具包在開機組 H 驗過：`H0_kit_sha.log`、`H4_board_check.log`）。
- 發佈檔裡沒有個人資料：映像只有公開預設密碼的雜湊；fixture 不含任何人物；執行檔裡沒有建置機器的路徑。
- 預設密碼不能被留著用：序列埠或 ssh 的第一次登入都要先改密碼；改之前，用 ssh 直接執行指令、用 scp 傳檔都被拒絕。限制：rootfs 是 initramfs，重開機會回到過期的預設密碼。

14.1 實際例子：把 driver 編進映像

recipe。整個檔案只有十幾行。`LIC_FILES_CHKSUM` 的 md5 是 `twn_fuse.c` 第 1 行（SPDX 授權宣告）的 checksum，Yocto 用它確認授權宣告沒有被改；`RPROVIDES` 讓套件也能用 Yocto 慣用的 `kernel-module-twn-fuse` 這個名字被依賴：

linux/project-spec/meta-user/recipes-modules/twn-fuse/twn-fuse.bb : 1--18

```
SUMMARY = "Linux platform driver for the ternary MobileNetV2 accelerator (twn_fuse)"
DESCRIPTION = "linux/driver/twn_fuse of the repository, built as a kernel module and loaded at boot"
LICENSE = "GPL-2.0-only | MIT"
LIC_FILES_CHKSUM = "file://twn_fuse.c;beginline=1;endline=1;md5=107a173833ee0b60d07de595965ecd52"

inherit module

# twn_fuse.c and twn_fuse_ioctl.h are copied from linux/driver/twn_fuse/ by linux/build_linux.sh,
# so the repository keeps one copy of the source; Makefile is the module-class one in files/.
SRC_URI = "file://Makefile \
           file://twn_fuse.c \
           file://twn_fuse_ioctl.h \
```

```
          "
S = "${WORKDIR}"

RPROVIDES:${PN} += "kernel-module-twn-fuse"
# loaded at boot; with the v1.1 device tree it binds the node by its compatible string
KERNEL_MODULE_AUTOLOAD += "twn_fuse"
```

給 module class 用的 Makefile。KERNEL_SRC 由 inherit module 傳進來，指向映像的 kernel 樹：

linux/project-spec/meta-user/recipes-modules/twn-fuse/files/Makefile : 3--11

```
obj-m := twn_fuse.o

SRC := $(shell pwd)

all:
    $(MAKE) -C $(KERNEL_SRC) M=$(SRC)

modules_install:
    $(MAKE) -C $(KERNEL_SRC) M=$(SRC) modules_install
```

放進映像、只保留一份原始碼。

linux/project-spec/meta-user/conf/petalinuxbsp.conf : 6--7

```
# v1.1: the accelerator's kernel driver, loaded at boot (recipes-modules/twn-fuse)
IMAGE_INSTALL:append = " twn-fuse"
```

linux/build_linux.sh : 104--107

```
# the kernel driver's recipe builds the repository's own source (one copy, in linux/driver/twn_fuse)
for f in twn_fuse.c twn_fuse_ioctl.h; do
    install -D -m 0644 "$HERE/driver/twn_fuse/$f"
    "project-spec/meta-user/recipes-modules/twn-fuse/files/$f"
done
```

開機後的狀態（module 已載入、是映像裡的檔案、在開機載入清單裡、裝置綁在 twn_fuse 上）見第 1 章的 H3_boot_state.log 第 4–13 行。

14.2 實際例子：預設密碼

密碼本身不進 repo，只把雜湊與過期設定寫進 PetaLinux 專案自己的設定檔：

linux/build_linux.sh : 134--146

```
# root password: kept out of the repo, written only into the project's own conf
BSP=project-spec/meta-user/conf/petalinuxbsp.conf
sed -i '/^# BOARD_PW begin/,/^# BOARD_PW end/d' "$BSP"
EXPIRE=
if [ -z "${BOARD_PW:-}" ]; then
    BOARD_PW=twn-v1.1; EXPIRE=" passwd-expire root;"
    echo "NOTE: BOARD_PW not set: published default password '$BOARD_PW', expired (the first login must change it)"
fi
case "$BOARD_PW" in *\'*|*\"*|*\\*|*\;*|*\$*) die "BOARD_PW must not contain ' \" \\ ; or \$";; esac
{ echo "# BOARD_PW begin"
  # usermod -p takes a crypt hash; the $ signs are escaped for the extrausers class
  echo "EXTRA_USERS_PARAMS:append = \" usermod -p '$(openssl passwd -6 "$BOARD_PW" | sed 's/\$/\\$/g')' root;$EXPIRE\""
  echo "# BOARD_PW end"; } >> "$BSP"
```

板上：ssh 遇到過期的密碼（開機組 H，在任何人改密碼之前）。從 PC 用 OpenSSH 試三種情形：直接執行指令、用 scp 傳檔、密碼打錯；再用 expect（照腳本對互動程式打字的工具）模擬一次互動登入：

results/raw/linux_driver_v11_20261027/H1_ssh_expired.log : 1-13,17-25（開機組 H）

```
== 2026-10-26 20:36 UTC  from the PC (OpenSSH 8.9p1 Ubuntu-3ubuntu0.18) to the board, boot H, before any password change
# sshpass -p twn-v1.1 ssh -o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null -o LogLevel=ERROR -o PubkeyAuthentication=no -o PreferredAuthentications=password root@192.168.10.2 'echo a command ran on the board'
WARNING: Your password has expired.
Password change required but no TTY available.
exit 1
# echo test > /tmp/h1_probe.txt; sshpass -p twn-v1.1 scp -o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null -o LogLevel=ERROR -o PubkeyAuthentication=no -o PreferredAuthentications=password /tmp/h1_probe.txt root@192.168.10.2:/tmp/
WARNING: Your password has expired.
Password change required but no TTY available.
lost connection
exit 1
# sshpass -p wrong-password ssh -o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null -o LogLevel=ERROR -o PubkeyAuthentication=no -o PreferredAuthentications=password root@192.168.10.2 'echo a command ran on the board'
Permission denied, please try again.
exit 5

root@192.168.10.2's password:
You are required to change your password immediately (administrator enforced).
Last login: Mon Oct 26 20:37:50 2026 from 192.168.10.163
Changing password for root.
Current password:
New password:
Retype new password:
(still open after 15 s, no further output)
exit 0
```

密碼正確時，sshd 先說密碼已過期；沒有終端機（直接執行指令與 scp）就沒辦法改，連線直接結

束，指令沒有執行（第 3–10 行）。密碼打錯時是一般的拒絕（第 12 行）。互動登入（第 17–23 行，同一次開機的第二次執行；第 14–16 行的說明與指令沒有印出）先要求改密碼；在 `New password:` 按 Ctrl-C 只結束了這一次輸入，接著問確認密碼，連線停在這裡，沒有拿到 shell。

用 ssh 完成第一次登入（開機組 H，重開機之後）。同一個腳本給了新密碼時，最後用互動的 ssh 把密碼改掉，接著用新密碼直接執行指令：

`results/raw/linux_driver_v11_20261027/H9_ssh_first_login.log : 1,24-39`（開機組 H）：前面四種情形與上面相同，這裡只印最後兩段

```
== 2026-10-26 21:19 UTC  from the PC (OpenSSH 8.9p1 Ubuntu-3ubuntu0.18) to the board, before any password change
# ssh -tt root@192.168.10.2 (typed by expect): the default password twice, then a new password twice (not echoed)
spawn ssh -tt -o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null -o LogLevel=ERROR -o PubkeyAuthentication=no
     -o PreferredAuthentications=password root@192.168.10.2
root@192.168.10.2's password:
You are required to change your password immediately (administrator enforced).
Last login: Mon Oct 26 21:19:51 2026 from 192.168.10.163
Changing password for root.
Current password:
New password:
Retype new password:

(no shell and no close after 30 s)
exit 0
# sshpass -p '<new password>' ssh -o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null -o LogLevel=ERROR -o
     PubkeyAuthentication=no -o PreferredAuthentications=password root@192.168.10.2 'echo a command ran on the board;
     cut -d: -f1,3 /etc/shadow | grep ^root:'
a command ran on the board
root:20752
exit 0
```

改完之後這個 ssh 連線 30 秒內沒有再輸出任何東西（第 33–34 行），但第 37–38 行用新密碼直接執行的指令成功了，root 的「上次修改日」也變成當天，表示密碼確實在 ssh 上改掉了。實際使用時，改完若畫面停住，就關掉這個連線（先按 Enter，再輸入 `~.`），用新密碼重新登入。

板上：序列埠上的第一次登入（開機組 H）。第 2 行列出實際按的鍵（板子不回顯密碼）。第一次登入在第一個 `New password:` 按 Ctrl-C；第二次先給空的新密碼，再給兩個不同的密碼：

results/raw/linux_driver_v11_20261027/H2_first_login.log : 1-9,11-21（開機組 H）

```
1  == 2026-10-26 20:38 UTC  serial console (COM14, 115200) of the board, boot H (the root password expired at build);
       control characters removed
2  # keys typed, in order (the board echoes no password): Enter; root; the default password; the default password
       (current password); Ctrl-C; root; the default password; the default password (current password); Enter (an empty
       new password); Enter (the empty password again); the new password; a different password; root; the default
       password; the default password (current password); the new password; the new password again; a command; exit
3  ternposev11 login: root
4  Password:
5  You are required to change your password immediately (administrator enforced).
6  Changing password for root.
7  Current password:
8  New password:
9  PetaLinux 2023.2+release-S10121051 ternposev11 ttyPS0
11 ternposev11 login: root
12 Password:
13 You are required to change your password immediately (administrator enforced).
14 Changing password for root.
15 Current password:
16 New password:
17 Retype new password:
18 No password has been supplied.
19 New password:
20 Retype new password:
21 Sorry, passwords do not match.
```

Ctrl-C 讓 `login` 整個結束，回到登入提示（第 9 行是重新出現的登入橫幅），沒有拿到 shell。空的新密碼被拒絕（第 18 行），兩次不同也被拒絕（第 21 行），兩種情形都沒有 shell。第三次才正確改掉：

results/raw/linux_driver_v11_20261027/H2_first_login.log : 26-33,37-41（開機組 H）

```
26 ternposev11 login: root
27 Password:
28 You are required to change your password immediately (administrator enforced).
29 Changing password for root.
30 Current password:
31 New password:
32 Retype new password:
33 Last login: Mon Oct 26 20:38:20 UTC 2026 from 192.168.10.163 on pts/0
37 root@ternposev11:~# cut -d: -f1,3 /etc/shadow | grep -E '^(root|petalinux):'
38 root:20752
39 petalinux:0
40 root@ternposev11:~# exit
41 logout
```

改完之後 root 的「上次修改日」變成 20752（1970 年起的天數，也就是 2026-10-26）；`petalinux` 仍是 0，它的第一次登入同樣要先設密碼。

踩過的坑

候選版 rootfs 的強制改密碼有漏洞。候選版 rootfs（開機組 E）不讓密碼過期，而是在 `/etc/profile.d/` 放一個腳本：互動式 root 登入時先執行 `passwd`，改成功前不給 shell（`E7_first_login.log`）。審稿時發現它只攔得住「互動的登入 shell」：`ssh root@板子指令`、`scp` 都不經過

> `profile.d`，拿公開的預設密碼就能以 root 執行任何指令。開機組 E 的 E1–E6 本身就是在第一次互動登入（E7）之前、用預設密碼經 ssh 跑的。改成讓密碼過期之後，檢查發生在 PAM 的帳號檢查，所有登入方式都要經過它，腳本就拿掉了。

PetaLinux 自己的 `petalinux` 帳號。映像保留了 PetaLinux 預設建立的這個帳號：密碼是空的、而且已經過期，可以用 `sudo` 執行任何指令，登入的 PAM 設定允許空密碼（`common-auth` 的 `nullok_secure`；這個映像的 `pam_unix` 沒有 Debian 那個只限 `/etc/securetty` 終端機的修補，等同 `nullok`）。由設定推得（沒有在板上測）：能接上序列埠的人可以用空密碼登入 `petalinux`、被要求設一個新密碼，再用 `sudo` 取得 root，不需要 root 的密碼。sshd 預設 `PermitEmptyPasswords no`，所以這只影響序列埠。能接序列埠通常也能直接換 SD 卡，所以這次只在發佈說明與這裡寫明，沒有鎖掉這個帳號；要鎖，可以在 `EXTRA_USERS_PARAMS` 加 `usermod -L petalinux;`。

14.3 實際例子：合成 fixture 與工具包

四張圖完全由公式產生（漸層、棋盤加圓、條紋、平滑雜訊）：

tools/make_synthetic_fixtures.py : 31--55

```
def images():
    """Four int8 CHW images, 3 x 256 x 256, built from formulas only."""
    y, x = np.mgrid[0:256, 0:256].astype(np.float64) / 255.0
    rng = np.random.default_rng(20261027)
    out = []
    # 0: smooth colour gradients
    out.append(np.stack([x, y, 1.0 - (x + y) / 2]))
    # 1: checkerboard with three discs
    cb = ((np.floor(x * 8) + np.floor(y * 8)) % 2) * 0.6 + 0.2
    disc = np.zeros_like(x)
    for cx, cy, r in ((0.3, 0.3, 0.15), (0.7, 0.4, 0.2), (0.5, 0.75, 0.12)):
        disc = np.maximum(disc, ((x - cx) ** 2 + (y - cy) ** 2 < r * r).astype(np.float64))
    out.append(np.stack([cb, np.clip(cb + disc * 0.5, 0, 1), disc]))
    # 2: stripes at three angles and frequencies
    out.append(np.stack([0.5 + 0.5 * np.sin(2 * np.pi * (6 * x + 2 * y)),
                         0.5 + 0.5 * np.sin(2 * np.pi * (3 * x - 5 * y)),
                         0.5 + 0.5 * np.sin(2 * np.pi * 9 * y)]))
    # 3: smoothed noise (box blur, 9 x 9)
    n = rng.random((3, 256 + 8, 256 + 8))
    c = np.cumsum(np.cumsum(n, axis=1), axis=2)
    c = np.pad(c, ((0, 0), (1, 0), (1, 0)))
    blur = (c[:, 9:, 9:] - c[:, :-9, 9:] - c[:, 9:, :-9] + c[:, :-9, :-9]) / 81.0
    blur = (blur - blur.min()) / (blur.max() - blur.min())
    out.append(blur)
    return [np.clip(np.round(im * (HI - LO) + LO), LO, HI).astype(np.int8) for im in out]
```

腳本若找得到 MPII 的 fixture，會先用同一個參考模型重算第 0 張 MPII fixture，要求結果和既有的標準答案完全相同，確認這個參考模型就是板子拿來比對的那一個，再產生合成的標準答案。四張合成 fixture 在開機組 D、E、H，兩種 head 位置都逐位元相同（`D4`、`E5`、`H5_synthetic.log`）。

踩過的坑

工具包裡的建置路徑。發佈前掃描發佈檔時，發現工具包裡兩個 app 的執行檔帶著建置機器上的路徑：它們沒有 strip，debug 資訊裡記著編譯器、SDK 與原始碼的絕對路徑。發佈的其他檔案（BOOT.BIN、bitstream、XSA、kernel、兩個 dtb、boot.scr、rootfs 的 2,513 個檔案、權重與模型檔）都沒有這個問題：`tools/scan_release_paths.py` 對發佈檔逐一掃描，找到 0 處，同一個工具對 strip 之前的 app 找到 8 處（`S7_release_path_scan.log`）。修法是 `board/make_kit.sh` 用 SDK 的 `strip` 去掉符號與 debug 資訊，strip 之後、打包之前再掃一次工具包的每個檔案，含有建置路徑就中止。去掉 debug 資訊不改變程式碼，但檔案的雜湊變了，所以工具包重新在板上驗過（開機組 E 的 `E9_release_kit.log`，發佈的版本在開機組 H）。

`make_kit.sh` 的 strip 與打包前的掃描：

board/make_kit.sh : 45--49

```
45 ${STRIP:-aarch64-xilinx-linux-strip} --strip-all "$K/mbv2_fuse_test_drv" "$K/mbv2_fuse_pipe_drv" \
46     "$K/twn_bench" "$K/twn_errtest"
47 if grep -r -l -a -E '/home/[a-z]|/mnt/[a-z]/|[A-Z]:[/\\]' "$K"; then
48     echo "ABORT: the files above still contain a build path"; rm -rf "$STAGE"; exit 1
49 fi
```

14.4 原理

開機時 module 是怎麼被載入的。有兩條路，這個映像兩條都有：

- **依 modalias**：kernel 建立平台裝置時就有一個 uevent，帶著 `MODALIAS`（內容是 `of:N...T...C` 加上節點的 `compatible`）。那時開機還很早，udev 還沒啟動，沒有人收；udev 啟動後把既有裝置的事件重送一次（coldplug），才用內建的 libkmod 載入它（效果等同執行 `modprobe`），也就是在 `modules.alias` 找到這個字串屬於 `twn_fuse`（來自第 1 章的 `MODULE_DEVICE_TABLE(of, ...)`）。
- **依清單**：systemd（這個映像的 init 系統）的 `systemd-modules-load` 讀 `/etc/modules-load.d/*.conf`，逐一載入。

不論哪一條先載入，第二條都只會發現它已經載入。載入後，driver core 用 `compatible` 比對，呼叫 `probe()`。`driver_override` 是在 `compatible` 對不上時強迫綁定的手段，節點已經是 driver 認得的字串，就不再需要；`bind.sh` 仍然保留：它用 `driver_override` 在 UIO 與 driver 之間切換（第 13 章的卸載重載測試就用它），也可以帶 module 參數重新載入。

`inherit module` 做了什麼。讓 recipe 依賴 `virtual/kernel`，編譯時把 `KERNEL_SRC` 指向這次建置的 kernel 樹，安裝時把 `.ko` 放進 `/lib/modules/<版本>/extra/`，並在映像建立時跑 `depmod`：它掃描所有 module，產生 `modules.alias` 這張「modalias 字串 → module 名稱」的對照表。

過期的密碼怎麼擋住所有登入方式。`/etc/shadow` 每一列的第 3 欄是上次修改密碼的日期（1970 年起的天數），0 的意思是「必須在下次登入時修改」。序列埠的 `login` 與 sshd 都用 PAM 檢查帳號，`pam_unix` 看到 0 就回報「需要新密碼」：`login` 接著要求改密碼，不改就結束；sshd 在有終端機時同樣要求改，沒有終端機（直接執行指令、scp、sftp）就印出 `Password change required but no TTY available` 並結束連線。檢查發生在登入的那一刻，和之後執行的是哪一種 shell 無關，這是它比

`profile.d` 腳本可靠的原因。為什麼不放真正的密碼：雜湊可以離線暴力破解，而且同一份映像會被很多人使用，等於共用一組密碼；公開預設密碼加上第一次登入強制修改，安全性來自「第一次登入的人一定要改掉」，所以第一次開機要在自己控制的網路上。

initramfs 的副作用。這個映像的 root file system 是開機時載入記憶體的 initramfs，改的密碼只存在記憶體裡，重開機就回到過期的預設密碼。要保存，得把 root file system 放到 SD 卡的分割區上；這次沒有做。

為什麼合成圖也能當驗證。驗證的目標是「PL 的計算和參考模型逐位元相同」，不是「姿態估得準不準」。任何輸入都會走過全部 65 個 operator，只要每個 operator 的輸出逐位元相同，用什麼圖都可以。四張圖分別有平滑區、尖銳邊緣、週期圖樣和雜訊，讓數值範圍盡量分散。

14.5 面試會問

面試會問（完整版，約 1 分鐘）：driver 怎麼做到開機就自動載入？

我寫了一個 Yocto 的 module recipe：`inherit module` 讓它用映像自己的 kernel 樹編譯，所以 vermagic 一定對；`KERNEL_MODULE_AUTOLOAD` 產生 `/etc/modules-load.d` 的設定，開機由 systemd 載入；`IMAGE_INSTALL` 把它放進 rootfs。device tree 在 `system-user.dtsi` 把 PetaLinux 產生的節點的 `compatible` 改成 driver 比對表裡的字串，所以載入後 driver core 直接比對成功、呼叫 probe，不再需要 `driver_override`。另外 udev 也會用 modalias 載入它，兩條路都在。板上開機第 4.8 秒 driver 就綁好，用中斷模式。

面試會問（完整版，約 1 分鐘）：你的發佈映像怎麼處理 root 密碼？

映像裡不放任何真正的密碼。建置腳本沒有給密碼時用公開的預設密碼，雜湊用 `openssl passwd -6` 算好、經 `EXTRA_USERS_PARAMS` 寫進去，同一行用 `passwd-expire` 讓它一開始就過期。序列埠和 ssh 的第一次登入都會先要求改密碼；改掉之前，ssh 帶指令與 scp 都回「Password change required but no TTY available」；序列埠上按 Ctrl-C 回到登入提示，空密碼和兩次不一致都被拒絕。候選版我用 `profile.d` 腳本，審稿時發現它擋不住非互動的 ssh，才改成讓密碼過期。限制是 rootfs 是 initramfs，重開機會回到過期的預設密碼。

14.6 關鍵函式速查

名稱	位置（`meta-user/` 指 `linux/project-spec/meta-user/`）	做什麼
`twn-fuse.bb`	`meta-user/recipes-modules/twn-fuse/twn-fuse.bb`: 1	module recipe、開機載入
`IMAGE_INSTALL`	`meta-user/conf/petalinuxbsp.conf`: 7	放進 rootfs
複製原始碼	`linux/build_linux.sh`: 105	recipe 用 repo 裡唯一的一份
預設密碼與過期	`linux/build_linux.sh`: 139	沒給 `BOARD_PW` 時寫入雜湊與 `passwd-expire`
`images()`	`tools/make_synthetic_fixtures.py`: 31	四張合成圖
`make_kit.sh`	`board/make_kit.sh`: 45	strip、掃描建置路徑、打包工具包

A 名詞表

依英文字母排序（不分大小寫）。「章」是第一次詳細說明的地方，「導」是導讀，「C」是附錄 C。

名詞	一句話說明	章
A/B 實驗	兩組設定只差一件事，用來確定是哪一件事造成差異	3
ABI	給使用者程式的介面約定；改了就會讓舊程式壞掉	8
`access_ok`	檢查一段位址是否完全落在 user space 範圍內	5
arena、buffer	PL 讀寫的那塊 DDR；v1.0.0 是 256 MB 保留區（arena），driver 版本是 8 MB DMA buffer	導
AXI、beat	ARM 的晶片內部匯流排 / 它一次傳輸的資料量，這個設計是 16 bytes	導
AxPROT	AXI 交易附帶的 3 個保護位元；bit 1 是 0 表示 Secure、1 表示 Non-secure	11
barrier	限制 CPU 記憶體存取先後順序的指令	2
binding	device tree 節點的規格檔，寫明可以有哪些屬性	1
bit-exact	逐位元相同：輸出和標準答案的每個 byte 都一樣	導
block RAM tile	FPGA 晶片上固定大小的記憶體區塊，計算 block RAM 用量的單位	12
`boot.scr`、bootargs	U-Boot 開機時執行的指令稿 / U-Boot 傳給 kernel 的命令列參數	C
bypass	SMMU 對沒有登記的 stream ID 不做轉換、直接放行；發佈的 kernel 是這樣設定	11
CCI	Cache Coherent Interconnect，會去查 CPU cache 的互連，HPC 埠經過它	3
checkpatch	kernel 的程式風格檢查腳本 `scripts/checkpatch.pl`	13
CLB	configurable logic block，FPGA 裡放 LUT 與暫存器的基本區塊	12
CMA	Contiguous Memory Allocator，開機劃出、平時借給可搬移用途的連續記憶體區	3
coherent（一致）	CPU 和裝置看到的記憶體內容保證相同	3
`compatible`	device tree 節點上的硬體名字，格式「製造商,型號」	1
completion	kernel 的同步機制：一方等待，另一方（常是 ISR）呼叫 `complete()` 叫醒它	6
`container_of`	由 struct 中某個欄位的位址，算出整個 struct 的位址	7
context fault	交易在 SMMU 轉換時找不到對應的頁，被擋下並回報	11
`copy_from_user`	安全地從 user 記憶體複製到 kernel，失敗時回傳未複製的 byte 數	5
CSR	Control/Status Registers，加速器的控制暫存器	導
debugfs	只給開發者用的 kernel 檔案系統，掛在 `/sys/kernel/debug`	8
depmod	掃描所有 module、產生 `modules.alias` 等對照表的工具	14
descriptor table	描述子表：17 個 block 的設定，經 driver 寫進 CSR，PL 依表執行	導
device tree	描述硬體有哪些周邊、位址在哪的資料結構，開機時交給 kernel	1
Device 記憶體	給周邊用的記憶體屬性：不走 cache、不預讀；nGnRnE 表示不合併、不重排、寫入要等終點回應	2
`devm_*`	device-managed 資源，driver 離開或 probe 失敗時自動釋放	1
directive	同一個 Vivado 指令的不同最佳化模式	12
`dma_addr_t`	裝置看到的位址；沒有 IOMMU 時等於實體位址，有 SMMU 時是 IOVA	3
DMA、DMA mask	裝置不經 CPU 直接讀寫記憶體 / 裝置能定址的位元數，這裡是 40	3
`dmb` / `dsb`	兩種 barrier：`dmb` 只排順序，`dsb` 要等存取真的完成	2
`driver_override`	sysfs 檔案，強制指定裝置只能交給哪個 driver	1
DSP	FPGA 晶片上的硬體乘加單元	12
dt-schema	用 JSON Schema 寫 device tree binding 的格式與檢查工具	1
ECO	engineering change order，在已經 route 好的設計上只改一小部分	12
`-EPROBE_DEFER`	probe 的回傳值，表示依賴的東西還沒準備好，kernel 稍後會再呼叫一次 probe	6
errno	系統呼叫失敗時的錯誤碼，例如 `EINVAL`、`EFAULT`	5
exception table	kernel 記錄「這些指令出錯時跳到哪裡修復」的表	5
fixture	測試輸入加上它的標準答案（golden）	導
GIC、SPI	ARM 的中斷控制器 / 所有 CPU 共用的外部中斷，GIC 編號是 SPI 編號加 32	6
HAL	Hardware Abstraction Layer，上層只依賴介面，底下可換實作	9

名詞	一句話說明	章
HP / HPC 埠	PL 存取 DDR 的 AXI 埠；HP 不經 CPU cache，HPC 可以經 CCI 保持一致	3
hrtimer	kernel 的高精度計時器，`usleep_range` 靠它叫醒	6
initramfs	開機時整個載入記憶體的 root file system，重開機就回到原樣	14
interrupt context	CPU 被中斷打斷、還沒回到任何 process 時執行的程式，不能睡眠	6
interrupt storm	level 中斷的來源沒清掉，ISR 被一直重複呼叫	6
ioctl	對裝置檔下指令的系統呼叫	5
IOMMU group / domain	group 是必須共用同一套轉換的一組裝置；domain 是一套轉換表	11
IOMMU、SMMU	替裝置做位址轉換與存取檢查的硬體；ARM 上叫 SMMU，ZynqMP 用 MMU-500	11
`ioremap`	在 kernel 頁表建立對周邊實體位址的映射	2
`IO_STRICT_DEVMEM`	kernel 選項：`/dev/mem` 也不能存取被 driver 佔用中的 I/O 區間	10
IOVA	I/O 虛擬位址：經過 SMMU 時，裝置看到的位址	11
ISR	Interrupt Service Routine，中斷發生時 kernel 呼叫的 driver 函式	6
jiffy	kernel 的時間刻度，`HZ=250` 時是 4 ms	6
level / edge	中斷的兩種觸發方式：線保持高電位的期間都算 / 只在變化的瞬間算一次	6
linear map（線性映射）	kernel 頁表裡把整個 DDR 一對一對應進來的那一段	3
LUT	look-up table，FPGA 裡實作組合邏輯的基本單元	12
mainline	Linus Torvalds 維護的官方 kernel 原始碼樹	1
misc device	主裝置號 10、次裝置號自動分配的字元裝置	5
`mmap`	把裝置或檔案的內容映射進 process 的位址空間	4
MMIO	Memory-Mapped I/O，用讀寫記憶體位址的方式操作暫存器	2
modalias	裝置的識別字串，udev 用它找出該載入哪個 module	14
module parameter	載入 module 時可以給的參數，例如 `use_irq=0`	1
MPII	一個人體姿態資料集；v1.0.0 的 fixture 是其中的照片，含人物，不能再散布	14
mutex、spinlock	拿不到就睡眠等待的鎖 / 拿不到就空轉等待的鎖，拿著時不能睡	7
NBLK	「這一幀跑幾個 block」的暫存器，平常是 17，reset 值也是 17	導
netlist	合成產生的電路元件與連線清單	12
Normal non-cacheable	一般記憶體屬性但不走 cache；寫入可以合併，可以非對齊存取	3
oops	kernel 程式出錯時印出錯誤現場並終止目前工作的機制	2
outer shareable	barrier 的範圍之一：CPU 與其他會存取記憶體的裝置	2
out-of-tree module	原始碼不在 kernel 原始碼樹裡、另外編譯的 module	14
page cache	kernel 快取檔案內容的記憶體	3
PAM	Linux 的登入驗證框架；`login` 與 sshd 都經過它檢查密碼與帳號狀態	14
PAN	Privileged Access Never，ARMv8.1 起的功能，禁止 kernel 直接存取 user 記憶體；A53 沒有	5
`passwd-expire`	Yocto `EXTRA_USERS_PARAMS` 的指令，把密碼設成已過期，下次登入必須先改	14
pblock	Vivado 的位置限制：把某些邏輯放在晶片的某一塊區域	12
PFN	page frame number，實體位址除以頁大小（4096）	10
phys_opt_design	Vivado 在 placement 或 route 之後，針對 timing 做的局部最佳化	12
pipeline	量吞吐量用的 app，讓 backbone 與 head 在不同幀上重疊	導
placement / route	決定每個邏輯單元放在晶片的哪個位置 / 決定它們之間怎麼繞線	12
platform device / driver	不能自動列舉的 SoC 周邊，和它們的 driver	1
PMU	Platform Management Unit，ZynqMP 的平台管理處理器；DDR 最上面 1 MB 保留給它	10
posted write	寫入送出後 CPU 就繼續執行，不等寫入真正到達	6
`probe()` / `remove()`	driver 接管、放開裝置時被呼叫的函式	1
PS / PL	Zynq 的 CPU 部分 / FPGA 邏輯部分	導
`readl_poll_timeout`	輪詢暫存器直到條件成立或逾時的巨集	6
recipe	Yocto 的建置描述檔（`.bb`），說明一個套件怎麼取得、編譯、安裝	14
retiming	把暫存器搬過組合邏輯，讓前後兩段路徑的延遲比較平均	12

名詞	一句話說明	章
ring overrun	PL 的列緩衝區在滿的時候被寫入；STATUS bit 16 記錄它	8
rootfs	root file system，開機後掛成根目錄的整個檔案系統	14
Secure / Non-secure	ARM TrustZone 的兩個世界；Linux 跑在 Non-secure	11
slack、WNS、WHS、TNS	路徑的時間餘裕；最差的 setup 餘裕 / 最差的 hold 餘裕 / 所有違規路徑的負餘裕加總	12
SoC	System on Chip，把 CPU 與周邊做在同一顆晶片上	1
sparse	kernel 專用的靜態分析工具，用 `make C=2`（或 `C=1`，只檢查重新編譯的檔案）執行	13
SPDX	用標準化的授權名稱寫在原始檔第一行的授權宣告	14
strategy	Vivado 預先組好的一整組 place & route 選項	12
stream ID	每筆交易進 SMMU 時帶的來源編號，SMMU 用它決定用哪一組轉換	11
streaming DMA	用 cacheable 記憶體，每次傳輸前後手動做 cache 維護	3
`STRICT_DEVMEM`	kernel 選項：`/dev/mem` 不能存取 System RAM	10
sysfs	把 kernel 物件攤成檔案樹的檔案系統，掛在 `/sys`	8
System RAM	kernel 管理的記憶體範圍（`/proc/iomem` 裡的同名項目）：device tree 的 `/memory` 節點，扣掉標成 `no-map` 的保留區	10
systemd	這個映像的 init 系統；`systemd-modules-load` 是其中載入 module 的服務	14
TBU	SMMU 的轉換單元，ZynqMP 有 6 個，各自負責一組匯流排埠	11
threaded IRQ	把中斷處理函式放到一個 kernel 執行緒裡、可以睡眠的處理方式	6
TLB	translation lookaside buffer，位址轉換結果的快取	11
TrustZone	ARM 把系統分成 Secure 與 Non-secure 兩個世界的硬體機制	11
uapi header	kernel 和 user space 共用的介面定義檔	5
U-Boot	開機載入程式：從 SD 卡讀 kernel、device tree 與 rootfs，再啟動 kernel	C
uevent / udev	kernel 通知 user space「有裝置出現」的訊息 / 收到後負責載入 module 的程式；udev 啟動後重送既有裝置的事件叫 coldplug	14
UIO	Userspace I/O，把裝置的暫存器開給 user space 的通用 driver	1
vendor prefix	`compatible` 字串逗號前面那一段，代表廠商或作者	1
vermagic	module 裡記錄的 kernel 版本與主要設定字串，載入時必須和執行中的 kernel 相同	14
VMA	`vm_area_struct`，描述 process 位址空間中一段連續範圍	4
W=1	kernel 編譯時打開額外警告的選項	13
write gathering	把連續的小寫入在 CPU 內合併成一筆大寫入	3
XSA	Vivado 匯出的硬體描述檔，PetaLinux 用它產生 device tree	6
Yocto	PetaLinux 底下用來建置整個 Linux 映像的系統	14

B 面試題：30 秒版本

每題先講結論，再講一個板上的數字或證據。完整版在括號中的章節。

1. compatible 字串怎麼讓 kernel 找到 driver、呼叫 probe ?(第 1 章)

開機時 device tree 的每個 PL 節點被建成 platform device。driver 的 `of_match_table` 放了 `"wjhuang,twn-fuse-1.1"` 與 `-1.0`，註冊時 platform bus 用 `platform_match()` 比對：有 `driver_override` 就只比 driver 名字，否則比 compatible。對上了就呼叫 `probe()`：映射暫存器、配 DMA buffer、申請中斷、建 `/dev/twn_fuse`，全部用 `devm_`。版本號讓舊 driver 不會去等新硬體才有的中斷。

2. 為什麼要 ioremap ? readl / writel 和 barrier 是什麼 ?(第 2 章)

開了 MMU 之後指標都是虛擬位址，kernel 頁表裡沒有周邊，直接存取會 page fault。`ioremap` 建一筆 Device 屬性的頁表：不走 cache、不合併、不重排。存取用 `readl / writel`：`writel` 寫入前有 `dma_wmb`，app 寫進 DDR 的輸入一定比 START 先讓 PL 看到；`readl` 之後有 `dma_rmb`，讀到 done 之後讀輸出不會被提前。

3. DMA buffer 怎麼配 ? 為什麼 driver 版本比較快 ?(第 3 章)

`dma_alloc_coherent` 同時給 CPU 位址和 DMA 位址；不用 `kmalloc`（cacheable、最大 4 MB、不給 DMA 位址），也不用 `/dev/mem`。HP 埠不查 CPU cache，kernel 把 buffer 映射成 non-cacheable。快的原因用四組 A/B 實驗把記憶體屬性單獨拆出來：「CPU 端屬性從 Device 改成 Normal non-cacheable」這一步有差，228.83 到 231.53 fps，因為寫入可以合併，寫輸入從 0.14–0.15 降到 0.09–0.10 ms（每次執行的最後一幀）。

4. app 為什麼只看得到 buffer ? 為什麼用 mmap 不用 read / write ?(第 4 章)

app 的頁表由 kernel 決定。driver 的 `.mmap` 只用 `dma_mmap_coherent` 映射那塊 buffer，檢查長度與 offset；暫存器只在 kernel 裡 ioremap。執行中的 app 的 `maps` 只有 8 MB 的 `/dev/twn_fuse`。每幀幾百 KB 的資料用 `read / write` 要多複製兩次，mmap 讓 app 和 PL 看同一塊記憶體。

5. ioctl 和其他介面怎麼選 ?(第 5 章)

依資料性質選：大量資料用 mmap 零複製；對裝置的動作和結構化參數用 ioctl，`_IOWR` 把方向和大小編進指令碼；給人看的狀態用 sysfs，除錯用 debugfs。所以 file operations 只有 open、release、mmap、unlocked_ioctl。一次 ioctl 1.62 µs，跑一幀的系統呼叫、複製加讀回 counter 合計 3.3 µs，介面不是瓶頸。

6. 為什麼用 copy_from_user？有了 SMMU 為什麼還只收 offset？(第 5、11 章)

user 指標可能沒有映射，或故意指向 kernel；`copy_from_user` 先用 `access_ok` 檢查範圍，出錯時由 exception table 接住，我回 `-EFAULT`；A53 沒有 PAN，硬體不會擋。只收 offset、檢查對齊與範圍 (含溢位) 再加上 DMA 位址，是第一道防線；SMMU 只擋得住帶著登記過的 stream ID 的 Non-secure 交易，也不管 buffer 內部寫到哪一區。14 項錯誤測試都符合預期。

7. 怎麼把輪詢改成中斷？(第 6 章)

RTL 加一個暫存器輸出的 level 中斷，等於致能 AND done，致能放在獨立暫存器，免得被 START 寫掉；IP 宣告成 interrupt 介面，接 `pl_ps_irq0`，device tree 是 SPI 89。driver 用 `devm_request_irq`，ISR 清 done、`complete()`，返回前讀回一次確保清除到達；`reinit_completion` 放在 START 之前。1000 幀剛好 1000 次中斷；只切換 `use_irq` 比較，每幀 CPU 時間從 465.8 降到 10.7 µs；另一次開機交替量三輪，吞吐量從 231.26 變成 233.13 fps。

8. 怎麼處理並行？為什麼用 mutex 不用 spinlock？(第 7 章)

兩層：`open` 用 `test_and_set_bit` 讓同一時間只有一次 open 成功，因為描述子表與 base 位址是整顆加速器共用的狀態；backbone 和 head 各一把 mutex，同一個引擎的暫存器操作不被插隊。拿著鎖時會睡在 completion 上好幾 ms，spinlock 不能睡。也拿掉了 sysfs 的 unbind，因為它會在 buffer 還被映射時釋放 buffer。

9. 狀態和除錯功能怎麼給？(第 8 章)

狀態放 sysfs：六個 cycle counter、STATUS、NBLK 各一個唯讀檔，交給 `.dev_groups`，driver core 在 probe 後建檔、remove 前移除；跑完一幀 `cycles_total` 約 99 萬 cycles。除錯放 debugfs，不當 ABI：`debugfs_create_regset32` 印暫存器，可寫的 `nblk` 檢查範圍、拿 backbone 的鎖，寫 99 回 `EINVAL`。

10. 為什麼限制 /dev/mem？(第 10 章)

它把整個實體位址開給 root。driver 用 DMA API 配 buffer、只 mmap buffer，app 不再需要 `/dev/mem`，256 MB 保留區也拿掉了。`STRICT_DEVMEM` 擋 System RAM，`IO_STRICT_DEVMEM` 擋 driver 佔用中的暫存器；板上 `devmem` 都回 `Operation not permitted`。邊界也講得出來：不是 System RAM、也沒有 driver 佔用的位址都還能映射，例如卸載 driver 後的暫存器、DDR 最上面 1 MB 的 PMU 保留區。

11. SMMU 怎麼讓 PL 只能碰自己的 buffer？(第 11 章)

device tree 打開 SMMU，給四個 HP 埠登記 stream ID；DMA API 自動把 buffer 映射到 IOVA，越界讀取變成 context fault。第一次失敗是因為 AXI master 送的是 Secure 交易，Linux 只設定 Non-secure 那一側；改成 AxPROT[1] = 1 才生效。另外三個埠的 ID 我推算錯了兩個，用 `disable_bypass` 讓 SMMU 回報才量出來。限制：發佈的 kernel 對沒登記的 stream ID 是 bypass。

12. 改了 RTL 之後 timing 收不了怎麼辦？(第 12 章)

先固定配方，只動 RTL。建置腳本在 route 後若 WNS 小於 0，自動跑 `phys_opt_design`，不行再重新 route。把 AxPROT[1] 從 0 改成 1，只改常數，route 完卻是 −0.032 ns，靠 route 後的 `phys_opt_design` (strategy 內建一次、建置腳本再兩次) 補到 0。備案是確認 Vivado 對同樣的輸入給出同樣的結果，重現上一版收斂的設計，再用 ECO 只改那幾根常數 pin；最後沒有用到。

13. 怎麼證明 driver 是對的？(第 13 章)

fixture 在兩種 head 擺法都和 golden 逐位元相同；14 項錯誤路徑回傳預期的錯誤碼，逾時 3.11 秒回來；裝置開著或只剩映射時 `rmmod` 被拒，5 次重載都 4/4；SMMU 關掉時 10 次各配 64 MB、總量超過 CMA，證明 buffer 有歸還。`W=1` 與 `checkpatch --strict` 0 個警告；`dt_binding_check` 與 sparse 沒跑，我會說明原因和補法。

14. driver 怎麼進映像？發佈映像怎麼處理密碼？(第 14 章)

Yocto 的 module recipe：`inherit module` 用映像自己的 kernel 樹編，vermagic 一定對；`KERNEL_MODULE_AUTOLOAD` 讓 systemd 開機載入，udev 也會依 modalias 載入；device tree 的 `compatible` 改成 driver 的字串。密碼用公開的預設值，建置時就用 `passwd-expire` 讓它過期，所以第一次登入一定要改；改之前 ssh 帶指令與 scp 都被拒絕。fixture 換成公式產生的合成圖，發佈檔沒有個人資料。

C 如何重現

指令都在 repo 根目錄執行：Vivado 與 PC 上的模擬在 Windows，PetaLinux、driver 與 app 的編譯在 WSL，連板子用 Git Bash 的 `plink` / `pscp` (`board/deploy.sh`)。每一步後面的括號是它產生的紀錄檔。

1. RTL 的 gate 與 bitstream (Windows)

```
powershell -File rtl\fuse\sim\run_csr_irq.ps1      (S1: interrupt unit gate)
powershell -File rtl\fuse\sim\run_hd_int.ps1       (S2: head integration gate)
vivado\build.cmd                                    (S4: timing and utilization reports)
```

2. Linux 映像與開機檔 (WSL)

```
bash linux/build_linux.sh          # no BOARD_PW: the published default password, expired
bash board/make_boot.sh build/system_fuse_ternpose.bit build/BOOT.BIN
python3 linux/driver/twn_fuse/test/binding_check.py <kernel-source> build/linux/sd/system.dtb   (S3)
```

3. app、測試程式與板上工具包 (WSL)

```
bash board/build_apps.sh                       # HAL=drv (default): build/arm-drv
HAL=uio bash board/build_apps.sh               # the UIO apps, for boot D only
make -C linux/driver/twn_fuse prepare all test # twn_fuse.ko (boot D), twn_errtest, twn_bench
bash board/make_kit.sh                         # build/twn_board_kit.tar.gz
make -C linux/driver/twn_fuse clean; make -C linux/driver/twn_fuse W=1                  (S6)
<kernel-source>/scripts/checkpatch.pl --no-tree --strict -f twn_fuse.c twn_fuse_ioctl.h  (S6)
```

4. 開機組 D（v1.0.0 的開機檔，D1–D6）

把 v1.0.0 的五個開機檔放上 SD 卡開機，複製 `twn_fuse.ko`、兩版 app（改名為 `_uio`、`_drv`）、`bind.sh`、`twn_bench`、`ab_measure.sh`、`v11_check.sh` 與 `data/`、`golden/`（`syn/` 放合成 fixture），然後：

```
sh v11_check.sh ab          (D1)     sh v11_check.sh bench      (D2)
sh v11_check.sh debugfs     (D3)     sh v11_check.sh synthetic  (D4)
sh v11_check.sh boot        (D5)     sh v11_check.sh split      (D6)
```

5. 開機組 E 與 H（v1.1 映像，E0–E9、H0–H10）

現在的原始碼建出的是開機組 H 的 rootfs（root 密碼過期）；開機組 E 的 rootfs 是 commit `b11c465` 之前的版本，其他開機檔兩組相同。`BOARD_PW` 是板子目前的 root 密碼：

```
bash board/deploy.sh pin
bash board/deploy.sh pushlinux build/BOOT.BIN build/linux/sd
# right after the reboot, before anyone logs in, from a Linux host:
bash linux/driver/twn_fuse/test/pw_expired_ssh.sh                                          (H1)
bash linux/driver/twn_fuse/test/pw_expired_ssh.sh 192.168.10.2 <new password>              (H9, after a
    reboot)
python linux/driver/twn_fuse/test/first_login_serial.py COM14 <new password> H2.log       (H2)
export BOARD_PW=<new password>
# on the board: tar xzf twn_board_kit.tar.gz && cd twn_board_kit
sha256sum -c SHA256SUMS (E0, H0)   sh v11_check.sh boot (E1, H3)   sh board_check.sh (E2, H4)
sh v11_check.sh bench (E3)   sh v11_check.sh debugfs (E4)   sh v11_check.sh synthetic (E5, H5)
sh v11_check.sh irqcount (E6)   sh v11_check.sh smmu (E8, H6)   sh v11_check.sh irqfps (H7)
sh v11_check.sh devmem (H8, one of its reads is outside System RAM)
sh v11_check.sh state (H10)
```

E2m 與 H4m 是同一個 `board_check.sh`，換成四張 MPII fixture（不在工具包裡）。E7 是候選版 rootfs 在序列埠（115200 baud）上的互動登入：在第一個 `New password:` 提示按 Ctrl-C，再輸入空的新密碼與不同的確認密碼，最後設定新密碼。H1、H2 是用與上面兩個腳本相同的步驟產生的（兩個腳本是事後從當時的暫存腳本整理進 repo 的，紀錄第 1 行的說明文字略有不同）。S7 是在發佈檔的資料夾執行 `python tools/scan_release_paths.py`（找發佈檔裡的建置路徑）。S8 是第 1 步建置時 Vivado 的 block design 訊息：`grep -n "PSU-0" vivado/build_bd_fuse_v11.log`。

6. 開機組 F（找 stream ID，F0–F1）

開機組 F 的 `BOOT.BIN`（`18a066f87cfbad8d`）和開機組 E 的 `BOOT.BIN` 帶著同一顆 bitstream，但是另一次打包的檔案；kernel 與 rootfs 是第一次建出的 v1.1 版本。要看到 F0 的 global fault，`system.dtb` 只能登記當時推算的三個 ID（F0 用的 dtb 是 `4889daa18dd6a320`）；再把 `boot.scr` 換成 `bootargs`（U-Boot 傳給 kernel 的命令列）多一個 `arm_smmu.disable_bypass=1` 的版本：在 `linux/boot.cmd` 的 `bootargs` 加上這個參數，再用 U-Boot 的 `mkimage` 工具編成 `boot.scr`（指令與 `linux/build_linux.sh` 第 5 步相同）。這樣沒有登記的 stream ID 就會被 SMMU 擋下並回報：

```
ph=$(fdtget -t x system.dtb /axi/smmu@fd800000 phandle)       # on the host
cp system.dtb system_f0.dtb
fdtput -t x system_f0.dtb /amba_pl@0/twn_fuse@a0000000 iommus $ph e80 $ph 1280 $ph 1680
# boot with system_f0.dtb as system.dtb, then on the board:
dmesg | grep -o "GFSYNR1 0x[0-9a-f]*" | sort | uniq -c         (F0: the unlisted IDs)
# back on the host, the measured IDs; boot with system_smmu.dtb as system.dtb:
```

```
sh linux/driver/twn_fuse/smmu_dtb.sh system.dtb system_smmu.dtb
sh v11_check.sh smmu                                                (F1)
```

7. 開機組 G 與 I（關掉 SMMU 的比較，G1–G4、I1–I2）

```
cp system.dtb system_nosmmu.dtb       # on the host
fdtput -t s system_nosmmu.dtb /axi/smmu@fd800000 status disabled
fdtput -d system_nosmmu.dtb /amba_pl@0/twn_fuse@a0000000 iommus
# copy it over system.dtb on the card (keep the original), reboot, then as in step 5:
sh v11_check.sh boot (G1)   sh board_check.sh (G2)   sh v11_check.sh bench (G3)   sh v11_check.sh debugfs
    (G4)
# the same dtb with the released files (boot I):
sh v11_check.sh state (I1)   sh v11_check.sh boot (I2)
```

8. 開機組 A、B、C（driver v1.0）

這三組是 driver v1.0（commit 7615324）在 v1.0.0 開機檔上的紀錄，這份講義只用它們呈現 v1.0.0 的狀態，以及和 driver v1.0 比較。重現要在 commit 7615324 的 worktree 裡，用它的 driver、app（那時 HAL 預設是 uio）、board_check.sh 與 patch_dtb.sh，照那個 commit 的講義附錄 C（docs/zh-TW/driver-handout/sections/C-reproduce.tex）做。

9. 重建這份講義

```
cd docs/zh-TW/driver-handout
latexmk                       # runs extract.py first; output build/handout.pdf
```

板上紀錄各檔的用途見 results/raw/linux_driver_v11_20261027/README.md 與 results/raw/linux_driver_20261026/README.md；數字整理在 results/board_results.json 的 linux_driver_twn_fuse_v11_20261027 與 linux_driver_twn_fuse_20261026。

回退。開始前先 deploy.sh pin（保存目前的 BOOT.BIN）；pushlinux 會把被換掉的檔案保存在卡上的 previous_linux/。板子開不起來時，可以在序列埠攔下 U-Boot（開機載入程式，負責從 SD 卡讀 kernel、device tree 與 rootfs），用 fatload 與 fatwrite 把保存的檔案寫回。